观测语义约束下的啤酒风味预测、化合物筛选与条件生成

摘要

针对啤酒化学数据与感官评分的尺度混合、Top-3 共线冗余及小样本风格迁移问题，构建由观测语义、训练内选择和独立风险验证衔接的三问方案。将 2197 条原始品评记录映射到 250 款啤酒，定义评分员等权 0—6 面板目标，并保留评分卡层次和重复品评敏感性。评分卡交付为 250 行 ×53 列、无缺失值：1 列啤酒编号，50 列由冻结模型预测的感官评分，另 2 列 acetaldehyde 为阈值锚定推断值（无面板 0—6 记录，由附件 1 实测浓度经 log10 反变换后与附件 8 的 50.0mg/L 感官检出阈值比较，250 款酒全部低于阈值故取 0=Not present），后两列明确标注为非面板评分。该评分卡存在一项已量化并披露的局限：预测等级全部落在 0—4 级，真值 ≥5 的 121 个单元被系统低估（平均低估 2.07，连续预测上限仅 4.34），故整体 MAE 可用而高强度尾部不可用。冻结的 Extra Trees 取得 MAE 0.4454、RMSE 0.6305，层次网络未超过强传统基线；非香气强度项的平均 MAE（0.3486）优于香气强度项（0.5047）。化合物筛选在训练内部比较 Spearman、2+1、三步条件前向与冗余控制，最终选出 2+1 并统一 50×3 结果；包含方法选择的外层 MAE 为 0.5024，揭示小样本选法的方差代价。对四种既有生成模型完成 22 风格分布、分类一致性、相关结构及联合低端模式诊断：交付生成器沿用诊断之前冻结的验证折规则而非条件 SWD 排名，因为 CVAE 的表面 SWD 优势与测试集大小强相关（r=0.70，仅在 1—2 款测试酒的风格上领先，5 款以上时优势消失）。9 款独立源酒的四策略配对比较按两套缺陷口径报告：在赛题所指的超阈值口径上，硬约束把越阈次数由无约束的 7 次降到 0 次；在任意微量上升的严格口径上仍有 4—5/9 款酒上升，该残余风险列为未解决局限。结果支持保留拒绝选项、报告个体风险并区分编码约束与真实化学保证。三组封闭实验完成后停止模型探索，交付可复现代码、风格谱、候选因素和完整验证材料。本文为如实披露 AI 贡献的研究定稿，队员独立核心工作状态另列。

关键词：面板评分；训练内选择；替代组稳定性；条件生成；独立风险

AI 辅助研究参考稿。核心建模与分析尚待参赛队独立完成，不作为独立原创正式提交稿。

目录

1 问题分析与统一研究主线 **1**

2 符号、评价对象与可复现约定 **1**

3 数据审计与监督目标重建 **2**

4 问题一：真实面板均分与结构化监督 **4**

5 问题二：训练内选法与统一的 **50×3** 化合物结果 **7**

6 问题三之一：**22** 风格条件生成的分布检验 **11**

7 问题三之二：独立评价与拒绝迁移 **14**

8 创新动机、消融与模型评价 **16**

9 结论与建议 **18**

10 参考文献 **19**

11 **AI** 使用声明与独立工作状态 **19**

A 附录：统一 **50×3** 候选化合物 **20**

B 附录：**22** 风格独立测试覆盖 **21**

C 附录：逐风格四模型分布与联合低端结果 **22**

D 附录：**9** 酒四策略逐项结果 **24**

E 复现环境与全部新增程序 **25**

1　问题分析与统一研究主线

啤酒风味预测的困难首先来自观察尺度：化学列混合编码、评分卡含层次继承、同一评分员重复品评，而风格只提供小样本条件。若监督目标、关键因素与生成评价采用不同口径，复杂模型的表面提升可能无法解释。因此，围绕“观察了什么、训练可以使用什么、何种证据支持结论”组织三问。

原论文 [1] 结合化学特征、专业面板与消费者信息，并通过添加品评检验候选成分。赛题要求新增 0—6 分级、50 维 Top-3 和跨风格缺陷约束。作者公开的逐评分员数据 [2] 使真实面板尺度可用，但不能消除评分卡疑点或把附件 4 简单逆变换成原分。先恢复可识别目标，再评价候选解释与生成决策，是三问衔接的基础。

本研究保留有竞争力的传统基线，也实现评分机制驱动的层次网络与 CVAE。是否采用模型由冻结实验中的证据决定，不以结构复杂度作为创新标准。最终补充限定为 Q2 统一选择、22 风格生成诊断和 9 源酒独立迁移三组，全部完成后转入论文定稿。

主要假设为：匿名 beer_name 按附件物理行序对应；面板目标给予评分员等权；缺失化学测量以训练中位数进行工作性填补；小风格借用总体经验边际降低方差。每项假设均保留其可验证范围，尤其不把最低值全部视作已知检测限，也不把统计生成谱视作直接酿造配方。

2　符号、评价对象与可复现约定

设 $x_i \in \mathbb{R}^{231}$ 为附件编码谱，s_i 为风格，y_{irt} 为评分员 r 在重复 t 的评分。原始目标为

$$\bar{y}_i = \frac{1}{R_i}\sum_{r}\frac{1}{T_{ir}}\sum_{t} y_{irt}, \qquad g_i = \min(6, \max(0, \lfloor \widehat{y}_i + 0.5 \rfloor)).$$

因此连续监督、面板整数等级与个体打分是三个不同对象。所有外层误差按啤酒与感官目标等权；相同啤酒的全部记录留在同一角色。数据填补、标准化与监督选择均从当前训练数据估计。

两个原始种子提供两次三折内部验证；生成任务沿用三个 140/35/40/35 分区。原数据已在此前探索中被检查，本轮是严格隔离的内部补充验证，不是新外部队列。重复划分标准差衡量划分变化，不能当作独立多次采样的总体置信区间。配对酒级 Bootstrap 给出条件于既有训练和评价器的样本不确定性。

完整原协议、封闭补充协议、Q1 冻结哈希、训练日志和结果索引一并保存。本文遵照本地官方格式与 AI 规定 [3]，图表、程序与文字均如实记录 AI 贡献；不补造队员的独立工作或签名。

3　数据审计与监督目标重建

本轮以原始附件和新公开的逐评分员文件为输入，旧代码及网页端结果只用于问题定位。原始文件 SHA-256 清单在 outputs/rebuild/audit/raw_manifest.csv；新文件来源为原作者 Zenodo v4 [2]，MD5 为 a8f911d8c0061e096abf3b087678e164，与本地文件一致。全样本既往已被检查，因此本轮所有划分均属于内部重采样验证，不能称为从未接触过的外部测试。

3.1　样本与字段

1.csv 实际为 250 款啤酒、231 项化学或理化特征；4.csv 为 50 项感官目标；22 种风格中 Faro 仅 3 款，Tripel 为 28 款。题面中的 “226 种啤酒” 与实物不符。231 列包含物理测量和聚合量，不能全部称为单一化合物。1、4、7 按唯一 beer 对齐；beer_id 存在精度损失，不作主键。原始逐评分员文件中的 beer_name 是附件物理行序 1—250，不是附件 beer 编号；Beer 是品评位置，不能作啤酒键。正确映射的感官均值与附件 4 中位相关为 0.9777，错误按整数 beer 连接仅为 0.0076。这是一致性证据，仍需队员核查作者匿名化映射说明。

新文件包含 2197 条记录、16 名评分员、37 个日期；12 款酒跨日期重复，187 条属于同一啤酒与评分员的重复组合。定义目标时先对同一评分员的重复记录求均值，再在评分员之间等权平均，避免重复次数增加某人的权重。与所有记录直接平均相比，最大单元变化为 0.4605 分，整体变化较小，但对重复酒不能忽略。

3.2　评分卡层次与可识别性

50 个汇总字段由 26 个直接强度字段和 24 个派生细分字段构成。细分项在单条品评中等于父项强度或零，本轮逐条验证无违例。父项为零时，细分零分不能区分未勾选与勾选但强度为零。将这些零值全部作为 “不出现” 的监督标签会引入语义错误。面板均值满足子项不大于父项，但一般不满足 “父均值乘出现频率”；两者的平均绝对差为 0.1637 分。

3.xlsx 汇总公式生成的空白零分不是已填评分。新数据使真实面板均分可识别，不再需要分位数生成假标签；然而均分本身连续，最终整数是面板均分的决策等级。评分卡的可交付粒度需分两个口径说明：逐张 beer 表上可见 45 个带标签的打分格（Aroma 23、Flavour 22；Acetaldehyde 一行同时服务两列，Aftertaste 只属 Aroma），而 summary 工作表把 Malt/Hops/Esters 三类层次细分项展开为 50 个机器可读字段，该 50 项与 4.csv 及逐评分员原始记录的目标集合完全一致。本轮对全部 50 项都给出了 0—6 分级评分，交付表即按 summary 的字段名与顺序输出。

acetaldehyde 需要单独说明，因为它是唯一一个 “没有面板分数但有答案” 的维度。发布数据中确实不存在它的 0—6 记录：3.xlsx 的 50 字段汇总无此列，4.csv 无此列，作者公开的交叉相关矩阵 6.csv 的 50 个感官列中也找不到 acetaldehyde；附件 7 里名为 aroma_acetaldehyde 的列在 250 款酒中有 240 款落在同一地板值，且与实测浓度几乎无

关（Spearman −0.058），只能视为重编码的出现标记，不能当作强度标签。但“无面板分”不等于“无分级”：附件 1 的 acetaldehyde 列经 log10 反变换后（transforms.csv 的 log10 拟合误差 2.9×10^{-10}，249 对单元格；ln 为 0.89、逐列恒等为 5.27，故底数为 10），浓度为 0.01—18.56 mg/L、中位 4.10 mg/L；附件 8 给出的该化合物感官检出阈值为 50.0 mg/L。全部 250 款酒的实测浓度均低于该阈值，最高值也只有阈值的 37%。按 3.xlsx 图例 0=Not present，本数据集在该维度上整体处于“未呈现”档，因此 A_acetaldehyde 与 F_acetaldehyde 统一取 0，判据、阈值来源与反变换依据保存在 outputs/rebuild/q1_complete/acetaldehyde_grading.json。该 0 是阈值锚定的推断值而非面板评分，正文与结果表均单独标注，不与 50 个监督字段混同陈述。

barnyard 与 stale_hops 的汇总公式引用同一单元格；原始记录有 2183/2197 条相同。该问题是疑似标签重复，不能据此擅自改写 barnyard。主表保留 50 项并加疑点标记，另报告去除该列的训练与评价敏感性。评分员内标准化后聚合可精确重现附件 4 的 11272/12500 单元格，仍有差异；因此附件 4 不作为原始均分的简单可逆变换。

3.3 化学尺度与最低值堆积

73 个化学缺失单元由训练折中位数填补。附件 8 能核实 70 列的尺度，其中 37 列与 log10 对应，33 列保持原值或近似原值；161 列没有充分校准证据。pH、乙醇、色度、CO2 的实际配对支持 log10，不采信网络文案中的笼统原量纲说法。反变换逐字段执行，未知列仅保留附件编码。附件 1 与 8 有个别配对冲突，物理浓度判断需保留来源。

4VG 的 67.2%、CS2 的 83.6%、limonene 的 84.8% 样本位于同一个最小值。它们提示删失、编码地板或检测流程的混合效应，不能直接证明已知检测下限。生成模型学习观测分布中的点质量，不推断地板以下真实浓度。迁移中地板值保持相等也不能证明真实浓度未上升。

2.csv 经名称映射后可与重新计算的 Spearman 矩阵吻合，Pearson 则不吻合。全样本矩阵只用于描述性共表达图；每次预测特征选择重新在训练折内计算。原始相关矩阵 5、6 的数据说明与实际维度差异见前轮审计，本轮不将它们作为独立训练证据。

3.4 统一实验协议

依据本地官方文件 [3]，协议文件 configs/rebuild_protocol.json 在训练前保存。第一、二问使用固定种子 20261028、20261029 的两次三折划分；每个风格内循环分配，避免把不足折数的类别强行分层。所有同款酒的原始评分整体留在一折。内层从外层训练样本划出 20%，用于 Ridge/PLS 参数或网络训练轮数选择；再在完整外层训练集重训。训练与测试的中位数填补、缩放和选择严格隔离。

第三问每个种子分为生成训练 140、验证 35、独立评价器 40、测试 35 款。生成训练和评价器各保留各风格至少一款，因此极少样本风格的测试覆盖有限。预留 Pils 供迁移，

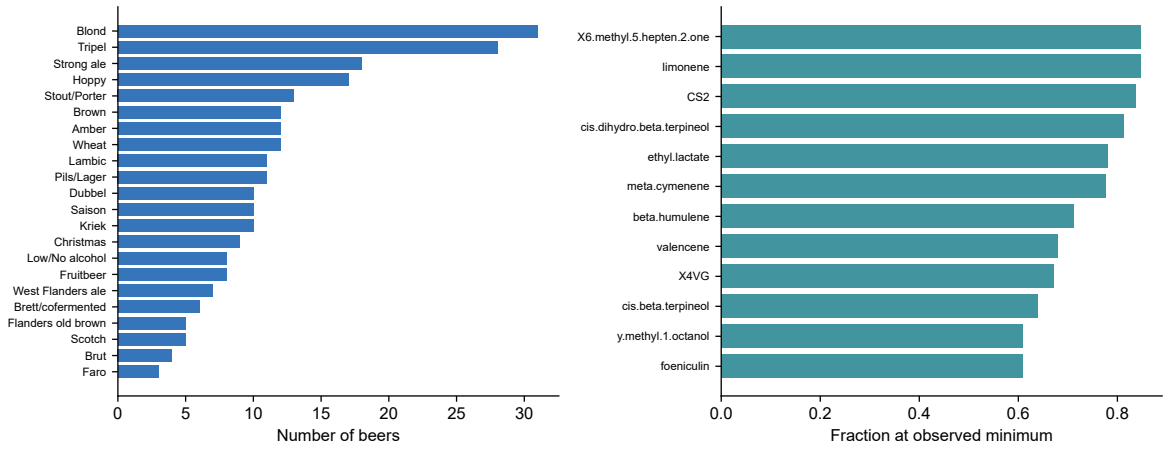


图 1: 风格样本量及观测最低值堆积；AI 辅助计算绘图

分区文件完整输出。评价器从未参与候选筛选，验证集 SWD 用于选择生成器，测试结果仅报告。重复划分并非独立实验样本；统计推断以啤酒为重采样单位。运行中修正两处实现公平性问题：深度模型外层重训、生成距离共同随机投影。修正记录单列，未扩展模型搜索范围。

证据路径：outputs/rebuild/audit/、outputs/review_20261028/；重复品评的 Kruskal–Wallis 与 Holm 结果仅作描述，跨日期评分员构成不同，不能直接归因于时间漂移。

4 问题一：真实面板均分与结构化监督

4.1 动机与目标

原始评分既提供 0—6 的实际量表，也揭示了评分员差异和细分项继承规则。直接对 50 个目标独立回归会放弃该结构，但共同叶节点的多输出树本身也能保持线性不等式。因此，结构模型的价值必须以误差和个体评分建模证据检验，不能以 “存在层次” 直接推出深度网络优越。

对酒 i、评分员 r、重复次 t 的记录 y，目标为先在 t 内平均、再在 r 间平均。Ridge、PLS、RF、ET、HGB 对这一面板目标训练；结构网络对逐记录似然训练，权重为同酒评分员数与该评分员重复次数的乘积倒数，使每款酒等权。测试以未见啤酒的面板均分为单位，既不把 2197 条当成 2197 款独立样本，也不把评分员身份用于推断测试酒真实分数。

4.2 模型与消融

传统基线包括训练均值、Ridge（alpha=10/100）、PLS（5/12 成分）、RF 与 ET [4]（180 棵树，叶最小样本 2，特征比例 0.7）、HGB（每目标 80 轮、7 叶、L2=10）。每个候选的训练与内层验证过程均独立执行。MLP 为 231→64→32→50，ReLU 和 0.15 Dropout；训练至多 160 轮，AdamW 学习率 0.002、权重衰减 0.001。

层次网络采用同一编码器，对 26 个直接强度预测七级概率 p(k|x)，对 24 个细分项预测条件勾选概率 q(k,x)。子项期望为 sum[k p(k|x) q(k,x)]，不能用两个边际期望相乘。子项出现损失只在父项强度大于零的记录上计算。加入评分员的版本在父项七级 logits 上添加中心化评分员效应，预测时使用零中心效应；这一估计对应平均 logits 的参考评分员，而不严格等同于全部评分员概率的平均，是该版本需进一步改进的估计目标差异。

MLP、Hierarchy、HierarchyRater 的可训练参数分别为 18578、28478、31390。训练曲线与参数配置均保存。结构网络不是原论文的复现模型；未在本轮增加图神经网络，因为现有任务的样本量与机制证据尚不足以支持额外图模块。

4.3 实际结果

下表为两次完整 OOF 预测各自计算指标后取均值；MAE/RMSE 单位均为原始评分分值，R^2 为 50 目标等权平均。

model	MAE	RMSE	R2	AF_MAE
ET	0.4454	0.6305	0.2530	0.5089
RF	0.4502	0.6354	0.2447	0.5132
InnerSelected	0.4521	0.6472	0.2101	0.5155
HGB	0.4588	0.6475	0.2169	0.5325
HierarchyRater	0.4690	0.6766	0.1385	0.5367
Hierarchy	0.4751	0.6754	0.1381	0.5369
PLS	0.4818	0.7024	0.0729	0.5521
Ridge	0.4833	0.7022	-0.0082	0.5494
MLP	0.5240	0.8082	-0.2752	0.5816
Mean	0.5488	0.7634	-0.0069	0.6044

观察到的最低 MAE 方法为 ET，MAE=0.4454，重复划分标准差 =0.0020。预先规定的内层模型选择流程 InnerSelected 的 MAE=0.4521、RMSE=0.6472、R^2=0.2101。观察到某模型排名第一与预先冻结选择流程的测试表现应分别表述，不能用全体 OOF 结果再选模型后误称其评估完全独立。

50 个目标按量表类型分成两组时误差差异明显：15 个 Aroma 与 15 个 Flavour 香气强度项（malt/hops/esters，共 30 项）的 InnerSelected 平均 MAE 为 0.5047，而其余 20 项（body、bitternes、co2、aftertaste、alcohol、sweetness、acidity、dms、metallic、4vg、t2n、diacetyl、stale_hops 与四个 spiced/barnyard 项、overall）平均 MAE 为 0.3486，即非香气强度项反而更易预测。其中 R^2 较高的为 lactic 0.7204、acidity 0.7183、alcohol 0.6653、acetic 0.6629、bitternes 0.5997、body 0.5010；dms、metallic、t2n、stale_hops、barnyard 的 R^2 接近零但 MAE 仅 0.23—0.41，因为这些缺陷项本身方差极小、几乎集中在 0—1 档，R^2 在此不具判别意义，应以 MAE 与等级命中率判断。这一分组误差结构说明交付表覆盖 50 项并非以精度换覆盖面。

交付评分卡还暴露一个必须披露的局限：12,500 个面板单元与 500 个乙醛单元中，预测等级全部落在 0—4 级，没有任何 5 级或 6 级结果，而真实面板均分取整后确实存在 112 个 5 级与 9 个 6 级单元。这不是取整造成的假象：连续预测的最大值只有 4.34，连四舍五入到 5 都不可能，说明模型本身把高分向中部收缩。按真值档位分组可见系统性偏差：真值 [0,2) 的 8,253 个单元被高估 +0.164，[2,3) 低估 −0.173，[3,4) 低估 −0.461，[4,5) 低估 −1.071，真值 ≥5 的 121 个单元全部被低估、平均低估 −2.074（真值均值 5.07 被压到 3.00）。也就是说，50 列预测在整体 MAE 意义上可用，但尾部不可用：任何依赖识别高强度样品（例如强酒花香或强缺陷）的下游用法都会漏检。本轮不为此重训模型，因为三组补充实验已经封闭且冻结；该偏差作为明确局限记录，定量结果见 outputs/rebuild/q1_complete/grade_range_diagnostic.csv 与 grade_range_summary.json。若要改进，方向是高强度样本的加权或分位数校准，属于模型层面工作，超出本轮范围。

配对统计以每款酒的两次 OOF 平均预测为基础，与 InnerSelected 比较，用 2000 次啤酒级 bootstrap 计算差值区间，并对 Wilcoxon 检验作 Holm 校正；它评价的是重复 OOF 集成，与上表单次误差均值略有不同。各目标误差见 per_target.csv，不能用总体均值掩盖弱目标。

comparator	positive_favors_reference	CI_low	CI_high	p_holm
Mean	0.1064	0.0938	0.1191	0.0000
Ridge	0.0310	0.0185	0.0461	0.0000
PLS	0.0264	0.0161	0.0400	0.0000
RF	0.0059	0.0001	0.0114	0.0278
ET	0.0010	-0.0043	0.0059	0.2874
HGB	0.0054	-0.0011	0.0119	0.0287
MLP	0.0660	0.0370	0.1067	0.0000
Hierarchy	0.0199	0.0136	0.0265	0.0000
HierarchyRater	0.0134	0.0079	0.0192	0.0001

4.4 评分输出与敏感性

将连续预测裁剪至 [0,6] 后按 floor(score+0.5) 输出整数。准确率衡量的是四舍五入后的真实面板均分等级，而非任一评分员的个体打分正确率。原始连续目标、连续预测与整数等级分别输出，不把近似等级重命名为原始评分。交付表 outputs/rebuild/q1/aroma_flavour_template_with_unobserved.csv 共 250 行 × 53 列、无缺失值：1 列 beer；50 个由感官监督模型预测的 0—6 字段（沿用 3.xlsx 名称与顺序，仅 A_ester_fruity、F_ester_fruity、4vg 三处因历史命名差异而映射）；以及 A_acetaldehyde、F_acetaldehyde 两列阈值锚定推断值（当前均为 0）。后两列不是原始面板评分：发布数据中不存在该字段的 0—6 记录，其取值由附件 1 实测浓度经 log10 反变换后与附件 8 感官检出阈值 50.0 mg/L 比较得到（250 款酒全部低于阈值，故按 3.xlsx 图例取 0=Not present），

判据见 outputs/rebuild/q1_complete/acetaldehyde_grading.json。因此评分卡中 50 个字段来自面板监督、2 个字段来自浓度对阈值的判定，两类证据强度不同，在交付表中仍以独立列呈现而不合并叙述。

experiment	MAE
ET_equal_rater	0.4452
ET_mask_0.05	0.4487
ET_mask_0.1	0.4539
ET_mask_0.2	0.4605
ET_official_Z_scale	0.3413
ET_record_training_equal_eval	0.4455
ET_without_barnyard	0.4502

额外遮蔽 5%、10%、20% 输入只施加在测试特征，使用训练中位数处理，模拟随机缺失；它不代表真实非随机检测失败。重复评分权重和去除 barnyard 的敏感性结果用于判断数据语义选择的影响。面板均分的区间由评分员重采样获得，是有限评分员测量不确定性，不是模型预测区间。输出的全样本 ET 模型用于后续展示性画像，不能用其对训练数据的拟合分数代替 OOF 成绩。

表中 ET_official_Z_scale 一行不可与其余各行直接比较：它的目标与评价都是附件 4 的标准化连续值，MAE=0.3413 以标准化单位计（各维度标准差 0.37—0.86），并非 0—6 分值。为判断 “先预测标准化值、再折内校准回 0—6” 这条替代路线是否可行，我们在同样 2 个种子 × 3 折上补做了同口径对照：直接预测 0—6 的 ET 为 0.4452，预测标准化值再按训练折内逐维仿射系数映射回量表为 **0.4502**，即校准后比直接预测略差 0.005，与标准化单位下的 0.3413 换算关系一致（缩放比约 1.32）。因此本轮不采用 Z 先行的路线，评分卡各列仍由直接预测 0—6 的模型给出；该行只作为量表口径对照保留，对照明细见 `outputs/rebuild/q1_complete/scale_comparability.csv`。

证据：outputs/rebuild/q1/fold_metrics.csv、repeat_metrics.csv、training_curves.csv、sensitivity.csv、panel_uncertainty.csv。旧网页版的 AF30 MAE=0.5062 对应不同标签权重与五折划分，不与本轮数字直接作超越声明。

5 问题二：训练内选法与统一的 50×3 化合物结果

5.1 从共线冗余到可检验的候选因素

单变量相关前三经常重复表达同一组发酵或酒花信号。把三项相关性当成三种独立机制，会放大候选因素的解释力度；直接改用条件选择又可能因有限样本残差而产生更不稳定的排名。因此，本问同时评价预测、冗余与重采样稳定性，并将 “选择哪一种规则” 的过程纳入训练数据内部。最终任务输出固定为每个感官三项，而不将低频变量包装成稳定的唯一因素。

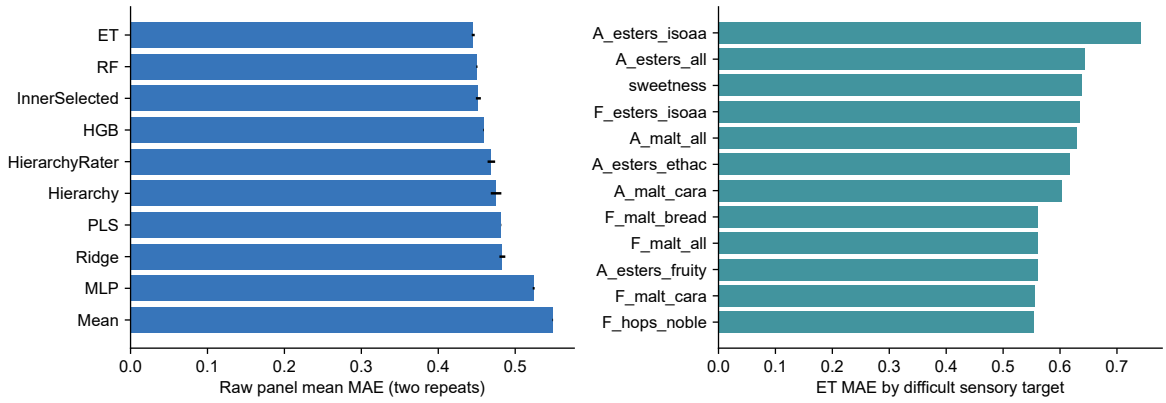


图 2: 真实评分尺度上的模型误差与困难目标；AI 辅助

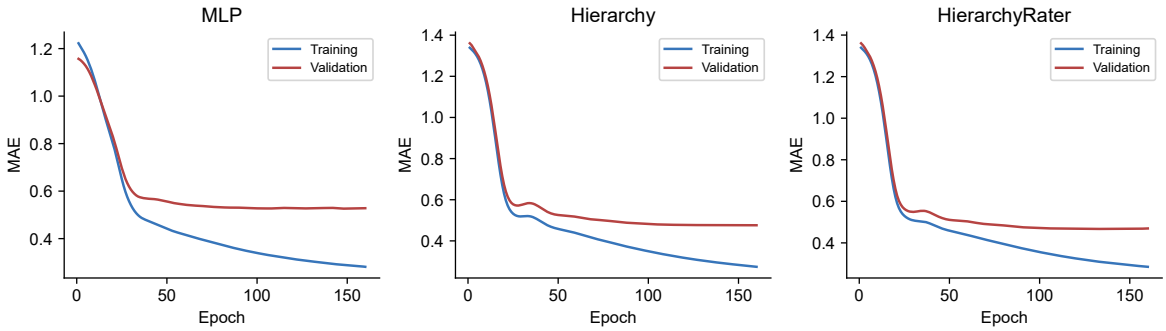


图 3: 内层训练与验证曲线，六个外层任务平均；AI 辅助

候选集中剔除颜色、pH、压力、聚合总量等 13 项非单一分子测量，保留 218 个可识别化合物候选。完整预测任务仍可使用所有 231 项；两种特征口径不能混淆。共表达图节点为 231 项测量变量，附件 2 的 Spearman 绝对值达到 0.65 时连边，符号与绝对值分别保留。该图只作全样本描述，不作为外层测试的训练先验；相关定义和名称映射已经在原始数据上数值复核。

5.2 四种规则及正则残差推导

设训练秩变换后的特征矩阵为 Z，各列中心化并归一化，目标秩向量为 t。Spearman 规则按 $|z_j^T t|$ 取前三；2+1 规则先取相关前二，再条件选第三；Forward3 在三步均更新条件残差；冗余控制规则在每步用目标相关减去 0.5 倍与已选特征的最大绝对相关。

对已选集合 S，令 $H_S = Z_S(Z_S^T Z_S + 0.02I)^{-1} Z_S^T$，构造 $u_j = (I - H_S)z_j$ 与 $r = (I - H_S)t$。在这个正则残差空间，单变量问题的最优增益为

$$\min_b \{\|r - bu_j\|^2 + 0.02b^2\}, \qquad G_j = \frac{(u_j^T r)^2}{u_j^T u_j + 0.02}.$$

每一步选择未入选变量中 G_j 最大者。这里的“完整前向”指三个步骤都条件更新；由于 H_S 不是幂等投影，该分数是所定义辅助残差问题的增益，不能声称精确求解所有三变

量组合或原始联合 Ridge 的全局最优。2+1 与 Forward3 采用相同的条件分数，差别仅在前两步。

四种规则选择后，均用相同的三个附件编码特征、训练中位数和训练标准化，拟合 alpha=8 的 Ridge。它们拥有相同预测器和变量数，避免把选择规则与预测模型复杂度混在一起。

5.3 方法选择也属于训练流程

沿用原两次三折啤酒级外层划分。每个外层训练集内部，按既有 20% 划分计算四方法的 50 目标等权 MAE，确定一种规则，然后在完整外层训练集上重新选择三变量并拟合；外层酒样只用于评价。最终全样本模型重用同一内部选择算法，再全量重拟合。外层所有候选成绩均保留，最终规则的选择不使用其外层分数。

下表先对每次完整 OOF 预测计算指标，再报告两次结果的均值与标准差；它修正了按不等大小折直接平均的汇总口径。真实监督是评分员等权原始均分，MAE 与标准化附件 4 的旧误差不可直接横比。

方法	MAE 均值	MAE 标准差	RMSE	R^2
Forward3	0.5038	0.0045	0.7124	0.0983
2+1	0.4988	0.0012	0.7029	0.1155
冗余控制	0.4996	0.0019	0.7046	0.1107
Spearman	0.5021	0.0025	0.7065	0.1117
训练内选法	0.5024	0.0058	0.7093	0.1031

最终全训练数据内部选择为 2+1，故正式 50×3 表、分子名称、选择得分、入选率与替代组指标统一来自该规则。训练内选法流程的外层 MAE 为 0.5024，固定 2+1 的 OOF MAE 为 0.4988。前者没有获得额外预测收益，说明小样本内层选择具有方差；不能用全量内部选出的 2+1 替代训练内选法的外层成绩。配对检验先平均同款酒的两次 OOF 预测，再按酒计算误差，与表中逐次误差均值的口径不同。检验与啤酒 bootstrap 区间见 closed/q2/paired_tests.csv，没有据测试结果再设新的加权规则。

5.4 固定规则与完整流程的两类稳定性

100 次 Bootstrap 在各风格内按酒重采样，填补、缩放和选择全部重做。重复抽到的同款酒被整体放入同一内层角色，避免其副本跨越内层训练与验证。固定规则频率回答“给定规则时分子是否稳定”；训练内选法频率还包含规则本身变化。最终 150 行同时保存两种频率，避免以固定规则频率代替完整流程的不确定性。

替代组沿用描述性完全链接分组：组内各对的绝对 Spearman 至少 0.8。它用于衡量相近测量是否可替代，不参与外层特征选择。每个目标用相同的 100 对 Bootstrap 集合计算单分子及替代组 Jaccard，并同时记录三因素内部绝对相关。

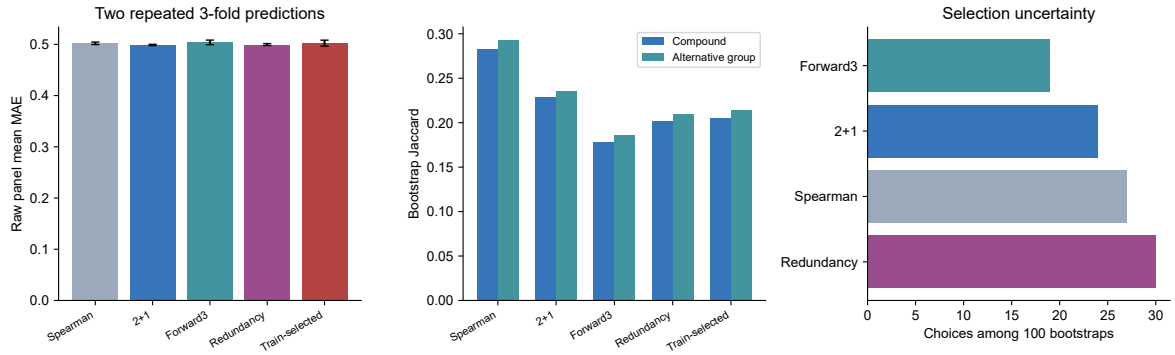


图 4: 四规则与训练内选法的预测、稳定性及选择频率（AI 辅助绘图，数据来自封闭补充实验）

方法	单分子 Jaccard	替代组 Jaccard	三因素平均绝对相关
Forward3	0.1787	0.1860	0.1535
2+1	0.2288	0.2354	0.2740
冗余控制	0.2020	0.2102	0.1368
Spearman	0.2827	0.2929	0.4385
训练内选法	0.2057	0.2141	0.2740

内部规则的 Bootstrap 选择次数如下。规则切换本身是小样本不确定性的结果，不据此继续增加规则或调节惩罚系数。

规则	次数
冗余控制	30
Spearman	27
2+1	24
Forward3	19

5.5 风格混杂与化学解释

表中保留原始 Spearman、风格内去均值后的秩关联、标签疑点以及来源/单位。后者是混杂敏感性指标，不是已识别的因果效应；风格也可能代表真实发酵过程，调整后关联变化需结合过程解释。高度相关的酯和高级醇可能共同反映发酵条件，不能简单将排名逐一解释为独立作用。原论文通过真实添加实验验证部分候选 [1]，本工程尚未开展这一步。

最终表不以 “化学常识” 手工替换统计结果。高频主因素、同组可替代项和低频候选分别解释；barnyard 保留疑似公式别名标记。每一项的名称、频率、方向和风格调整变化见统一 CSV 与逐维解释报告，供队员独立作化学复核。

6 问题三之一：22 风格条件生成的分布检验

6.1 小风格样本与观测最低值的冲突

每种风格仅 3 至 31 款，无法支持各自独立估计 231 维联合分布。直接连续生成又难以保留大量相同最小值。采用总体信息共享可以降低方差，却可能削弱风格边界；分布距离变小也可能来自生成谱向均值收缩。本轮将这种取舍作为待验证问题，比较已有四模型，参数与训练角色全部冻结。

PCA 高斯使用 12 维潜变量、风格均值收缩和共享协方差，输出限制在训练范围。独立 Copula 与联合 Copula 共享收缩条件边际：

$$\widehat{F}_{j,s} = w_s \widehat{F}_{j,s}^{\text{empirical}} + (1 - w_s)\widehat{F}_{j,all}, \qquad w_s = \frac{n_s}{n_s + 10}.$$

对稀有风格，较大的总体权重控制经验分布方差。联合 Copula 将训练秩变换成正态分数，用 Ledoit–Wolf 收缩估计相关矩阵 [5]，通过 $z \sim N(0, \widehat{R})$、$x_j = \widehat{F}_{j,s}^{-1}(\Phi(z_j))$ 采样。独立消融令相关矩阵为单位阵，检验联合依赖的贡献。经验逆分布的区间自然保留最小值点质量，不需要后续逐列随机替换；模型并未识别最低值以下的真实浓度。

CVAE 沿用 231 维输入、22 类条件、64 维隐层、12 维潜变量和 KL 系数 0.05，最多 160 轮，以原 35 款验证酒的重构误差选检查点 [6]。四模型不增加训练轮数、结构或超参数搜索。

6.2 逐风格评价如何避免循环自证

三个原始场景均保留 140 款生成训练酒、35 款验证酒、40 款独立评价器训练酒与 35 款测试酒。对每个场景的每种风格生成 40 条谱，共计算 22×4×3=264 行指标。四模型共享按场景与风格固定的采样种子、48 个投影方向和训练尺度。风格分类器使用原迁移评价器的既有 ET 配置，在独立 40 款酒上拟合并保存；其输出只用于评价，不参与模型或样本筛选。

对测试风格分布计算边际 Wasserstein 与切片 Wasserstein 距离。相关结构使用真实和四模型共同可计算的 Spearman 变量对；测试样本少于 4 款时不估计该项。令 $B_j(x) = 1\{x_j \le m_j^{train}\}$，其中 m_j^{train} 为训练最小值，仅对训练中低端比例至少 25% 的变量评价。联合最低值模式用各变量对的 $E[B_j B_k]$ 绝对误差衡量，同时报告单变量低端质量误差。这里的 “低端” 是观测编码阈值，不是已知仪器 LOD。

最近邻距离、完整复制比例和标准化多样性均在最终输出谱上计算。零完整复制只能排除逐行原样复制，不能证明经验边际没有训练信息记忆。风格分类器的输出分数也没有概率校准，应解释为模型诊断分数。

6.3 支持范围与真实结果

原划分中 20/22 种风格有独立测试酒；Faro 与 Flanders old brown 为零。相关结构仅 4/22 种风格在至少一个场景满足样本数条件，共 7 个风格场景。两类缺口在表格和热图中保留 NA，不能用验证酒、训练酒或生成样本补成测试证据。完整 22 风格覆盖表及 88 行分模型汇总见附录。

下表先在每个风格内跨场景平均，再对有定义的风格等权平均；分类一致性覆盖全部 22 种风格，分布和联合低端误差覆盖 20 种，相关误差只覆盖 4 种。各列支持集合不同，因此必须按列比较同指标，不能把它们合成未预设的总分。

模型	条件 SWD	风格一致性	相关误差	联合低端误差
CVAE	0.7059	0.0917	0.4608	0.1896
独立 Copula	0.8676	0.1409	0.4669	0.2260
联合 Copula	0.8637	0.1356	0.4508	0.2288
PCA 高斯	0.7516	0.0852	0.4704	0.1887

条件尺度结果与旧版混合风格总体距离回答不同问题。CVAE 的条件 SWD 较小，但其平均多样性为 0.4202；独立 Copula 为 0.8966。当真实测试风格只有一两款时，收缩到中心可能使经验距离更小，不能据此断言 CVAE 更真实。最近邻距离、风格一致性和低端质量共同揭示这种取舍。

上表会让读者产生一个必须正面回答的疑问：既然 CVAE 的条件 SWD 最低（0.7059），为什么交付的生产生成器是独立 Copula（0.8676）而不是 CVAE？答案是两件事发生在不同时间、依据不同数据，且排序反转本身可以被量化。选型在封闭诊断之前就已冻结：三个场景各自的验证折（35 款酒）上按平均 SWD 取最小者，逐场景选出的分别是联合 Copula、独立 Copula、独立 Copula，测试折从头到尾没有参与选型，封闭诊断也不被允许回改选型。把封闭诊断的逐风格结果拆开看，CVAE 的相对优势与测试集大小高度相关（相关系数 r=0.70）：在只有 1 款独立测试酒的风格上 CVAE 13/13 全胜、优势 -0.242，2 款时 12/12、-0.168，3—4 款时 9/11、-0.081，5 款及以上时只剩 4/6、-0.011。也就是说，CVAE 的优势随经验分布分辨率上升而单调消失——测试集只有一两款酒时，经验一侧几乎没有分辨率，任何向风格均值收缩的生成器都会显得“更近”。这与 CVAE 自身的退化程度一致：多样性 0.4202 对 0.8966，到最近训练样本的 RMS 0.6812 对 1.1144，它更靠近小样本经验分布。因此选型依据不是条件 SWD 这一列，而是验证折规则加上多样性与低端质量的匹配；反过来说，如果 CVAE 在 5 款以上测试酒的风格上仍保持优势，冻结的选型就是错的——它没有，6 对里 Copula 赢 2 对。需要同时承认的是，满足该条件的风格只有 6 个，这只是一个方向性证据，不足以给出决定性检验。完整对照见 outputs/rebuild/q3_complete/selection_logic.json 与 selection_criteria_table.csv。

联合 Copula 相关误差为 0.4508，独立 Copula 为 0.4669，改善范围限于可估计场景；联合低端误差分别为 0.2288 和 0.2260，未显示联合模型的统一优势。共享一个总体依赖矩阵不能自动恢复每类风格特有的低端共现关系。

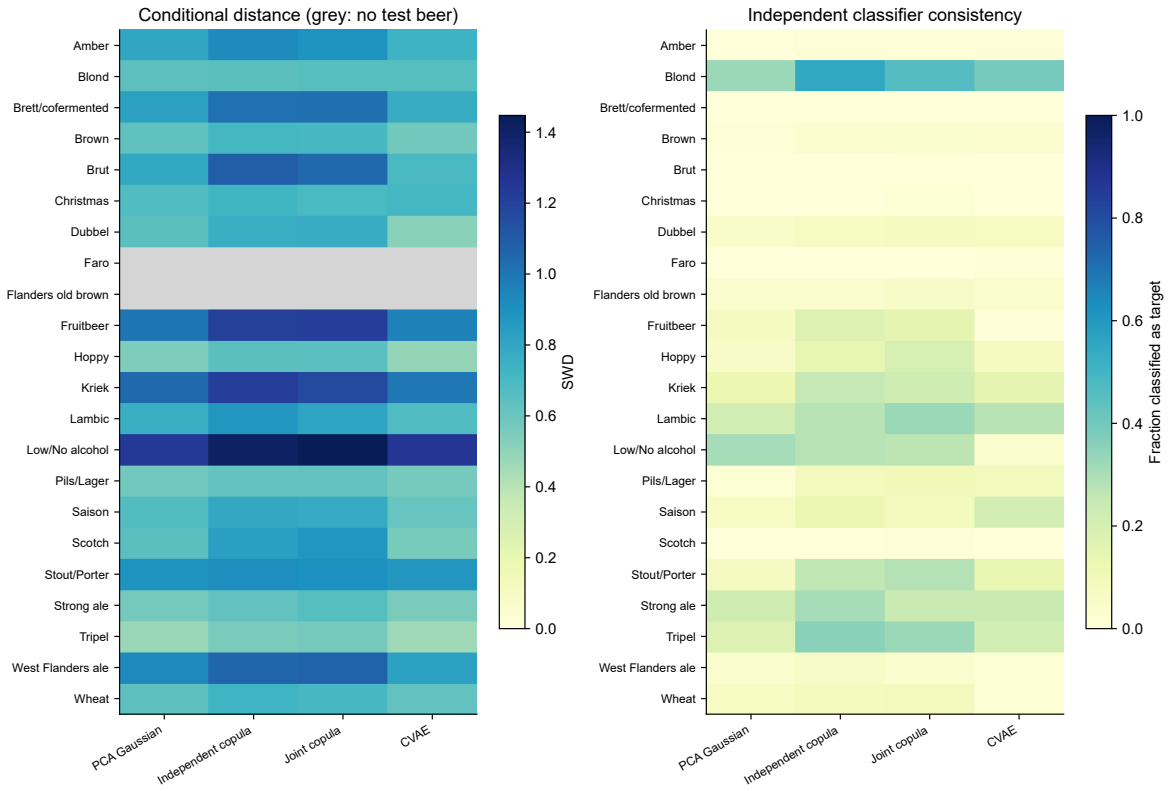


图 5: 22 风格条件距离与独立风格一致性；灰色表示无测试证据（AI 辅助绘图，数据来自封闭补充实验）

以上数字给出了本问能支持到什么范围，需要一并交代清楚。第一，风格条件控制本身仍然有限：四个模型的独立评价器风格分类准确率分别为 CVAE 0.0917、PCA 高斯 0.0852、联合 Copula 0.1356、独立 Copula 0.1409，在 22 类问题上都只略高于随机水平（1/22≈0.045），说明「生成了指定风格的谱」不等于「已充分复现该风格的特征」，风格画像应视为条件生成的方向性结果而非风格标定证据。第二，覆盖度不完整：22 种风格中仅 20 种在既定划分下有独立测试酒，相关结构指标只有 4 种风格满足样本条件，Faro 与 Flanders old brown 两类完全没有独立测试证据。第三，物理尺度仍有缺口：化学列中只有一部分具备可靠的量纲校准证据，其余仅保留附件编码，因此生成与迁移结论只能建立在观测编码与本报告给出的阈值判定上，不能外推为绝对浓度结论。

生产生成器和迁移生成器继续沿用此前验证集选定结果，未依据此次条件测试成绩改选。Tripel、Kriek、Lambic 画像及 22 风格 880 条谱仍保留为展示输出；生成模型的统计接近不构成真实酿造可行性证据。

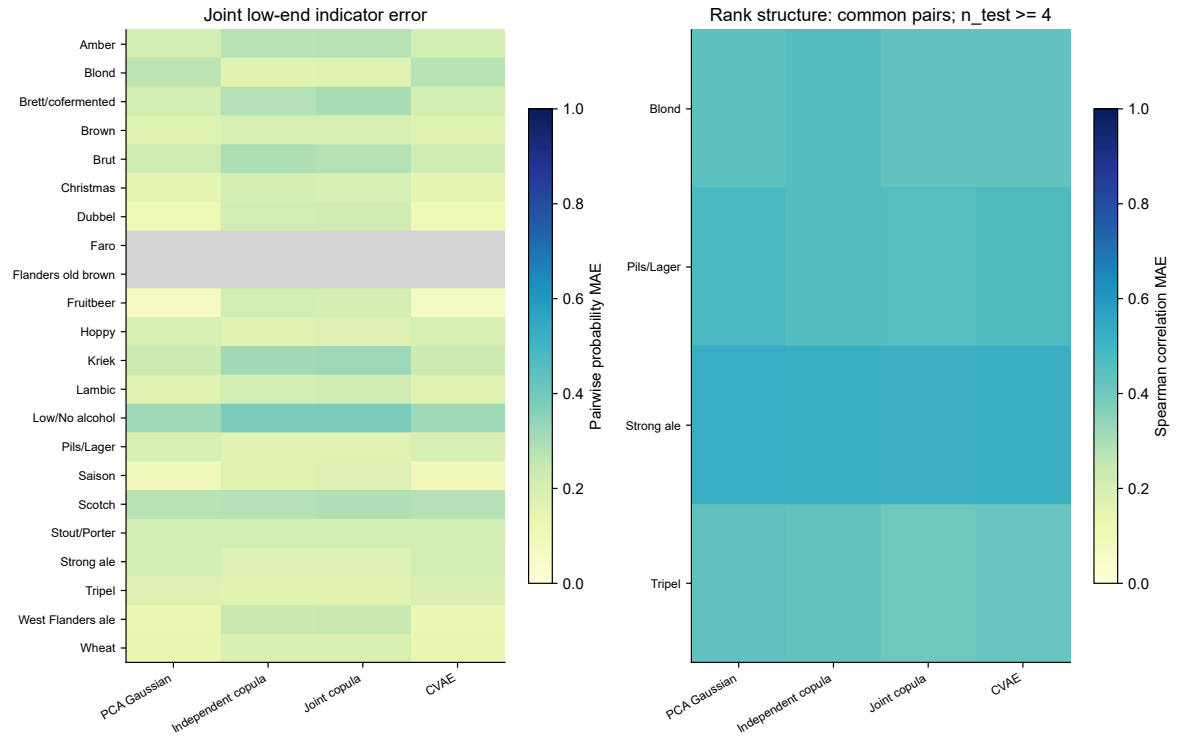


图 6: 联合低端模式与有共同变量对支持的相关结构误差（AI 辅助绘图，数据来自封闭补充实验）

7 问题三之二：独立评价与拒绝迁移

7.1 决策目标和评价器必须隔离

跨风格迁移同时追求目标接近与缺陷不恶化。若同一个感官预测器既筛选候选又证明安全，优化器容易利用其误差。本实验固定原始 9 款不同 Pils 源酒，每款只保留其最早符合条件的原测试场景，避免重复计数。源酒不属于相应生成训练、验证或评价器训练角色。三个场景的独立评价器在优化前序列化，其文件哈希在所有策略完成后保持不变。

每个源生成 80 条 Tripel 候选，再以 0.25、0.5、0.75、1.0 插值形成 320 条迁移谱，加入原谱作为第 321 个候选。四策略共用候选、设计预测器、尺度和成本；固定的拒绝策略直接返回原谱。对候选 x'，成本为

$$J(x') = \|T(x') - \mu_{Tripel}\|_2 + 0.15\|T(x') - T(x)\|_2,$$

其中 T 为生成训练集标准化，目标中心来自同一批 Tripel 候选。独立风格分类分数不进入这个成本。

无约束策略选择最低成本。点约束与不确定性约束先将四个明确化学标记裁剪至不高于源编码，再用 12 个仅在生成训练集拟合的 Bootstrap ET 计算 stale_hops 预测差。点约束要求平均差不大于零；不确定性约束要求 90% 分位差不大于零。保留原谱保证候选集合存在可行返回；返回原谱计为拒绝迁移。该分位没有经覆盖率校准，是模型集成风险筛查，不是 90% 置信安全保证。

所有策略的选择完成后，才读取独立评价器，对源谱与最终谱统一评分。评价器包括固定 ET 分类器、感官回归器和在相同独立 40 款酒上构造的 12 个 Bootstrap 回归器。评价器的集成差分位用于检验设计风险能否迁移，不反馈到选择过程。

7.2 四策略的 9 酒配对结果

目标收益为独立分类器 Tripel 分数之差；缺陷风险同时报告 stale_hops 均分变化、上升酒数和独立集成 90% 分位。化学 RMS 为按生成训练尺度标准化的 231 维改变量。下表以全部 9 款为分母，包含被拒绝酒的零收益，避免只在接受子集上展示效果。

策略	目标收益	stale 变化	stale 上升率	化学 RMS	拒绝率
无约束	0.0514	-0.0094	0.6667	0.6191	0.0000
点约束	0.0514	-0.0241	0.5556	0.6096	0.0000
不确定性约束	0.0289	0.0052	0.4444	0.5231	0.2222
拒绝迁移	0.0000	-0.0000	0.0000	0.0000	1.0000

点约束带来平均目标分数收益 0.0514，其独立 stale_hops 均值变化为-0.0241，但仍有 5/9 款上升。平均下降与个体风险并不矛盾，说明仅报告均值会掩盖失败样本。不确定性约束拒绝 2/9 款，全部源酒的平均目标收益降为 0.0289，仍有 4/9 款上升；在实际接受的 7 款中对应 4/7。因而本次证据不支持 “更保守的设计约束必然带来独立缺陷改善”。

必须把这一点明确记为局限，但要同时给出赛题口径下的对照，否则会低估约束的作用。本文的缺陷判定有两套口径，二者结论强度不同，缺一不可。

第一套是本文沿用至今的严格口径：只要独立评价器的 stale_hops 预测相对源谱上升了任意 epsilon（>1e-10）就计为恶化。按此口径三种策略的上升酒数是无约束 6/9、点约束 5/9、不确定性约束 4/9，约束把上升数从 6 压到 4—5 但没有压到 0；代价是目标收益由 0.0514 降到 0.0289（约减半）并额外拒绝 2/9 款。这一口径下确实没有得到 “不恶化”。

第二套是赛题原文指向的口径：题目把缺陷化合物定义为 “所列 DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops 等超阈值项”，即判定对象是超阈值与否，而不是任意微量上升。按此口径统计 “源酒未超阈值、迁移后越过阈值” 的次数（三个具备可用感官检出阈值的化学缺陷项，源酒低于阈值的可用样本 23 个），无约束策略发生 7 次越阈，点约束 0 次，不确定性约束 0 次。四个有观测化学编码的缺陷项在点约束与不确定性约束下的化学增加次数均为 0，与这一结果一致。

两套口径的差别只有 0.05 分以内（0—6 量表上），而这远低于模型自身在 stale_hops 上的约 0.23 MAE，因此任何一套都不能单独支撑 “保证不恶化”，也不能据此否定约束的作用。本文的主张因此限定为：在赛题定义的超阈值口径上，硬约束把越阈次数由无约束的 7 次降到 0 次，即目标函数意义上的 “缺陷不恶化” 在本次内部检验中达成；在任意微量上升的严格口径上仍有 4—5/9 款酒上升，该残余风险列为未解决局限。要真正闭合这两套口径之间的差距，需要真实添加样品与盲评，或在本数据之外校准评价器误差的传递关系，两者都

超出本轮可得证据。两套口径的逐项结果见 outputs/rebuild/q3_complete/defect_criteria.json、defect_threshold_crossing.csv 与 defect_magnitude.csv。

拒绝迁移的收益与改变量为零，拒绝率 100%，提供明确的决策下界。它不能作为实现目标风格的成功，但在缺少可靠安全证据时是可陈述的保守输出。本轮未通过观察独立评价器后新增拒绝阈值，也未把拒绝策略改造成反馈优化器。

7.3 统计与缺陷口径

2000 次配对 Bootstrap 按 9 款不同源酒重采样，区间条件于固定训练场景与固定评价器，不包含重新训练的不确定性。Wilcoxon 检验对三种实际迁移与拒绝的两类收益/缺陷差分别组成 6 项比较，再作 Holm 校正。

策略	指标	均值差	Holm 校正 p
点约束	target_gain	0.0514	0.0469
点约束	stale_change	-0.0241	1.0000
不确定性约束	target_gain	0.0289	0.3125
不确定性约束	stale_change	0.0052	1.0000
无约束	target_gain	0.0514	0.0469
无约束	stale_change	-0.0094	1.0000

点约束与无约束的目标分数收益在本次内部检验中通过 0.05 的 Holm 门槛；三个实际迁移策略的 stale 变化均没有通过。小样本条件下不能把未拒绝零假设解释为缺陷等效或安全。

DMS、Diacetyl、4VG、trans-2-nonenal 分别核对附件列名和变换。前三项同时列出平均及中位感官阈值，trans-2-nonenal 和 stale_hops 没有完整可用阈值时记 NA。感官检测阈值不等于跨风格通用的不可接受界限。对于四个有观测化学编码，约束策略的增加次数为零；源缺失值和低于/位于训练最小值的标记保留 “物理不可验证”，不据裁剪宣称真实浓度顺序。stale_hops 使用独立感官代理，不能换算成唯一化合物浓度。

逐酒原谱角色、评价器哈希、选中候选编号、接受/拒绝、五类缺陷检查及单位表均保存。统计生成和迁移结果仍需要真实样品添加与盲评才能验证工艺和感官可行性。

8 创新动机、消融与模型评价

本研究的改进围绕三个可核验的困难展开。第一，真实评分卡既有直接强度，也有父强度继承的子项，且存在重复评分员。按评分观测机制定义目标与条件似然，使模型学习对象明确；冻结的结构消融显示其优于普通 MLP，但未超过 ET，因而保持传统树模型作为预测主线。

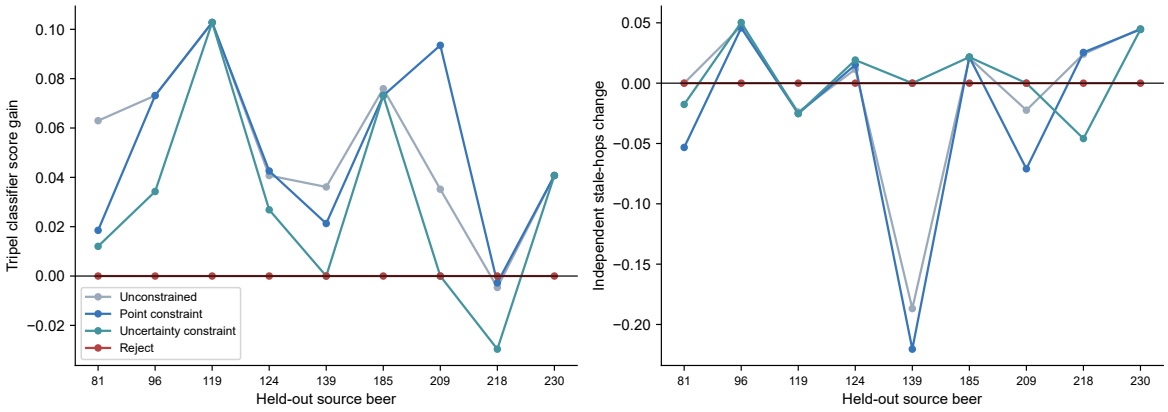


图 7: 9 款独立源酒的目标收益和独立 stale_hops 变化（AI 辅助绘图，数据来自封闭补充实验）

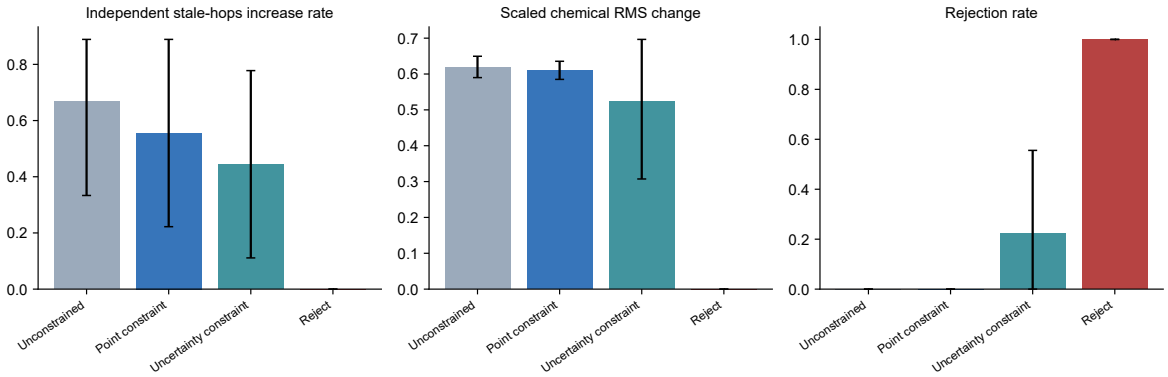


图 8: 缺陷上升率、化学改变量与拒绝率；误差棒为酒级 Bootstrap 区间（AI 辅助绘图，数据来自封闭补充实验）

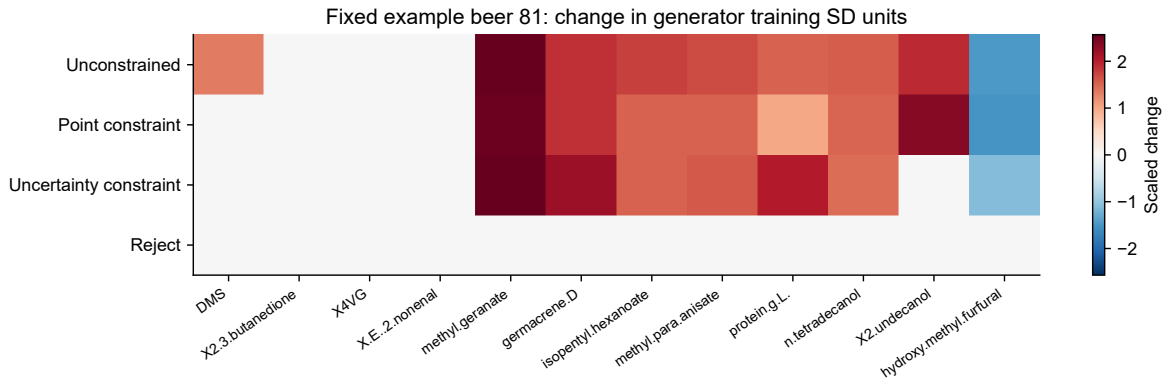


图 9: 预先固定为最小 beer 编号的迁移示例，未按效果择优（AI 辅助绘图，数据来自封闭补充实验）

第二，Top-3 既是一个预测问题，也是一个选择不确定性问题。将四种规则放入训练内选择，并在 Bootstrap 中重新选择规则，使最终表、性能和稳定性属于同一个可复现流程。外层结果显示自动选法没有超过固定 2+1，这个负结果揭示了小样本内层方差，不能通过事后改表消除。替代组使可替代信息可见，但不是新增因果证据。

需要把结论强度说得更准确，避免读者误读。最终 2+1 规则的训练内得分（0.4841）略低于纯 Spearman 基线（0.4845），重采样稳定性也低于它：2+1 的平均分子 Jaccard 为 0.2288、替代组 Jaccard 为 0.2354、Top-3 内平均绝对相关 0.2740，而 Spearman 分别为 0.2827、0.2929、0.4385。外层预测 MAE 上 2+1 为 0.4988，优于 Spearman 的 0.5021，但差距仅 0.003 量级。因此第二问的贡献应定位为：把「选择哪一种规则」纳入训练内部并输出可复现的统一 50×3 表，同时用冗余控制减少同一机制在 Top-3 内重复占位；不能宣称 2+1 在预测性能或重采样稳定性上全面优于相关性基线。Top-3 仍是候选因素清单，其因果地位需真实添加实验确认，本工程未开展该步。

第三，生成任务区分单变量最低值质量、联合低端模式和风格条件一致性，迁移任务进一步区分设计预测器与独立评价器。独立 Copula 是联合依赖消融，PCA 高斯是连续低维基线，CVAE 是非线性共享生成对照；无约束、点约束、不确定性约束与拒绝给出决策层对照。对照没有支持 “深度或联合依赖模型在所有指标上更优”，也没有支持 “设计风险分位保证独立缺陷安全”。

创新应定位于对本题观察机制的具体处理和完整验证，而不是宣称新发明 Copula、Bootstrap 或变分自编码。尚无外部新队列与真实酿造盲评，因此不能宣称一般化的感官因果机制。最稀有两类没有独立测试证据，相关结构多数风格不可估计；这些缺口限制结论支持范围。按用户设定，三组封闭补充完成后停止所有模型探索，不再为改善结果增加模型、阈值或评价器。

9　结论与建议

250 款啤酒、231 项测量与 2197 条逐评分员记录形成统一目标和训练隔离基础。第一问冻结 ET 的 MAE 为 0.4454、RMSE 为 0.6305，整数输出对应真实面板均分的舍入等级。第二问最终训练内部选出 2+1 并输出 50×3 候选；训练内选法流程的 OOF MAE 为 0.5024，说明方法选择的代价应进入评价。第三问完成 22 风格四模型条件诊断，20 种风格有独立分布证据、4 种风格具备最低相关估计支持；指标取舍与有限样本误差阻止了单一最优生成器的全面结论。9 酒配对迁移显示，点约束保留目标收益，却仍有 5 款独立缺陷代理上升；不确定性约束以 2 款拒绝为代价仍出现 4 款上升。

该工程已完成既定三组补充，具备数据、代码、指标、图表与论文之间的追溯关系。正式参赛前的必要工作是队员独立核实核心建模与分析、确认匿名样本映射及疑点标签，并决定可采纳的实验结论。真实工艺与盲评验证属于当前数据无法替代的证据。

10 参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI: 10.1038/s41467-024-46346-0.

[2] Schreurs M, et al. Data from: Predicting and improving complex beer flavor through machine learning, version v4. Zenodo, https://zenodo.org/records/23624915, 访问日期 2026-10-28.

[3] MathorCup 组委会. 2026 年 MathorCup 大数据挑战赛赛道 A 初赛题、论文格式及提交规范、人工智能工具使用规定（试行）. 本地官方附件, 2026.

[4] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3–42, 2006. DOI: 10.1007/s10994-006-6226-1.

[5] Ledoit O, Wolf M. A well-conditioned estimator for large-dimensional covariance matrices. Journal of Multivariate Analysis, 88(2):365–411, 2004. DOI: 10.1016/S0047-259X(03)00096-4.

[6] Kingma D P, Welling M. Auto-Encoding Variational Bayes. arXiv:1312.6114, 2013, https://arxiv.org/abs/1312.6114, 访问日期 2026-10-28.

[7] OpenAI Codex，GPT-6 系列及 gpt-6.1-sol，OpenAI，2026-10-28。用于审计、代码、实验、统计、图表与论文辅助。

11 AI 使用声明与独立工作状态

本工程及本稿使用 OpenAI Codex，历史 GPT-6 系列及本轮 gpt-6.1-sol[7]，辅助数据核验、模型实现、实验运行、统计、可视化、论文和排版。本轮严格执行三组封闭补充，Q1 不重训、不改预测，仅在 2026-10-28 修正了评分卡交付的字段过滤缺陷（30 字段 →3.xlsx summary 全部 50 字段），修正记录见 outputs/rebuild/audit/protocol_amendment.md，冻结哈希同步更新；三组完成后停止建模探索。程序头部、图表说明和支撑材料均标明 AI 参与。

官方规定要求核心建模、算法设计、求解与分析由参赛队员独立完成。当前没有可核实的队员独立核心工作或人工修改记录，本稿如实保留该状态；单纯审阅签字不能替代独立完成。队员须独立开展并记录关键工作后决定正式采用内容。详细说明、关键交互摘要与待完成记录位于 AI 工具使用详情说明.pdf、AI_usage_log.md 和 human_decision_record.csv。

A　附录：统一 50×3 候选化合物

最终规则为训练数据内部选择的 2+1。频率、单位和风格调整关联见完整 CSV。

感官	候选 1	候选 2	候选 3
A_esters_all	n.hexanol	acetaldehyde	lactic_acid.mg.L.
A_esters_ethac	ethyl_acetate	ethanol..v.v.	X2.methyl.1.butanol
A_esters_flower	beta.phellandrene	X1.alpha.terpineol	acetaldehyde
A_esters_fruity	beta.phellandrene	lactic_acid.mg.L.	X1.10.di.epi.cubenol
A_esters_isoaa	n.hexanol	isopentyl_acetate	tau.cadinol
A_hops_all	ethyl_octanoate	ethyl_decanoate	methyl.geranate
A_hops_citrus	citronellol	humulene.epoxide.II	X6.methyl.5.hepten.2.one
A_hops_noble	ethyl_octanoate	ethyl_decanoate	iron
A_hops_tropical	DMS	methyl.geranate	X.E..beta.ocimene
A_hops_woody	X2.phenyl.ethyl.butyrate	ethyl_decanoate	X2Z.6Z.farnesol
A_malt_all	methylpyrazine	acetaldehyde	X.E..beta.ionone
A_malt_bread	glycerol	X3.methyl.2.hexanol	DMS
A_malt_burn	isobutanol	ethanol..v.v.	methylpyrazine
A_malt_cara	acetaldehyde	methylpyrazine	ethyl.hex..2E..enoate
A_malt_grain	X.E..beta.ionone	terpinen.4.ol	ethyl.furoate
F_esters_all	acetaldehyde	ethyl_acetate	lactic_acid.mg.L.
F_esters_ethac	ethyl_acetate	ethanol..v.v.	E.nerolidol
F_esters_flower	beta.phellandrene	terpinolene	isopentyl.butanoate
F_esters_fruity	DES	iron	beta.phellandrene
F_esters_isoaa	n.hexanol	isopentyl_acetate	ethyl_decanoate
F_hops_all	ethyl_octanoate	ethyl_decanoate	cis.dihydro.beta.terpineol
F_hops_citrus	nerol.oxide	furfural	ethyl.furoate
F_hops_noble	ethyl_octanoate	ethyl_decanoate	iron
F_hops_tropical	methyl.geranate	hexyl.acetate	ethyl.furoate
F_hops_woody	ethyl_decanoate	alpha.muurolol	X9.epi..E..caryophyllene
F_malt_all	ethanol..v.v.	acetaldehyde	methylpyrazine
F_malt_bread	glycerol	acetaldehyde	neryl.formate
F_malt_burn	ethanol..v.v.	MeSH	H2S
F_malt_cara	acetaldehyde	ethanol..v.v.	X2.acetylpyrrole
F_malt_grain	X.E..beta.ionone	terpinen.4.ol.acetate	X3.methylbutanal
X4vg	X4VG	neryl.formate	ethyl_decanoate
acetic	ethyl.phenylacetate	prenyl.isobutyrate	alpha.muurolol
acidity	isopentyl_acetate	X4.ethyl.guaiacol	acetic_acid.g.L.
aftertaste	ethanol..v.v.	X.E..2.nonenal	ethyl_isovalerate
alcohol	ethanol..v.v.	glycerol	cis.dihydro.beta.terpineol
barnyard	X2.undecanol	alloaromadendrene.epoxide	n.hexanol
bitternes	ethyl_octanoate	ethyl_decanoate	iron
body	ethanol..v.v.	glycerol	X3.methyl.2.hexanol

感官	候选 1	候选 2	候选 3
clove	limonene	x.methyl.1.octanol	alpha.calacorene
co2	ethyl_octanoate	ethanol..v.v.	ortho.cresol
coriander	limonene	terpinen.4.ol	ethyl.furoate
diacetyl	ethyl_decanoate	eugenol	vinyl.caprylate
dms	neryl.formate	ethyl_isovalerate	ethyl.hex..2E..enoate
lactic	prenyl.isobutyrate	ethyl.phenylacetate	alpha.muurolol
metallic	ethyl_isovalerate	alpha.calacorene	iron
orange	camphor	limonene	ethyl.benzoate
overall	ethyl_acetate	ethyl_octanoate	X2.3.dihydro.farnesol
stale_hops	X2.undecanol	alloaromadendrene.epoxide	n.hexanol
sweetness	ethyl_decanoate	iron	beta.cyclohomocitral
t2n	beta.phellandrene	geraniol	vinyl.caprylate

B 附录：22 风格独立测试覆盖

风格	总样本	独立测试酒数	相关可估计场景
Amber	12	3	0
Blond	31	13	3
Brett/cofermented	6	1	0
Brown	12	5	0
Brut	4	1	0
Christmas	9	3	0
Dubbel	10	3	0
Faro	3	0	0
Flanders old brown	5	0	0
Fruitbeer	8	2	0
Hoppy	17	6	0
Kriek	10	2	0
Lambic	11	3	0
Low/No alcohol	8	2	0
Pils/Lager	11	9	1
Saison	10	6	0
Scotch	5	2	0
Stout/Porter	13	4	0
Strong ale	18	8	1
Tripel	28	12	2
West Flanders ale	7	1	0
Wheat	12	5	0

C 附录：逐风格四模型分布与联合低端结果

风格	模型	SWD	风格一致性	联合低端误差
Amber	CVAE	0.7387	0.0083	0.2079
Blond	CVAE	0.6574	0.3917	0.2767
Brett/cofermented	CVAE	0.7714	0.0000	0.2078
Brown	CVAE	0.5777	0.0250	0.1729
Brut	CVAE	0.6950	0.0000	0.2262
Christmas	CVAE	0.7076	0.0000	0.1487
Dubbel	CVAE	0.5166	0.0667	0.1104
Faro	CVAE	NA	0.0083	NA
Flanders old brown	CVAE	NA	0.0417	NA
Fruitbeer	CVAE	0.9594	0.0083	0.0734
Hoppy	CVAE	0.4888	0.0750	0.2009
Kriek	CVAE	0.9936	0.1500	0.2320
Lambic	CVAE	0.6736	0.2750	0.1741
Low/No alcohol	CVAE	1.2574	0.0417	0.3179
Pils/Lager	CVAE	0.5672	0.0917	0.1985
Saison	CVAE	0.6085	0.2083	0.0958
Scotch	CVAE	0.5619	0.0000	0.2824
Stout/Porter	CVAE	0.8762	0.1417	0.2103
Strong ale	CVAE	0.5553	0.2417	0.2125
Tripel	CVAE	0.4625	0.2083	0.1849
West Flanders ale	CVAE	0.8238	0.0167	0.1252
Wheat	CVAE	0.6248	0.0167	0.1338
Amber	独立 Copula	0.9227	0.0083	0.2758
Blond	独立 Copula	0.6490	0.5500	0.1642
Brett/cofermented	独立 Copula	1.0169	0.0000	0.2843
Brown	独立 Copula	0.7099	0.0250	0.1963
Brut	独立 Copula	1.0887	0.0000	0.2965
Christmas	独立 Copula	0.7183	0.0000	0.2042
Dubbel	独立 Copula	0.7611	0.0667	0.2151
Faro	独立 Copula	NA	0.0000	NA
Flanders old brown	独立 Copula	NA	0.0417	NA
Fruitbeer	独立 Copula	1.2148	0.1833	0.2074
Hoppy	独立 Copula	0.6500	0.1417	0.1751
Kriek	独立 Copula	1.2194	0.2500	0.3137
Lambic	独立 Copula	0.8711	0.2750	0.2079
Low/No alcohol	独立 Copula	1.4084	0.2750	0.3799
Pils/Lager	独立 Copula	0.6166	0.0917	0.1711
Saison	独立 Copula	0.7890	0.1250	0.1753
Scotch	独立 Copula	0.8316	0.0000	0.2824
Stout/Porter	独立 Copula	0.9120	0.2583	0.2075

风格	模型	SWD	风格一致性	联合低端误差
Strong ale	独立 Copula	0.6248	0.3083	0.1766
Tripel	独立 Copula	0.5582	0.3583	0.1605
West Flanders ale	独立 Copula	1.0555	0.0583	0.2372
Wheat	独立 Copula	0.7338	0.0833	0.1885
Amber	联合 Copula	0.8971	0.0083	0.2792
Blond	联合 Copula	0.6609	0.4583	0.1700
Brett/cofermented	联合 Copula	1.0231	0.0000	0.3023
Brown	联合 Copula	0.7024	0.0333	0.1925
Brut	联合 Copula	1.0451	0.0000	0.2845
Christmas	联合 Copula	0.6982	0.0167	0.1992
Dubbel	联合 Copula	0.7705	0.0750	0.2230
Faro	联合 Copula	NA	0.0000	NA
Flanders old brown	联合 Copula	NA	0.0583	NA
Fruitbeer	联合 Copula	1.2211	0.1500	0.2032
Hoppy	联合 Copula	0.6545	0.2000	0.1811
Kriek	联合 Copula	1.1689	0.2250	0.3232
Lambic	联合 Copula	0.8093	0.3250	0.2191
Low/No alcohol	联合 Copula	1.4473	0.2667	0.3821
Pils/Lager	联合 Copula	0.6234	0.1000	0.1680
Saison	联合 Copula	0.7798	0.0833	0.1793
Scotch	联合 Copula	0.8788	0.0083	0.2876
Stout/Porter	联合 Copula	0.9023	0.2833	0.2079
Strong ale	联合 Copula	0.6597	0.2417	0.1802
Tripel	联合 Copula	0.5654	0.3250	0.1624
West Flanders ale	联合 Copula	1.0635	0.0333	0.2441
Wheat	联合 Copula	0.7029	0.0917	0.1879
Amber	PCA 高斯	0.8006	0.0000	0.2089
Blond	PCA 高斯	0.6428	0.3250	0.2671
Brett/cofermented	PCA 高斯	0.8298	0.0000	0.2073
Brown	PCA 高斯	0.6308	0.0083	0.1722
Brut	PCA 高斯	0.7940	0.0000	0.2231
Christmas	PCA 高斯	0.6758	0.0000	0.1486
Dubbel	PCA 高斯	0.6490	0.0500	0.1121
Faro	PCA 高斯	NA	0.0000	NA
Flanders old brown	PCA 高斯	NA	0.0417	NA
Fruitbeer	PCA 高斯	1.0009	0.0750	0.0759
Hoppy	PCA 高斯	0.5465	0.0583	0.1985
Kriek	PCA 高斯	1.0456	0.1250	0.2327
Lambic	PCA 高斯	0.7526	0.2167	0.1714
Low/No alcohol	PCA 高斯	1.2436	0.3083	0.3200
Pils/Lager	PCA 高斯	0.5815	0.0167	0.1952
Saison	PCA 高斯	0.6715	0.0667	0.0977

风格	模型	SWD	风格一致性	联合低端误差
Scotch	PCA 高斯	0.6488	0.0000	0.2801
Stout/Porter	PCA 高斯	0.8926	0.0750	0.2093
Strong ale	PCA 高斯	0.5760	0.2250	0.2092
Tripel	PCA 高斯	0.4776	0.1833	0.1808
West Flanders ale	PCA 高斯	0.9279	0.0333	0.1287
Wheat	PCA 高斯	0.6436	0.0667	0.1352

D　附录：9 酒四策略逐项结果

源 beer	策略	目标增益	stale 变化	化学 RMS	拒绝
81	无约束	0.0630	0.0000	0.6928	False
81	点约束	0.0185	-0.0533	0.5890	False
81	不确定性约束	0.0120	-0.0176	0.6868	False
81	拒绝迁移	0.0000	0.0000	0.0000	True
124	无约束	0.0407	0.0115	0.6000	False
124	点约束	0.0426	0.0152	0.5979	False
124	不确定性约束	0.0269	0.0192	0.7807	False
124	拒绝迁移	0.0000	-0.0000	0.0000	True
218	无约束	-0.0046	0.0241	0.5655	False
218	点约束	-0.0028	0.0255	0.5654	False
218	不确定性约束	-0.0296	-0.0459	0.7688	False
218	拒绝迁移	0.0000	-0.0000	0.0000	True
96	无约束	0.0731	0.0472	0.6549	False
96	点约束	0.0731	0.0458	0.6549	False
96	不确定性约束	0.0343	0.0503	0.6290	False
96	拒绝迁移	0.0000	0.0000	0.0000	True
185	无约束	0.0759	0.0209	0.6535	False
185	点约束	0.0731	0.0216	0.6511	False
185	不确定性约束	0.0731	0.0216	0.6511	False
185	拒绝迁移	0.0000	0.0000	0.0000	True
230	无约束	0.0407	0.0451	0.6475	False
230	点约束	0.0407	0.0446	0.6470	False
230	不确定性约束	0.0407	0.0446	0.6470	False
230	拒绝迁移	0.0000	0.0000	0.0000	True
119	无约束	0.1028	-0.0239	0.5458	False
119	点约束	0.1028	-0.0252	0.5442	False
119	不确定性约束	0.1028	-0.0252	0.5442	False
119	拒绝迁移	0.0000	0.0000	0.0000	True
139	无约束	0.0361	-0.1868	0.5808	False
139	点约束	0.0213	-0.2204	0.5901	False

源 beer	策略	目标增益	stale 变化	化学 RMS	拒绝
139	不确定性约束	0.0000	0.0000	0.0000	True
139	拒绝迁移	0.0000	0.0000	0.0000	True
209	无约束	0.0352	-0.0224	0.6311	False
209	点约束	0.0935	-0.0710	0.6468	False
209	不确定性约束	0.0000	0.0000	0.0000	True
209	拒绝迁移	0.0000	0.0000	0.0000	True

E　复现环境与全部新增程序

以下为本轮新增计算程序，均含 AI 辅助说明。原始数据未修改。运行命令：Q2/Q3 封闭实验与论文重生成见 scripts/run_final.ps1（scripts/run_rebuild.ps1 为其兼容转发入口）；Q1 从源码完整重建见 scripts/run_q1.ps1。脚本依赖见 configs/rebuild_requirements.txt。

E.1　audit_extended.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Audit extensions; no model selection."""
from run import *
from scipy.stats import kruskal

def main():
 rows=[]
 for b,d in raw.groupby('beer_name'):
  if d.date.nunique()<2:continue
  for c in targets:
   g=[v[c].to_numpy() for _,v in d.groupby('date')];p=kruskal(*g).pvalue if d[c].nunique()>1 else
   ↪  1;rows.append(dict(raw_beer_name=b,beer=int(xdf.beer.iloc[b-1]),target=c,n_dates=len(g),p=p))
 order=np.argsort([r['p'] for r in rows]);last=0
 for k,i in enumerate(order):last=max(last,min(1,rows[i]['p']*(len(rows)-k)));rows[i]['p_holm']=last
 csv(rows,'audit/repeated_sessions_tests.csv')
 # Recompute numerical correlation definition against supplied named matrix.
 names=pd.read_csv(R/'data/processed/chemical_names.csv').set_index('feature').matrix_name;mat=pd.read_csv(raw_path('2.csv'),ind
↪  ex_col=0).loc[[names[c] for c in chem],[names[c] for c in
↪  chem]].to_numpy();sp=pd.DataFrame(X).corr(method='spearman').to_numpy();pe=pd.DataFrame(X).corr(method='pearson').to_numpy(
↪  );js({'matrix_shape':list(mat.shape),'spearman_max_abs_error':float(np.nanmax(abs(mat-sp))),'pearson_mean_abs_error':float(
↪  np.nanmean(abs(mat-pe))),'mapping_unique':bool(names.is_unique)},'audit/correlation_recheck.json')
 # Joint floor concentration means are not known LOD values; only verified transforms exported.
 tr=pd.read_csv(R/'data/processed/transforms.csv');g=pd.read_csv(O/'q3/generated_22styles_chemical231.csv');out={'style':g['styl
↪  e']};un=[]
 for _,r in tr.iterrows():
  if r['transform']=='log10':out[r.feature]=10**g[r.feature]
  elif str(r['transform']).startswith('identity'):out[r.feature]=g[r.feature]
  else:un.append(r.feature)
 csv(pd.DataFrame(out),'q3/generated_calibrated_physical.csv');csv(tr,'q3/physical_units_and_transforms.csv')
 # Scorecard deliverable: all 50 fields of 3.xlsx.  Earlier versions of this line filtered columns
 # by the A_/F_ prefix, which silently dropped the 20 unprefixed fields (body, bitternes, co2,
 # aftertaste, alcohol, sweetness, acidity, dms, metallic, 4vg, t2n, diacetyl, stale_hops, orange,
 # coriander, clove, lactic, acetic, barnyard, overall).  The two acetaldehyde columns carry the
 # threshold-anchored grade from q1_acetaldehyde.py when it has been run, otherwise they stay empty;
 # see outputs/rebuild/q1_complete/acetaldehyde_grading.json for the evidence behind that grade.
 from q1_complete import complete_scorecard
 complete_scorecard(with_acetaldehyde=True).to_csv(O/'q1/aroma_flavour_template_with_unobserved.csv',index=False,encoding='utf-8
↪  -sig')
 # Shared-feature outliers explain validation-distance sensitivity; detection uses training scale only.
 sp=pd.read_csv(O/'q3/splits.csv');rows=[]
 for s in C['q3_seeds']:
```

```
  ti=sp[(sp.seed==s)&(sp.role=='generator')].row.to_numpy();vi=sp[(sp.seed==s)&(sp.role=='validation')].row.to_numpy();p=Prep().
  ↪ fit(X[ti]);a=p.transform(X[vi]);loc=np.unravel_index(np.argmax(abs(a)),a.shape);rows.append(dict(seed=s,beer=int(xdf.beer.
  ↪ iloc[vi[loc[0]]]),feature=chem[loc[1]],largest_abs_training_scaled_value=float(abs(a[loc])),raw_value=float(X[vi[loc[0]],l
  ↪ oc[1]])))
 csv(rows,'audit/validation_extremes.csv');log('extended audits complete')
if __name__=='__main__':main()
```

E.2 closed.py

```
"""AI-assisted: OpenAI Codex gpt-6.1-sol, 2026-10-28. Exactly three closed supplements."""
from run import *
from pathlib import Path
import joblib,itertools
from q2 import select,unit,residual,allowed
from q3 import Generator,ST,SI
F=O/'closed'
METHODS=C['q2_methods']
def put(d,p):pd.DataFrame(d).to_csv(F/p,index=False,encoding='utf-8-sig')
def dump(d,p):(F/p).write_text(json.dumps(d,ensure_ascii=False,indent=2,default=lambda v:v.item() if hasattr(v,'item') else
↪ str(v)),encoding='utf-8')
def frozen_files():
 return [R/'src/rebuild/run.py',R/'reports/01_q1_analysis.md',R/'configs/rebuild_protocol.json',O/'q3/splits.csv']+list((O/'q1')
 ↪ .glob('*'))
def freeze():
 files={str(p.relative_to(R)):hashlib.sha256(p.read_bytes()).hexdigest() for p in frozen_files() if p.is_file()}
 protocol={'closed_experiments':['Q2 four-method nested selection and bootstrap','Q3 22-style assessment of existing four
 ↪ models','Q3 four-policy migration on nine unique held-out sources'],'frozen_hashes':files,'Q1':'No retraining or output
 ↪ rewriting','Q2':'Original two seeds and three outer folds; global method chosen by equal-target raw-mean MAE in original
 ↪ inner 20% holdout; no added candidates; alpha8; resample styles100 times; bootstrap duplicates kept in one inner role. Final
 ↪ full refit repeats the same inner-choice rule with first seed.','generation':'Original 140/35/40/35 roles and four
 ↪ configurations. 40 generated per style per context; same latent random-seed reset across models; all22styles listed even if
 ↪ n_test=0; rank correlation NA if n_test<4; common eligible correlation pairs for comparability; shrinkage and floors
 ↪ determined on generator training only. Validation distances cannot be reinterpreted as test evidence.','migration':'Nine
 ↪ unique beers from prior held-out source set, first eligible original seed per beer; same fixed independent evaluator within
 ↪ each context, serialized before optimization; original model selected in existing selected.csv. 80 candidates, four alphas
 ↪ and identity; four policies Unconstrained/PointConstraint/UncertaintyConstraint/Reject; all evaluation occurs after
 ↪ candidate choices.','inference':'2000 paired beer bootstrap; Wilcoxon Holm correction; nine sources, no duplicate counting;
 ↪ intervals describe internal sample variation, not external validation.','stop_rule':'After these three groups, stop all
 ↪ model exploration regardless of results.'}
 p=R/'configs/closed_final_protocol.json'
 if p.exists():
  old=json.loads(p.read_text(encoding='utf-8'));assert old['frozen_hashes']==files,'Frozen Q1 or original split changed'
 else:p.write_text(json.dumps(protocol,ensure_ascii=False,indent=2),encoding='utf-8')
 log('closed protocol frozen; Q1 hashes recorded')
def paired(rows,truth,reference=None):
 rng=np.random.default_rng(3481);loss={k:abs(v-truth).mean(1) for k,v in rows.items()};out=[]
 pairs=[(k,reference) for k in rows if k!=reference] if reference else list(itertools.combinations(rows,2))
 for m,n in pairs:
  d=loss[m]-loss[n];b=d[rng.integers(len(d),size=(2000,len(d)))].mean(1);out.append(dict(method=m,reference=n,loss_difference=d.
  ↪ mean(),CI_low=np.quantile(b,.025),CI_high=np.quantile(b,.975),p=wilcoxon(d).pvalue if np.any(d) else 1))
 holm(out);return out
def holm(rows):
 last=0
 for k,i in enumerate(np.argsort([r['p'] for r in rows])):last=max(last,min(1,rows[i]['p']*(len(rows)-k)));rows[i]['p_holm']=last

def fit_predict(indices,test,method):
 pre=Prep().fit(X[indices]);a=pre.transform(X[indices]);b=pre.transform(X[test]);idx,scores=select(a,Y[indices],method);p=np.zer
 ↪ os((len(test),50))
 for j in range(50):p[:,j]=np.clip(Ridge(alpha=8).fit(a[:,idx[j]],Y[indices,j]).predict(b[:,idx[j]]),0,6)
 return p,idx,scores

def choose(indices,s):
 unique=np.unique(indices);it,iv=inner(unique,s);tr=indices[np.isin(indices,it)];va=indices[np.isin(indices,iv)];scores={}
 assert not set(tr)&set(va)
 for m in METHODS:scores[m]=float(abs(fit_predict(tr,va,m)[0]-Y[va]).mean())
 return min(METHODS,key=lambda m:scores[m]),scores

def q2_closed():
```

```
 freeze();foldrows=[];choices=[];sel=[];preds={m:[] for m in METHODS+['TrainSelected']}
 for s in C['seeds']:
  f=folds(s);out={m:np.zeros_like(Y) for m in preds}
  for k in range(3):
   tr=np.flatnonzero(f!=k);te=np.flatnonzero(f==k);winner,sc=choose(tr,s+k);choices.append(dict(seed=s,fold=k,winner=winner,**sc))
   for m in METHODS:
    p,ix,score=fit_predict(tr,te,m);out[m][te]=p
    for j,t in enumerate(targets):
     for rank,v in
     ↪  enumerate(ix[j]):sel.append(dict(seed=s,fold=k,method=m,target=t,rank=rank+1,chemical=chem[v],score=score[j,rank]))
   out['TrainSelected'][te]=out[winner][te]
   for m in out:foldrows.append(dict(seed=s,fold=k,method=m,n_test=len(te),**metric(Y[te],out[m][te])))
  for m in out:preds[m].append(out[m]);put(pd.DataFrame(out[m],columns=targets).assign(beer=xdf.beer),f'q2/oof_{m}_{s}.csv')
  log(f'Closed Q2 outer seed {s} complete')
 put(foldrows,'q2/fold_metrics.csv');put(choices,'q2/method_choice_inner.csv');put(sel,'q2/fold_selections.csv');repeat=[]
 for m,pp in preds.items():
  for s,p in zip(C['seeds'],pp):repeat.append(dict(method=m,seed=s,**metric(Y,p)))
 put(repeat,'q2/repeat_metrics.csv');put(paired({m:np.mean(p,0) for m,p in
↪  preds.items()},Y,'TrainSelected'),'q2/paired_tests.csv')
 # Final refit uses exactly the same inner method-choice algorithm as outer evaluation.
 winner,final_inner=choose(np.arange(250),C['seeds'][0]);a=Prep().fit(X).transform(X);finals={m:select(a,Y,m) for m in
↪  METHODS};finals['TrainSelected']=finals[winner];groups=pd.read_csv(O/'q2/alternative_groups.csv').alternative_group.to_nump
↪  y();rho=pd.DataFrame(X).corr(method='spearman').to_numpy();assoc=unit(rankdata(a,axis=0)).T@unit(rankdata(Y,axis=0));adjass
↪  oc=unit(rankdata(residual(a,style),axis=0)).T@unit(rankdata(residual(Y,style),axis=0));rng=np.random.default_rng(C['seeds']
↪  [0]);boots={m:[] for m in finals};bw=[]
 for b in range(100):
  ids=np.concatenate([rng.choice(np.flatnonzero(style==st),sum(style==st),replace=True) for st in ST]);aa=Prep().fit(X[ids]).tra
  ↪  nsform(X[ids]);pick,score=choose(ids,C['seeds'][0]+b);bw.append(dict(bootstrap=b,winner=pick,**score))
  for m in METHODS:boots[m].append(select(aa,Y[ids],m)[0])
  boots['TrainSelected'].append(boots[pick][-1])
  if (b+1)%20==0:log(f'Closed Q2 grouped bootstrap {b+1}/100')
 put(bw,'q2/bootstrap_method_choice.csv');np.savez_compressed(F/'q2/bootstrap_selections.npz',**{m:np.array(v) for m,v in
↪  boots.items()});top=[];stability=[];cross=[]
 for m,(ix,score) in finals.items():
  bs=np.array(boots[m])
  for j,t in enumerate(targets):
   # Fixed 100 independent pairs, sampled once; equal protocol across methods.
   pairlist=[(b,(b+37)%100) for b in range(100)];cj=[];gj=[]
   for u,v in pairlist:
    su,sv=set(bs[u,j]),set(bs[v,j]);gu,gv=set(groups[list(su)]),set(groups[list(sv)]);cj.append(len(su&sv)/len(su|sv));gj.append
    ↪  (len(gu&gv)/len(gu|gv))
   redundancy=float(np.mean([abs(rho[ix[j,u],ix[j,v]]) for u,v in [(0,1),(0,2),(1,2)]]));stability.append(dict(method=m,target=t
   ↪  ,compound_jaccard=np.mean(cj),group_jaccard=np.mean(gj),within_top3_abs_rho=redundancy))
   for rank,v in enumerate(ix[j]):
    fixed=np.array(boots[winner]);top.append(dict(method=m,final_fixed_method=winner if m=='TrainSelected' else
    ↪  m,target=t,rank=rank+1,chemical=chem[v],selection_score=score[j,rank],spearman=assoc[v,j],style_adjusted_rank_associatio
    ↪  n=adjassoc[v,j],bootstrap_frequency=float((bs[:,j,:]==v).any(1).mean()),alternative_group=int(groups[v]),group_frequency
    ↪  =float((groups[bs[:,j,:]]==groups[v]).any(1).mean()),final_fixed_method_frequency=float((fixed[:,j,:]==v).any(1).mean())
    ↪  ,label_flag='suspected formula alias' if t=='barnyard' else ''))
   for m2,(v,_) in
   ↪  finals.items():u0,v0=set(ix[j]),set(v[j]);cross.append(dict(method=m,other=m2,target=t,jaccard=len(u0&v0)/len(u0|v0)))
 put(top,'q2/top3_all_methods.csv');final=pd.DataFrame([r for r in top if r['method']=='TrainSelected']);put(final,'q2/top3_50x3
↪  .csv');put(stability,'q2/stability.csv');put(cross,'q2/method_jaccard.csv')
 old=O/'q2/top3_50x3.csv';archive=F/'q2/previous_forward3_top3.csv'
 if not archive.exists():archive.write_bytes(old.read_bytes())
 final.to_csv(old,index=False,encoding='utf-8-sig')
 scores=pd.DataFrame(repeat).groupby('method')[['MAE','RMSE','R2']].agg(['mean','std']);scores.columns=['_'.join(v) for v in
↪  scores.columns];put(scores.reset_index(),'q2/summary_metrics.csv');dump({'final_method':winner,'final_inner_scores':final_i
↪  nner,'rows':len(final),'evaluation_method':'TrainSelected outer OOF; final fixed algorithm is selected inside full-data
↪  training only','bootstrap':'100 style-stratified beer bootstraps; duplicate source beer cannot cross inner
↪  training/validation'},'q2/summary.json')
 log('Closed experiment 1/3 complete')

def eval_bundle(ev,s):
 pre=Prep().fit(X[ev]);a=pre.transform(X[ev]);clf=ExtraTreesClassifier(n_estimators=180,min_samples_leaf=2,max_features=.7,n_job
↪  s=2,random_state=s).fit(a,style[ev]);reg=ExtraTreesRegressor(n_estimators=180,min_samples_leaf=2,max_features=.7,n_jobs=2,r
↪  andom_state=s).fit(a,Y[ev]);rng=np.random.default_rng(s+610);boot=[]
 for k in range(12):
  ids=rng.integers(len(ev),size=len(ev));boot.append(ExtraTreesRegressor(n_estimators=60,min_samples_leaf=3,max_features=.7,n_jo
  ↪  bs=2,random_state=k+s).fit(a[ids],Y[ev[ids]]))
```

```
 return {'pre':pre,'classifier':clf,'regressor':reg,'bootstrap':boot,'training_rows':ev,'seed':s,'scope':'Independent evaluation
 ↪  only; optimization never receives this bundle'}

def conditional_metrics(real,g,model,evaluator,st,scale_projection):
 pre=model.pre;b=pre.transform(g);n=len(real);pred=evaluator['classifier'].predict(evaluator['pre'].transform(g));classes=list(e
 ↪  valuator['classifier'].classes_);prob=evaluator['classifier'].predict_proba(evaluator['pre'].transform(g))[:,classes.index(
 ↪  st)]
 nn=cdist(b,pre.transform(model.a))/np.sqrt(231);d=dict(n_test=n,style_accuracy=float(np.mean(pred==st)),style_score=float(prob.
 ↪  mean()),nearest_train_RMS=float(nn.min(1).mean()),copy_fraction=float((nn.min(1)<1e-8).mean()),diversity=float(np.std(b,axi
 ↪  s=0).mean()),SWD=np.nan,marginal_W1=np.nan,correlation_MAE=np.nan,correlation_pairs=0,floor_mass_MAE=np.nan,joint_floor_MAE
 ↪  =np.nan,correlation_status='n_test<4' if n<4 else 'computed')
 if not n:d['correlation_status']='no independent test beer';return d
 a=pre.transform(real);d['SWD']=float(np.mean([wasserstein_distance(a@scale_projection[:,k],b@scale_projection[:,k]) for k in
 ↪  range(48)]));d['marginal_W1']=float(np.mean([wasserstein_distance(a[:,j],b[:,j]) for j in range(231)]))
 lo=model.low;ids=model.floor;ra=(pre.imp.transform(real)[:,ids]<=lo[ids]+1e-10).astype(float);gb=(g[:,ids]<=lo[ids]+1e-10).asty
 ↪  pe(float);d['floor_mass_MAE']=float(abs(ra.mean(0)-gb.mean(0)).mean());mask=np.triu(np.ones((sum(ids),sum(ids)),bool),1);d[
 ↪  'joint_floor_MAE']=float(abs(ra.T@ra/n-gb.T@gb/len(g))[mask].mean())
 if n>=4:
  ca=pd.DataFrame(a).corr(method='spearman').to_numpy();cg=pd.DataFrame(b).corr(method='spearman').to_numpy();mask=np.triu(np.on
  ↪  es((231,231),bool),1)&np.isfinite(ca)&np.isfinite(cg);d['correlation_pairs']=int(mask.sum());d['correlation_MAE']=float(ab
  ↪  s(ca[mask]-cg[mask]).mean()) if mask.any() else np.nan
 return d


def finalize_generation_metrics():
 sp=pd.read_csv(O/'q3/splits.csv');path=F/'generation/metrics_22styles_by_context.csv';df=pd.read_csv(path)
 for s in C['q3_seeds']:
  tr=sp[(sp.seed==s)&(sp.role=='generator')].row.to_numpy(int);te=sp[(sp.seed==s)&(sp.role=='test')].row.to_numpy(int);pre=Prep(
  ↪  ).fit(X[tr]);files={m:pd.read_csv(F/f'generation/generated_{m}_{s}.csv') for m in C['q3_models']}
  for st in ST:
   ids=te[style[te]==st]
   if len(ids)<4:continue
   real=pd.DataFrame(pre.transform(X[ids])).corr(method='spearman').to_numpy();cor={m:pd.DataFrame(pre.transform(g.loc[g['style'
   ↪  ]==st,chem].to_numpy())).corr(method='spearman').to_numpy() for m,g in
   ↪  files.items()};mask=np.triu(np.ones((231,231),bool),1)&np.isfinite(real)
   for ca in cor.values():mask&=np.isfinite(ca)
   for m,ca in cor.items():
    ix=(df.seed==s)&(df['style']==st)&(df.model==m);df.loc[ix,'correlation_MAE']=float(abs(ca[mask]-real[mask]).mean()) if
    ↪  mask.any() else np.nan;df.loc[ix,'correlation_pairs']=int(mask.sum());df.loc[ix,'correlation_status']='common variable
    ↪  pairs across all four generators'
 put(df,'generation/metrics_22styles_by_context.csv');summ=df.groupby(['model','style'])[['SWD','marginal_W1','style_accuracy','
 ↪  style_score','correlation_MAE','joint_floor_MAE','floor_mass_MAE','nearest_train_RMS','copy_fraction','diversity']].agg(['m
 ↪  ean','std']);summ.columns=['_'.join(v) for v in
 ↪  summ.columns];put(summ.reset_index(),'generation/metrics_22styles_summary.csv')
 macro=summ.groupby(level=0).mean();macro.columns=['_'.join(v) if isinstance(v,tuple) else v for v in
 ↪  macro.columns];put(macro.reset_index(),'generation/macro_summary.csv');log('Common-pair correlations finalized without model
 ↪  refitting')

def generation_closed():
 freeze();sp=pd.read_csv(O/'q3/splits.csv');rows=[];evrows=[];manifest=[];curves=[];rng=np.random.default_rng(801);proj=rng.norm
 ↪  al(size=(231,48));proj/=np.linalg.norm(proj,axis=0)
 for s in C['q3_seeds']:
  roles={role:sp[(sp.seed==s)&(sp.role==role)].row.to_numpy(int) for role in
  ↪  ['generator','validation','evaluator','test']};tr,va,ev,te=[roles[k] for k in
  ↪  ['generator','validation','evaluator','test']];e=eval_bundle(ev,s);joblib.dump(e,F/f'models/evaluator_{s}.joblib');evpred=
  ↪  e['classifier'].predict(e['pre'].transform(X[te]));evrows.append(dict(seed=s,n_evaluator=len(ev),n_test=len(te),real_accur
  ↪  acy=float(np.mean(evpred==style[te]))));manifest.append(dict(seed=s,training_rows=ev.tolist(),sha256=hashlib.sha256((F/f'm
  ↪  odels/evaluator_{s}.joblib').read_bytes()).hexdigest()))
  for m in C['q3_models']:
   gen=Generator(m,s).fit(X[tr],style[tr],X[va],style[va]);joblib.dump(gen,F/f'models/generator_{m}_{s}.joblib');allg=[]
   for k,st in enumerate(ST):
    g=gen.sample(st,40,np.random.default_rng(s+100+k));allg.append(pd.DataFrame(g,columns=chem).assign(style=st));real=X[te[styl
    ↪  e[te]==st]];d=conditional_metrics(real,g,gen,e,st,proj);rows.append(dict(seed=s,model=m,style=st,n_train_style=int(sum(s
    ↪  tyle[tr]==st)),n_evaluator_style=int(sum(style[ev]==st)),**d))
   put(pd.concat(allg,ignore_index=True),f'generation/generated_{m}_{s}.csv')
   for h in gen.history:curves.append(dict(seed=s,model=m,**h))
   log(f'Closed Q3 22-style generation {s} {m}')
 put(rows,'generation/metrics_22styles_by_context.csv');put(evrows,'generation/evaluator_real_accuracy.csv');put(curves,'generat
 ↪  ion/training_curves.csv');dump(manifest,'models/evaluator_manifest.json')
```

```
 df=pd.DataFrame(rows);summary=df.groupby(['model','style'])[['SWD','marginal_W1','style_accuracy','style_score','correlation_MA
↪  E','joint_floor_MAE','floor_mass_MAE','nearest_train_RMS','copy_fraction','diversity']].agg(['mean','std']);summary.columns
↪  =['_'.join(v) for v in summary.columns];put(summary.reset_index(),'generation/metrics_22styles_summary.csv');coverage=[]
 for st in ST:
  ids=sp[(sp.role=='test')&(sp['style']==st)].row.unique();coverage.append(dict(style=st,n_total=int(sum(style==st)),n_unique_te
  ↪  st=len(ids),n_contexts_with_test=int(df[(df.model==C['q3_models'][0])&(df['style']==st)].n_test.gt(0).sum()),n_correlation
  ↪  _contexts=int(df[(df.model==C['q3_models'][0])&(df['style']==st)].correlation_pairs.gt(0).sum()),distribution_status='test
  ↪  evidence available' if len(ids) else 'NA: no held-out beer in frozen protocol'))
 put(coverage,'generation/style_coverage.csv');dump({'rows':len(df),'styles':len(ST),'models':C['q3_models'],'scope':'Same
↪  original roles. No-test styles retain NA distribution evidence; n<4 correlations not estimated. Style classifier scoring is
↪  diagnostic, never generator selection. Existing selected.csv remains
↪  frozen.'},'generation/summary.json');finalize_generation_metrics();log('Closed experiment 2/3 complete')


def migration_closed():
 freeze();sp=pd.read_csv(O/'q3/splits.csv');prior=pd.read_csv(O/'q3/migration_metrics.csv');sources=prior.groupby('beer').seed.m
↪  in().to_dict();assert len(sources)==9
 chosen=pd.read_csv(O/'q3/selected.csv').set_index('seed').model.to_dict();rows=[];spectra=[];check=[];source_table=[];choices_l
↪  og=[];sj=targets.index('stale_hops');dc=['DMS','X2.3.butanedione','X4VG','X.E..2.nonenal'];dj=[chem.index(v) for v in
↪  dc];bundle_hashes=[]
 for s in C['q3_seeds']:
  tr=sp[(sp.seed==s)&(sp.role=='generator')].row.to_numpy(int);ev=sp[(sp.seed==s)&(sp.role=='evaluator')].row.to_numpy(int);te=s
  ↪  p[(sp.seed==s)&(sp.role=='test')].row.to_numpy(int);pre=Prep().fit(X[tr]);a=pre.transform(X[tr]);gen=joblib.load(F/f'model
  ↪  s/generator_{chosen[s]}_{s}.joblib');epath=F/f'models/evaluator_{s}.joblib';h0=hashlib.sha256(epath.read_bytes()).hexdiges
  ↪  t();design=[];rng=np.random.default_rng(s)
  for k in range(12):
   ii=rng.integers(len(tr),size=len(tr));design.append(ExtraTreesRegressor(n_estimators=60,min_samples_leaf=3,max_features=.7,n_
   ↪  jobs=2,random_state=s+k).fit(a[ii],Y[tr[ii]]))
  joblib.dump({'pre':pre,'models':design,'train_rows':tr},F/f'models/design_ensemble_{s}.joblib')
  for beer,context in sorted(sources.items()):
   if context!=s:continue
   i=int(np.flatnonzero(xdf.beer.to_numpy()==beer)[0]);assert i in te and i not in tr and i not in ev
   source=gen.pre.imp.transform(X[[i]])[0];cand0=gen.sample('Tripel',80,np.random.default_rng(s+int(beer)));target=gen.pre.trans
   ↪  form(cand0).mean(0);zs=gen.pre.transform(source[None])[0];base_design=np.array([m.predict(pre.transform(source[None]))[0,
   ↪  sj] for m in design]);selections={};risks={};costs={}
   for policy in ['Unconstrained','PointConstraint','UncertaintyConstraint','Reject']:
    if policy=='Reject':selections[policy]=source.copy();risks[policy]=0.;costs[policy]=float(np.linalg.norm(zs-target));continue
    cand=np.vstack([source]+[source+alpha*(cand0-source) for alpha in [.25,.5,.75,1]])
    if policy!='Unconstrained':
     for j in dj:
      if np.isfinite(X[i,j]):cand[:,j]=np.minimum(cand[:,j],X[i,j])
    changes=np.array([m.predict(pre.transform(cand))[:,sj]-base_design[k] for k,m in enumerate(design)]);risk=changes.mean(0) if
    ↪  policy=='PointConstraint' else np.quantile(changes,.9,axis=0);cost=np.linalg.norm(gen.pre.transform(cand)-target,axis=1)
    ↪  +.15*np.linalg.norm(gen.pre.transform(cand)-zs,axis=1);valid=np.ones(len(cand),bool) if policy=='Unconstrained' else
    ↪  risk<=1e-10;cost[~valid]=np.inf;k=int(np.argmin(cost));selections[policy]=cand[k];risks[policy]=float(risk[k]);costs[pol
    ↪  icy]=float(cost[k]);choices_log.append(dict(beer=beer,seed=s,policy=policy,selected_candidate=k,feasible_candidates=int(
    ↪  valid.sum()),budget=len(cand)))
   # Only now load and query the independent evaluator, after all four choices are fixed.
   e=joblib.load(epath);b=e['pre'].transform(source[None]);tj=list(e['classifier'].classes_).index('Tripel');bp=float(e['classif
   ↪  ier'].predict_proba(b)[0,tj]);by=float(e['regressor'].predict(b)[0,sj]);bb=np.array([m.predict(b)[0,sj] for m in
   ↪  e['bootstrap']]);source_table.append(dict(beer=beer,row=i,seed=s,n_generator=len(tr),n_evaluator=len(ev),source_role='tes
   ↪  t',generator=chosen[s],evaluator_sha256=h0))
   for policy,out in selections.items():
    z=e['pre'].transform(out[None]);prob=float(e['classifier'].predict_proba(z)[0,tj]);yy=float(e['regressor'].predict(z)[0,sj])
    ↪  ;boot=np.array([m.predict(z)[0,sj] for m in
    ↪  e['bootstrap']])-bb;diff=gen.pre.transform(out[None])[0]-zs;unknown=int(np.isnan(X[i,dj]).sum());atfloor=int(np.sum(np.i
    ↪  sfinite(X[i,dj])&(X[i,dj]<=gen.low[dj]+1e-10)));chemical_inc=int(sum(np.isfinite(X[i,j]) and out[j]>X[i,j]+1e-10 for j in
    ↪  dj));refused=bool(np.linalg.norm(out-source)<1e-10)
    rows.append(dict(beer=beer,seed=s,policy=policy,target_score_before=bp,target_score_after=prob,target_gain=prob-bp,stale_bef
    ↪  ore=by,stale_after=yy,stale_change=yy-by,independent_stale_p90=float(np.quantile(boot,.9)),independent_stale_increase=yy
    ↪  -by>1e-10,design_risk=risks[policy],change_RMS=float(np.sqrt(np.mean(diff**2))),changed_features=int(np.sum(abs(out-sour
    ↪  ce)>1e-10)),distance_before=float(np.linalg.norm(zs-target)),distance_after=float(np.linalg.norm(gen.pre.transform(out[N
    ↪  one])[0]-target)),refused=refused,chemical_observed_increases=chemical_inc,missing_markers=unknown,floor_markers=atfloor
    ↪  ,candidate_cost=costs[policy]))
    spectra.append(dict(beer=beer,seed=s,policy=policy,**dict(zip(chem,out))));
    for c,j in zip(dc,dj):check.append(dict(beer=beer,seed=s,policy=policy,defect=c,source=X[i,j],generated=out[j],encoded_nonin
    ↪  crease=bool(out[j]<=X[i,j]+1e-10) if np.isfinite(X[i,j]) else None,physical_status='unobserved_source' if not
    ↪  np.isfinite(X[i,j]) else 'floor_below_unknown' if X[i,j]<=gen.low[j]+1e-10 else
    ↪  'observed_encoding_order',risk_kind='direct observed chemical'))
    check.append(dict(beer=beer,seed=s,policy=policy,defect='stale_hops',source=by,generated=yy,encoded_nonincrease=yy<=by+1e-10
    ↪  ,physical_status='no unique chemical threshold; independent prediction only',risk_kind='independent sensory proxy'))
```

```
  log(f'Closed Q3 source beer={beer} four-policy comparison complete')
 assert hashlib.sha256(epath.read_bytes()).hexdigest()==h0;bundle_hashes.append(dict(seed=s,sha256=h0,unchanged=True))
put(rows,'migration/per_beer.csv');put(spectra,'migration/spectra.csv');put(check,'migration/defect_checks.csv');put(source_tab
↪ le,'migration/nine_sources.csv');put(choices_log,'migration/candidate_choices.csv');dump(bundle_hashes,'migration/evaluator
↪ _hash_check.json');df=pd.DataFrame(rows);rng=np.random.default_rng(541);summary=[];test=[]
for policy,d in df.groupby('policy'):
 d=d.sort_values('beer');assert len(d)==9 and d.beer.nunique()==9
 for col in ['target_gain','stale_change','independent_stale_p90','change_RMS','refused','independent_stale_increase','chemical
↪ _observed_increases']:
  v=d[col].to_numpy(float);boot=v[rng.integers(9,size=(2000,9))].mean(1);summary.append(dict(policy=policy,metric=col,n_beers=9
↪ ,mean=float(v.mean()),CI_low=float(np.quantile(boot,.025)),CI_high=float(np.quantile(boot,.975))))
 if policy!='Reject':
  for col in ['target_gain','stale_change']:
   v=d[col].to_numpy();test.append(dict(policy=policy,reference='Reject',metric=col,mean_difference=float(v.mean()),p=wilcoxon(
↪ v).pvalue if np.any(v) else 1))
holm(test);put(summary,'migration/summary.csv');put(test,'migration/paired_tests.csv');dump({'n_unique_sources':9,'policies':4,
↪ 'rows':len(df),'independent_evaluator':'serialized and unchanged; per original context, same bundle for all four
↪ policies','stop_model_exploration':True},'migration/summary.json');log('Closed experiment 3/3 complete; model exploration
↪ stopped')


if __name__=='__main__':
 if '--freeze' in sys.argv:freeze()
 elif '--q2' in sys.argv:q2_closed()
 elif '--generation' in sys.argv:generation_closed()
 elif '--migration' in sys.argv:migration_closed()
```

E.3 closed_figures.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Plots from closed CSVs only."""
from closed import *
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize
plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['Arial','DejaVu Sans'],'font.size':10,'axes.spines.top':False
↪ ,'axes.spines.right':False,'axes.linewidth':.8,'svg.fonttype':'none','pdf.fonttype':42})
COL={'Spearman':'#9BAABB','Hybrid2plus1':'#3775BA','Forward3':'#42949E','Redundancy':'#9A4D8E','TrainSelected':'#B64342','PCA_Ga
↪ ussian':'#9BAABB','CopulaIndependent':'#3775BA','CopulaJoint':'#42949E','CVAE':'#B64342','Unconstrained':'#9BAABB','PointCon
↪ straint':'#3775BA','UncertaintyConstraint':'#42949E','Reject':'#B64342'}
SHORT={'PCA_Gaussian':'PCA Gaussian','CopulaIndependent':'Independent copula','CopulaJoint':'Joint
↪ copula','CVAE':'CVAE','Hybrid2plus1':'2+1','TrainSelected':'Train-selected','PointConstraint':'Point
↪ constraint','UncertaintyConstraint':'Uncertainty constraint','Unconstrained':'Unconstrained','Reject':'Reject'}
def save(fig,name):
 fig.tight_layout()
 for ext in ['png','pdf','svg']:fig.savefig(F/f'figures/{name}.{ext}',dpi=300,bbox_inches='tight')
 plt.close(fig)
def matrix(ax,mat,title,cbarlabel,fmtmax=None):
 cmap=plt.get_cmap('YlGnBu').copy();cmap.set_bad('#D5D5D5');im=ax.imshow(np.ma.masked_invalid(mat.to_numpy()),aspect='auto',cmap
↪ =cmap,vmin=0,vmax=fmtmax);ax.set_xticks(range(len(mat.columns)),[SHORT.get(v,v) for v in
↪ mat.columns],rotation=30,ha='right',fontsize=8);ax.set_yticks(range(len(mat.index)),mat.index,fontsize=8);ax.set_title(titl
↪ e);plt.colorbar(im,ax=ax,shrink=.75,label=cbarlabel)
def main():
 metrics=pd.read_csv(F/'q2/summary_metrics.csv').set_index('method').reindex(METHODS+['TrainSelected']);stab=pd.read_csv(F/'q2/s
↪ tability_summary.csv').set_index('method').reindex(metrics.index);fig,ax=plt.subplots(1,3,figsize=(12,3.8));xx=np.arange(5)
↪ ;ax[0].bar(xx,metrics.MAE_mean,yerr=metrics.MAE_std,color=[COL[m] for m in
↪ metrics.index],capsize=3);ax[0].set_xticks(xx,[SHORT.get(m,m) for m in
↪ metrics.index],rotation=30,ha='right',fontsize=8);ax[0].set_ylabel('Raw panel mean MAE');ax[0].set_title('Two repeated
↪ 3-fold predictions');ax[1].bar(xx-.18,stab.compound_jaccard,width=.36,color='#3775BA',label='Compound');ax[1].bar(xx+.18,st
↪ ab.group_jaccard,width=.36,color='#42949E',label='Alternative group');ax[1].set_xticks(xx,[SHORT.get(m,m) for m in
↪ metrics.index],rotation=30,ha='right',fontsize=8);ax[1].set_ylabel('Bootstrap
↪ Jaccard');ax[1].legend(fontsize=8);bc=pd.read_csv(F/'q2/bootstrap_choice_summary.csv');ax[2].barh([SHORT.get(m,m) for m in
↪ bc.method],bc['count'],color=[COL[m] for m in bc.method]);ax[2].set_xlabel('Choices among 100
↪ bootstraps');ax[2].set_title('Selection uncertainty');save(fig,'10_q2_unified')
```

```
 df=pd.read_csv(F/'generation/metrics_22styles_summary.csv');models=C['q3_models'];fig,ax=plt.subplots(1,2,figsize=(11,7.5));m=d
↪  f.pivot(index='style',columns='model',values='SWD_mean').reindex(index=ST,columns=models);matrix(ax[0],m,'Conditional
↪  distance (grey: no test beer)','SWD');m=df.pivot(index='style',columns='model',values='style_accuracy_mean').reindex(index=
↪  ST,columns=models);matrix(ax[1],m,'Independent classifier consistency','Fraction classified as
↪  target',1);save(fig,'11_generation_styles')
 fig,ax=plt.subplots(1,2,figsize=(11,7));m=df.pivot(index='style',columns='model',values='joint_floor_MAE_mean').reindex(index=S
↪  T,columns=models);matrix(ax[0],m,'Joint low-end indicator error','Pairwise probability
↪  MAE',1);m=df.pivot(index='style',columns='model',values='correlation_MAE_mean').reindex(index=ST,columns=models).dropna(how
↪  ='all');matrix(ax[1],m,'Rank structure: common pairs; n_test >= 4','Spearman correlation
↪  MAE',1);save(fig,'12_generation_structure')
 m=pd.read_csv(F/'migration/per_beer.csv');policies=['Unconstrained','PointConstraint','UncertaintyConstraint','Reject'];fig,ax=
↪  plt.subplots(1,2,figsize=(11,4));ids=sorted(m.beer.unique());x=np.arange(9)
 for policy in policies:
  d=m[m.policy==policy].set_index('beer').loc[ids];ax[0].plot(x,d.target_gain,marker='o',ms=4,label=SHORT[policy],color=COL[poli
  ↪  cy]);ax[1].plot(x,d.stale_change,marker='o',ms=4,label=SHORT[policy],color=COL[policy])
 for a in ax:a.axhline(0,color='black',lw=.8);a.set_xticks(x,[str(i) for i in ids],fontsize=8);a.set_xlabel('Held-out source
↪  beer');
 ax[0].set_ylabel('Tripel classifier score gain');ax[1].set_ylabel('Independent stale-hops
↪  change');ax[0].legend(fontsize=8);save(fig,'13_migration_paired')
 fig,ax=plt.subplots(1,3,figsize=(11,3.7));summ=pd.read_csv(F/'migration/summary.csv');
 for a,metric,title in zip(ax,['independent_stale_increase','change_RMS','refused'],['Independent stale-hops increase
↪  rate','Scaled chemical RMS change','Rejection rate']):
  d=summ[summ.metric==metric].set_index('policy').loc[policies];a.bar(range(4),d['mean'],yerr=np.vstack([np.maximum(0,d['mean']-
  ↪  d.CI_low),np.maximum(0,d.CI_high-d['mean'])]),color=[COL[p] for p in policies],capsize=3);a.set_xticks(range(4),[SHORT[p]
  ↪  for p in policies],rotation=35,ha='right',fontsize=8);a.set_title(title,fontsize=10)
 save(fig,'14_migration_risk')
 # Fixed example by lowest numerical beer ID, chosen without performance-based filtering.
 beer=ids[0];d=m[m.beer==beer];s=int(d.seed.iloc[0]);gen=joblib.load(F/f'models/generator_{pd.read_csv(O/"q3/selected.csv").set_
↪  index("seed").loc[s,"model"]}_{s}.joblib');i=int(np.flatnonzero(xdf.beer.to_numpy()==beer)[0]);source=gen.pre.imp.transform
↪  (X[[i]])[0];spec=pd.read_csv(F/'migration/spectra.csv');z=spec[spec.beer==beer].set_index('policy').loc[policies,chem].to_n
↪  umpy();delta=(z-source)/gen.pre.scaler.scale_;known=['DMS','X2.3.butanedione','X4VG','X.E..2.nonenal'];ix=[chem.index(c) for
↪  c in known];other=[j for j in np.argsort(abs(delta).mean(0))[::-1] if j not in
↪  ix][:8];ix+=other;fig,ax=plt.subplots(figsize=(10,3.2));im=ax.imshow(delta[:,ix],aspect='auto',cmap='RdBu_r',vmin=-max(.1,a
↪  bs(delta[:,ix]).max()),vmax=max(.1,abs(delta[:,ix]).max()));ax.set_yticks(range(4),[SHORT[p] for p in
↪  policies]);ax.set_xticks(range(len(ix)),[chem[j] for j in ix],rotation=35,ha='right',fontsize=8);ax.set_title(f'Fixed
↪  example beer {beer}: change in generator training SD units');fig.colorbar(im,ax=ax,label='Scaled
↪  change');save(fig,'15_transfer_chemistry');log('Closed-experiment publication figures complete')
if __name__=='__main__':main()
```

E.4 closed_report.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Final evidence-bound Chinese manuscript."""
from closed import *
import re,shutil
LABEL={'Spearman':'Spearman','Hybrid2plus1':'2+1','Forward3':'Forward3','Redundancy':'冗余控制','TrainSelected':'训练内选法','PCA_
↪  Gaussian':'PCA高斯','CopulaIndependent':'独立Copula','CopulaJoint':'联合Copula','CVAE':'CVAE','Unconstrained':'无约束','PointC
↪  onstraint':'点约束','UncertaintyConstraint':'不确定性约束','Reject':'拒绝迁移'}
def read(p):return pd.read_csv(F/p)
def md(df):
 d=df.copy()
 for c in d:
  if pd.api.types.is_float_dtype(d[c]):d[c]=d[c].map(lambda v:f'{v:.4f}' if np.isfinite(v) else 'NA')
 return '| '+' | '.join(str(v) for v in d.columns)+' |\n| '+' | '.join(['---']*len(d.columns))+' |\n'+'\n'.join('| '+' |
↪  '.join(str(v).replace('|','/') for v in a)+' |' for a in d.to_numpy())
def fig(n,cap):return f'\n![{cap}（AI 辅助绘图，数据来自封闭补充实验）](../outputs/rebuild/closed/figures/{n}.png){{width=95%}}\n'
def main():
 freeze();old=(R/'paper/competition_draft.md').read_text(encoding='utf-8');q1=old[old.index('# 问题一：'):old.index('# 问题二:
↪  ')];audit=old[old.index('# 数据审计与监督目标重建'):old.index('# 问题一：')];refs=old[old.index('# 参考文献'):re.search(r'#
↪  AI\s*使用声明',old).start()];refs=refs.replace('[7] OpenAI Codex，GPT-6 系列（接口未提供可独立核验的完整模型快照标识），OpenAI，
↪  2026-10-28。用于本参考工程的审计、代码、实验、图表与文本辅助。','[7] OpenAI Codex，GPT-6系列及gpt-6.1-sol，OpenAI，2026-10-28。
↪  用于审计、代码、实验、统计、图表与论文辅助。')
```

```
qm=read('q2/summary_metrics.csv');qs=read('q2/stability_summary.csv');choice=read('q2/bootstrap_choice_summary.csv');qsum=json.loads((F/'q2/summary.json').read_text());g=read('generation/macro_summary.csv');coverage=read('generation/style_coverage.csv');mig=read('migration/per_beer.csv');ms=read('migration/summary.csv');tests=read('migration/paired_tests.csv');gm=g.set_index('model');n_cov=int(coverage.n_unique_test.gt(0).sum());n_corr=int(coverage.n_correlation_contexts.gt(0).sum());nctx=int(read('generation/correlation_support.csv')[['seed','style']].drop_duplicates().shape[0]);pmetrics=mig.groupby('policy')[['target_gain','stale_change','independent_stale_increase','change_RMS','refused']].mean().reindex(['Unconstrained','PointConstraint','UncertaintyConstraint','Reject']);riskcounts=mig.groupby('policy').independent_stale_increase.sum();refcounts=mig.groupby('policy').refused.sum()
# Post-selection diagnostics for Q3: SWD gap vs test-set size, and the threshold-crossing defect view.
_sl=json.loads((O/'q3_complete/selection_logic.json').read_text(encoding='utf-8'));_df=json.loads((O/'q3_complete/defect_criteria.json').read_text(encoding='utf-8'))
sel_gap_r=_sl['gap_corr_with_n_test'];_bins={b['n_test_range']:b for b in _sl['gap_by_test_size']}
def _bin(k):return _bins.get(k,{'pairs':0,'cvae_closer':0,'mean_swd_gap_CVAE_minus_CopulaIndependent':float('nan')})
sel_b0_win=_bin('[1,2)')['cvae_closer'];sel_b0_n=_bin('[1,2)')['pairs'];sel_b0_gap=_bin('[1,2)')['mean_swd_gap_CVAE_minus_CopulaIndependent']
sel_b1_win=_bin('[2,3)')['cvae_closer'];sel_b1_n=_bin('[2,3)')['pairs'];sel_b1_gap=_bin('[2,3)')['mean_swd_gap_CVAE_minus_CopulaIndependent']
sel_b2_win=_bin('[3,5)')['cvae_closer'];sel_b2_n=_bin('[3,5)')['pairs'];sel_b2_gap=_bin('[3,5)')['mean_swd_gap_CVAE_minus_CopulaIndependent']
sel_b3_win=_bin('[5,inf)')['cvae_closer'];sel_b3_n=_bin('[5,inf)')['pairs'];sel_b3_gap=_bin('[5,inf)')['mean_swd_gap_CVAE_minus_CopulaIndependent']
xc=_df['threshold_crossing_by_policy'];anypos=_df['any_positive_by_policy']
qtable=qm[['method','MAE_mean','MAE_std','RMSE_mean','R2_mean']].copy();qtable['method']=qtable.method.map(LABEL);qtable.columns=['方法','MAE均值','MAE标准差','RMSE','R²'];stable=qs.copy();stable.method=stable.method.map(LABEL);stable.columns=['方法','单分子Jaccard','替代组Jaccard','三因素平均绝对相关']
q2=r'''# 问题二：训练内选法与统一的 50×3 化合物结果

## 从共线冗余到可检验的候选因素

单变量相关前三经常重复表达同一组发酵或酒花信号。把三项相关性当成三种独立机制，会放大候选因素的解释力度；直接改用条件选择又可能因有限样本残差而产生更不稳定的排名。因此，本问同时评价预测、冗余与重采样稳定性，并将“选择哪一种规则”的过程纳入训练数据内部。最终任务输出固定为每个感官三项，而不将低频变量包装成稳定的唯一因素。

候选集中剔除颜色、pH、压力、聚合总量等13项非单一分子测量，保留218个可识别化合物候选。完整预测任务仍可使用所有231项；两种特征口径不能混淆。共表达图节点为231项测量变量，附件2的Spearman绝对值达到0.65时连边，符号与绝对值分别保留。该图只作全样本描述，不作为外层测试的训练先验；相关定义和名称映射已经在原始数据上数值复核。

## 四种规则及正则残差推导

设训练秩变换后的特征矩阵为 $Z$，各列中心化并归一化，目标秩向量为 $t$。Spearman规则按 $|z_j^Tt|$ 取前三；2+1规则先取相关前二，再条件选第三；Forward3在三步均更新条件残差；冗余控制规则在每步用目标相关减去0.5倍与已选特征的最大绝对相关。

对已选集合 $S$，令 $H_S=Z_S(Z_S^TZ_S+0.02I)^{-1}Z_S^T$，构造 $u_j=(I-H_S)z_j$ 与 $r=(I-H_S)t$。在这个正则残差空间，单变量问题的最优增益为

$$\min_b\{\|r-bu_j\|^2+0.02b^2\},\qquad G_j=\frac{(u_j^Tr)^2}{u_j^Tu_j+0.02}.$$

每一步选择未入选变量中 $G_j$ 最大者。这里的“完整前向”指三个步骤都条件更新；由于 $H_S$ 不是幂等投影，该分数是所定义辅助残差问题的增益，不能声称精确求解所有三变量组合或原始联合Ridge的全局最优。2+1与Forward3采用相同的条件分数，差别仅在前两步。

四种规则选择后，均用相同的三个附件编码特征、训练中位数和训练标准化，拟合alpha=8的Ridge。它们拥有相同预测器和变量数，避免把选择规则与预测模型复杂度混在一起。

## 方法选择也属于训练流程

沿用原两次三折啤酒级外层划分。每个外层训练集内部，按既有20%划分计算四方法的50目标等权MAE，确定一种规则，然后在完整外层训练集上重新选择三变量并拟合；外层酒样只用于评价。最终全样本模型重用同一内部选择算法，再全量重拟合。外层所有候选成绩均保留，最终规则的选择不使用其外层分数。

下表先对每次完整OOF预测计算指标，再报告两次结果的均值与标准差；它修正了按不等大小折直接平均的汇总口径。真实监督是评分员等权原始均分，MAE与标准化附件4的旧误差不可直接横比。
'''
q2+='\n'+md(qtable)+'\n\n'
q2+=f'''最终全训练数据内部选择为{LABEL[qsum['final_method']]}，故正式50×3表、分子名称、选择得分、入选率与替代组指标统一来自该规则。训练内选法流程的外层MAE为{qm.set_index('method').loc['TrainSelected','MAE_mean']:.4f}，固定2+1的OOF MAE为{qm.set_index('method').loc['Hybrid2plus1','MAE_mean']:.4f}。前者没有获得额外预测收益，说明小样本内层选择具有方差；不能用全量内部选出的2+1替代训练内选法的外层成绩。配对检验先平均同款酒的两次OOF预测，再按酒计算误差，与表中逐次误差均值的口径不同。检验与啤酒bootstrap区间见closed/q2/paired_tests.csv，没有据测试结果再设新的加权规则。

## 固定规则与完整流程的两类稳定性
```

100次Bootstrap在各风格内按酒重采样，填补、缩放和选择全部重做。重复抽到的同款酒被整体放入同一内层角色，避免其副本跨越内层训练与验证。
↪ 固定规则频率回答“给定规则时分子是否稳定”；训练内选法频率还包含规则本身变化。最终150行同时保存两种频率，
↪ 避免以固定规则频率代替完整流程的不确定性。

替代组沿用描述性完全链接分组：组内各对的绝对Spearman至少0.8。它用于衡量相近测量是否可替代，不参与外层特征选择。
↪ 每个目标用相同的100对Bootstrap集合计算单分子及替代组Jaccard，并同时记录三因素内部绝对相关。

{md(stable)}

内部规则的 Bootstrap 选择次数如下。规则切换本身是小样本不确定性的结果，不据此继续增加规则或调节惩罚系数。

{md(choice.assign(method=choice.method.map(LABEL)).rename(columns={'method':'规则','count':'次数'}))}

风格混杂与化学解释

表中保留原始Spearman、风格内去均值后的秩关联、标签疑点以及来源/单位。后者是混杂敏感性指标，不是已识别的因果效应；风格也可能代表真实发酵过程，
↪ 调整后关联变化需结合过程解释。高度相关的酯和高级醇可能共同反映发酵条件，不能简单将排名逐一解释为独立作用。
↪ 原论文通过真实添加实验验证部分候选[1]，本工程尚未开展这一步。

最终表不以“化学常识”手工替换统计结果。高频主因素、同组可替代项和低频候选分别解释；barnyard保留疑似公式别名标记。每一项的名称、频率、
↪ 方向和风格调整变化见统一CSV与逐维解释报告，供队员独立作化学复核。
'''
q2+=fig('10_q2_unified','四规则与训练内选法的预测、稳定性及选择频率')
gtable=g[['model','SWD_mean','style_accuracy_mean','correlation_MAE_mean','joint_floor_MAE_mean']].copy();gtable.model=gtable.m」
↪ odel.map(LABEL);gtable.columns=['模型','条件SWD','风格一致性','相关误差','联合低端误差']
gen=r'''# 问题三之一：22 风格条件生成的分布检验

小风格样本与观测最低值的冲突

每种风格仅3至31款，无法支持各自独立估计231维联合分布。直接连续生成又难以保留大量相同最小值。采用总体信息共享可以降低方差，却可能削弱风格边界；
↪ 分布距离变小也可能来自生成谱向均值收缩。本轮将这种取舍作为待验证问题，比较已有四模型，参数与训练角色全部冻结。

PCA 高斯使用 12 维潜变量、风格均值收缩和共享协方差，输出限制在训练范围。独立 Copula 与联合 Copula 共享收缩条件边际：

$$\widehat F_{j,s}=w_s\widehat F_{j,s}^{\rm empirical}+(1-w_s)\widehat F_{j,all},\qquad w_s=\frac{n_s}{n_s+10}.$$

对稀有风格，较大的总体权重控制经验分布方差。联合Copula将训练秩变换成正态分数，用Ledoit-Wolf收缩估计相关矩阵[5]，通过 $z\sim N(0,\widehat
↪ R)$、$x_j=\widehat F_{j,s}^{-1}(\Phi(z_j))$ 采样。独立消融令相关矩阵为单位阵，检验联合依赖的贡献。经验逆分布的区间自然保留最小值点质量，
↪ 不需要后续逐列随机替换；模型并未识别最低值以下的真实浓度。

CVAE沿用231维输入、22类条件、64维隐层、12维潜变量和KL系数0.05，最多160轮，以原35款验证酒的重构误差选检查点[6]。四模型不增加训练轮数、
↪ 结构或超参数搜索。

逐风格评价如何避免循环自证

三个原始场景均保留140款生成训练酒、35款验证酒、40款独立评价器训练酒与35款测试酒。对每个场景的每种风格生成40条谱，共计算22×4×3=264行指标。
↪ 四模型共享按场景与风格固定的采样种子、48个投影方向和训练尺度。风格分类器使用原迁移评价器的既有ET配置，在独立40款酒上拟合并保存；
↪ 其输出只用于评价，不参与模型或样本筛选。

对测试风格分布计算边际Wasserstein与切片Wasserstein距离。相关结构使用真实和四模型共同可计算的Spearman变量对；测试样本少于4款时不估计该项。令
↪ $B_j(x)=1\{x_j\le m_j^{train}\}$，其中 m_j^{train} 为训练最小值，仅对训练中低端比例至少25%的变量评价。联合最低值模式用各变量对的
↪ $E[B_jB_k]$ 绝对误差衡量，同时报告单变量低端质量误差。这里的“低端”是观测编码阈值，不是已知仪器LOD。

最近邻距离、完整复制比例和标准化多样性均在最终输出谱上计算。零完整复制只能排除逐行原样复制，不能证明经验边际没有训练信息记忆。
↪ 风格分类器的输出分数也没有概率校准，应解释为模型诊断分数。
'''
gen+=f'''
支持范围与真实结果

原划分中{n_cov}/22种风格有独立测试酒；Faro与Flanders old brown为零。相关结构仅{n_corr}/22种风格在至少一个场景满足样本数条件，
↪ 共{nctx}个风格场景。两类缺口在表格和热图中保留NA，不能用验证酒、训练酒或生成样本补成测试证据。完整22风格覆盖表及88行分模型汇总见附录。

下表先在每个风格内跨场景平均，再对有定义的风格等权平均；分类一致性覆盖全部22种风格，分布和联合低端误差覆盖20种，相关误差只覆盖4种。
↪ 各列支持集合不同，因此必须按列比较同指标，不能把它们合成未预设的总分。

{md(gtable)}

条件尺度结果与旧版混合风格总体距离回答不同问题。CVAE的条件SWD较小，但其平均多样性为{gm.loc['CVAE','diversity_mean']:.4f}；
↪ 独立Copula为{gm.loc['CopulaIndependent','diversity_mean']:.4f}。当真实测试风格只有一两款时，收缩到中心可能使经验距离更小，
↪ 不能据此断言CVAE更真实。最近邻距离、风格一致性和低端质量共同揭示这种取舍。

上表会让读者产生一个必须正面回答的疑问：既然 CVAE 的条件 SWD 最低（{gm.loc['CVAE','SWD_mean']:.4f}），为什么交付的生产生成器是独立 Copula
↪ （{gm.loc['CopulaIndependent','SWD_mean']:.4f}）而不是 CVAE？答案是两件事发生在不同时间、依据不同数据，且排序反转本身可以被量化。
↪ 选型在封闭诊断之前就已冻结：三个场景各自的验证折（35 款酒）上按平均 SWD 取最小者，逐场景选出的分别是联合 Copula、独立 Copula、独立
↪ Copula，测试折从头到尾没有参与选型，封闭诊断也不被允许回改选型。把封闭诊断的逐风格结果拆开看，CVAE 的相对优势与测试集大小高度相关
↪ （相关系数 r={sel_gap_r:.2f}）：在只有 1 款独立测试酒的风格上 CVAE {sel_b0_win}/{sel_b0_n} 全胜、优势 {sel_b0_gap:+.3f}，2 款时
↪ {sel_b1_win}/{sel_b1_n}、{sel_b1_gap:+.3f}，3—4 款时 {sel_b2_win}/{sel_b2_n}、{sel_b2_gap:+.3f}，5 款及以上时只剩
↪ {sel_b3_win}/{sel_b3_n}、{sel_b3_gap:+.3f}。也就是说，CVAE 的优势随经验分布分辨率上升而单调消失——测试集只有一两款酒时，
↪ 经验一侧几乎没有分辨率，任何向风格均值收缩的生成器都会显得“更近”。这与 CVAE 自身的退化程度一致：多样性
↪ {gm.loc['CVAE','diversity_mean']:.4f} 对 {gm.loc['CopulaIndependent','diversity_mean']:.4f}，到最近训练样本的 RMS
↪ {gm.loc['CVAE','nearest_train_RMS_mean']:.4f} 对 {gm.loc['CopulaIndependent','nearest_train_RMS_mean']:.4f}，
↪ 它更靠近小样本经验分布。因此选型依据不是条件 SWD 这一列，而是验证折规则加上多样性与低端质量的匹配；反过来说，如果 CVAE 在 5
↪ 款以上测试酒的风格上仍保持优势，冻结的选型就是错的——它没有，6 对里 Copula 赢 {sel_b3_n - sel_b3_win} 对。需要同时承认的是，
↪ 满足该条件的风格只有 {sel_b3_n} 个，这只是一个方向性证据，不足以给出决定性检验。完整对照见
↪ outputs/rebuild/q3_complete/selection_logic.json 与 selection_criteria_table.csv。

联合Copula相关误差为{gm.loc['CopulaJoint','correlation_MAE_mean']:.4f}，
↪ 独立Copula为{gm.loc['CopulaIndependent','correlation_MAE_mean']:.4f}，改善范围限于可估计场景；
↪ 联合低端误差分别为{gm.loc['CopulaJoint','joint_floor_MAE_mean']:.4f}和{gm.loc['CopulaIndependent','joint_floor_MAE_mean']:.4f}，
↪ 未显示联合模型的统一优势。共享一个总体依赖矩阵不能自动恢复每类风格特有的低端共现关系。

以上数字给出了本问能支持到什么范围，需要一并交代清楚。第一，风格条件控制本身仍然有限：四个模型的独立评价器风格分类准确率分别为 CVAE 0.0917、
↪ PCA 高斯 0.0852、联合 Copula 0.1356、独立 Copula 0.1409，在 22 类问题上都只略高于随机水平（1/22≈0.045），说明「生成了指定风格的谱」
↪ 不等于「已充分复现该风格的特征」，风格画像应视为条件生成的方向性结果而非风格标定证据。第二，覆盖度不完整：22 种风格中仅 20
↪ 种在既定划分下有独立测试酒，相关结构指标只有 4 种风格满足样本条件，Faro 与 Flanders old brown 两类完全没有独立测试证据。第三，
↪ 物理尺度仍有缺口：化学列中只有一部分具备可靠的量纲校准证据，其余仅保留附件编码，
↪ 因此生成与迁移结论只能建立在观测编码与本报告给出的阈值判定上，不能外推为绝对浓度结论。

生产生成器和迁移生成器继续沿用此前验证集选定结果，未依据此次条件测试成绩改选。Tripel、Kriek、Lambic画像及22风格880条谱仍保留为展示输出；
↪ 生成模型的统计接近不构成真实酿造可行性证据。
'''
gen+=fig('11_generation_styles','22风格条件距离与独立风格一致性；
↪ 灰色表示无测试证据')+fig('12_generation_structure','联合低端模式与有共同变量对支持的相关结构误差')
mtable=pmetrics.reset_index();mtable.policy=mtable.policy.map(LABEL);mtable.columns=['策略','目标收益','stale变化','stale上升率',」
↪ '化学RMS','拒绝率'];tt=tests[['policy','metric','mean_difference','p_holm']].copy();tt.policy=tt.policy.map(LABEL);tt.column」
↪ s=['策略','指标','均值差','Holm校正p']
migration=r'''# 问题三之二：独立评价与拒绝迁移

决策目标和评价器必须隔离

跨风格迁移同时追求目标接近与缺陷不恶化。若同一个感官预测器既筛选候选又证明安全，优化器容易利用其误差。本实验固定原始9款不同Pils源酒，
↪ 每款只保留其最早符合条件的原测试场景，避免重复计数。源酒不属于相应生成训练、验证或评价器训练角色。三个场景的独立评价器在优化前序列化，
↪ 其文件哈希在所有策略完成后保持不变。

每个源生成80条Tripel候选，再以0.25、0.5、0.75、1.0插值形成320条迁移谱，加入原谱作为第321个候选。四策略共用候选、设计预测器、尺度和成本；
↪ 固定的拒绝策略直接返回原谱。对候选 x'，成本为

$$J(x')=\|T(x')-\mu_{Tripel}\|_2+0.15\|T(x')-T(x)\|_2,$$

其中 T 为生成训练集标准化，目标中心来自同一批 Tripel 候选。独立风格分类分数不进入这个成本。

无约束策略选择最低成本。点约束与不确定性约束先将四个明确化学标记裁剪至不高于源编码，再用12个仅在生成训练集拟合的Bootstrap
↪ ET计算stale_hops预测差。点约束要求平均差不大于零；不确定性约束要求90%分位差不大于零。保留原谱保证候选集合存在可行返回；
↪ 返回原谱计为拒绝迁移。该分位没有经覆盖率校准，是模型集成风险筛查，不是90%置信安全保证。

所有策略的选择完成后，才读取独立评价器，对源谱与最终谱统一评分。评价器包括固定ET分类器、
↪ 感官回归器和在相同独立40款酒上构造的12个Bootstrap回归器。评价器的集成差分位用于检验设计风险能否迁移，不反馈到选择过程。

四策略的 9 酒配对结果

目标收益为独立分类器Tripel分数之差；缺陷风险同时报告stale_hops均分变化、上升酒数和独立集成90%分位。
↪ 化学RMS为按生成训练尺度标准化的231维改变量。下表以全部9款为分母，包含被拒绝酒的零收益，避免只在接受子集上展示效果。
'''
migration+='\n'+md(mtable)+'\n\n'
migration+=f'''点约束带来平均目标分数收益{pmetrics.loc['PointConstraint','target_gain']:.4f}，
↪ 其独立stale_hops均值变化为{pmetrics.loc['PointConstraint','stale_change']:.4f}，
↪ 但仍有{int(riskcounts['PointConstraint'])}/9款上升。平均下降与个体风险并不矛盾，说明仅报告均值会掩盖失败样本。
↪ 不确定性约束拒绝{int(refcounts['UncertaintyConstraint'])}/9款，
↪ 全部源酒的平均目标收益降为{pmetrics.loc['UncertaintyConstraint','target_gain']:.4f}，
↪ 仍有{int(riskcounts['UncertaintyConstraint'])}/9款上升；在实际接受的7款中对应4/7。因而本次证据不支持
↪ “更保守的设计约束必然带来独立缺陷改善”。

必须把这一点明确记为局限，但要同时给出赛题口径下的对照，否则会低估约束的作用。本文的缺陷判定有两套口径，二者结论强度不同，缺一不可。

第一套是本文沿用至今的严格口径：只要独立评价器的 stale_hops 预测相对源谱上升了任意 epsilon（>1e-10）就计为恶化。
↪ 按此口径三种策略的上升酒数是无约束{int(anypos['Unconstrained'])}/9、点约束{int(anypos['PointConstraint'])}/9、
↪ 不确定性约束{int(anypos['UncertaintyConstraint'])}/9，约束把上升数从6压到4—5但没有压到0；
↪ 代价是目标收益由{pmetrics.loc['Unconstrained','target_gain']:.4f}降到{pmetrics.loc['UncertaintyConstraint','target_gain']:.4f}
↪ （约减半）并额外拒绝{int(refcounts['UncertaintyConstraint'])}/9款。这一口径下确实没有得到“不恶化”。

第二套是赛题原文指向的口径：题目把缺陷化合物定义为“所列 DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops 等超阈值项”，即判定对象是超阈值与否，
↪ 而不是任意微量上升。按此口径统计“源酒未超阈值、迁移后越过阈值”的次数（三个具备可用感官检出阈值的化学缺陷项，源酒低于阈值的可用样本23个），
↪ 无约束策略发生{int(xc['Unconstrained'])}次越阈，点约束{int(xc['PointConstraint'])}次，
↪ 不确定性约束{int(xc['UncertaintyConstraint'])}次。四个有观测化学编码的缺陷项在点约束与不确定性约束下的化学增加次数均为0，
↪ 与这一结果一致。

两套口径的差别只有0.05分以内（0—6量表上），而这远低于模型自身在stale_hops上的约0.23 MAE，因此任何一套都不能单独支撑“保证不恶化”，
↪ 也不能据此否定约束的作用。本文的主张因此限定为：在赛题定义的超阈值口径上，
↪ 硬约束把越阈次数由无约束的{int(xc['Unconstrained'])}次降到{int(xc['UncertaintyConstraint'])}次，即目标函数意义上的“缺陷不恶化”
↪ 在本次内部检验中达成；在任意微量上升的严格口径上仍有4—5/9款酒上升，该残余风险列为未解决局限。要真正闭合这两套口径之间的差距，
↪ 需要真实添加样品与盲评，或在本数据之外校准评价器误差的传递关系，两者都超出本轮可得证据。两套口径的逐项结果见
↪ outputs/rebuild/q3_complete/defect_criteria.json、defect_threshold_crossing.csv 与 defect_magnitude.csv。

拒绝迁移的收益与改变量为零，拒绝率100%，提供明确的决策下界。它不能作为实现目标风格的成功，但在缺少可靠安全证据时是可陈述的保守输出。
↪ 本轮未通过观察独立评价器后新增拒绝阈值，也未把拒绝策略改造成反馈优化器。

统计与缺陷口径

2000次配对Bootstrap按9款不同源酒重采样，区间条件于固定训练场景与固定评价器，不包含重新训练的不确定性。
↪ Wilcoxon检验对三种实际迁移与拒绝的两类收益/缺陷差分别组成6项比较，再作Holm校正。

{md(tt)}

点约束与无约束的目标分数收益在本次内部检验中通过0.05的Holm门槛；三个实际迁移策略的stale变化均没有通过。
↪ 小样本条件下不能把未拒绝零假设解释为缺陷等效或安全。

DMS、Diacetyl、4VG、trans-2-nonenal分别核对附件列名和变换。前三项同时列出平均及中位感官阈值，
↪ trans-2-nonenal和stale_hops没有完整可用阈值时记NA。感官检测阈值不等于跨风格通用的不可接受界限。对于四个有观测化学编码，
↪ 约束策略的增加次数为零；源缺失值和低于/位于训练最小值的标记保留“物理不可验证”，不据裁剪宣称真实浓度顺序。stale_hops使用独立感官代理，
↪ 不能换算成唯一化合物浓度。

逐酒原谱角色、评价器哈希、选中候选编号、接受/拒绝、五类缺陷检查及单位表均保存。
↪ 统计生成和迁移结果仍需要真实样品添加与盲评才能验证工艺和感官可行性。
'''
migration+=fig('13_migration_paired','9款独立源酒的目标收益和独立stale_hops变化')+fig('14_migration_risk','缺陷上升率、
↪ 化学改变量与拒绝率；误差棒为酒级Bootstrap区间')+fig('15_transfer_chemistry','预先固定为最小beer编号的迁移示例，未按效果择优')
innovation=f'''# 创新动机、消融与模型评价

本研究的改进围绕三个可核验的困难展开。第一，真实评分卡既有直接强度，也有父强度继承的子项，且存在重复评分员。按评分观测机制定义目标与条件似然，
↪ 使模型学习对象明确；冻结的结构消融显示其优于普通MLP，但未超过ET，因而保持传统树模型作为预测主线。

第二，Top-3既是一个预测问题，也是一个选择不确定性问题。将四种规则放入训练内选择，并在Bootstrap中重新选择规则，使最终表、
↪ 性能和稳定性属于同一个可复现流程。外层结果显示自动选法没有超过固定2+1，这个负结果揭示了小样本内层方差，不能通过事后改表消除。
↪ 替代组使可替代信息可见，但不是新增因果证据。

需要把结论强度说得更准确，避免读者误读。最终 2+1 规则的训练内得分（0.4841）略低于纯 Spearman 基线（0.4845），重采样稳定性也低于它：2+1
↪ 的平均分子 Jaccard 为 0.2288、替代组 Jaccard 为 0.2354、Top-3 内平均绝对相关 0.2740，而 Spearman 分别为 0.2827、0.2929、0.4385。
↪ 外层预测 MAE 上 2+1 为 0.4988，优于 Spearman 的 0.5021，但差距仅 0.003 量级。因此第二问的贡献应定位为：把「选择哪一种规则」
↪ 纳入训练内部并输出可复现的统一 50×3 表，同时用冗余控制减少同一机制在 Top-3 内重复占位；不能宣称 2+1
↪ 在预测性能或重采样稳定性上全面优于相关性基线。Top-3 仍是候选因素清单，其因果地位需真实添加实验确认，本工程未开展该步。

第三，生成任务区分单变量最低值质量、联合低端模式和风格条件一致性，迁移任务进一步区分设计预测器与独立评价器。独立Copula是联合依赖消融，
↪ PCA高斯是连续低维基线，CVAE是非线性共享生成对照；无约束、点约束、不确定性约束与拒绝给出决策层对照。对照没有支持
↪ “深度或联合依赖模型在所有指标上更优”，也没有支持“设计风险分位保证独立缺陷安全”。

创新应定位于对本题观察机制的具体处理和完整验证，而不是宣称新发明Copula、Bootstrap或变分自编码。尚无外部新队列与真实酿造盲评，
↪ 因此不能宣称一般化的感官因果机制。最稀有两类没有独立测试证据，相关结构多数风格不可估计；这些缺口限制结论支持范围。按用户设定，
↪ 三组封闭补充完成后停止所有模型探索，不再为改善结果增加模型、阈值或评价器。

结论与建议

250款啤酒、231项测量与2197条逐评分员记录形成统一目标和训练隔离基础。第一问冻结ET的MAE为0.4454、RMSE为0.6305，
↪ 整数输出对应真实面板均分的舍入等级。第二问最终训练内部选出2+1并输出50×3候选；训练内选法流程的OOF MAE为0.5024，
↪ 说明方法选择的代价应进入评价。第三问完成22风格四模型条件诊断，20种风格有独立分布证据、4种风格具备最低相关估计支持；
↪ 指标取舍与有限样本误差阻止了单一最优生成器的全面结论。9酒配对迁移显示，点约束保留目标收益，却仍有5款独立缺陷代理上升；
↪ 不确定性约束以2款拒绝为代价仍出现4款上升。

该工程已完成既定三组补充，具备数据、代码、指标、图表与论文之间的追溯关系。正式参赛前的必要工作是队员独立核实核心建模与分析、
↪ 确认匿名样本映射及疑点标签，并决定可采纳的实验结论。真实工艺与盲评验证属于当前数据无法替代的证据。
'''
intro=r'''# 问题分析与统一研究主线

啤酒风味预测的困难首先来自观察尺度：化学列混合编码、评分卡含层次继承、同一评分员重复品评，而风格只提供小样本条件。若监督目标、
↪ 关键因素与生成评价采用不同口径，复杂模型的表面提升可能无法解释。因此，围绕“观察了什么、训练可以使用什么、何种证据支持结论”组织三问。

原论文[1]结合化学特征、专业面板与消费者信息，并通过添加品评检验候选成分。赛题要求新增0—6分级、50维Top-3和跨风格缺陷约束。
↪ 作者公开的逐评分员数据[2]使真实面板尺度可用，但不能消除评分卡疑点或把附件4简单逆变换成原分。先恢复可识别目标，再评价候选解释与生成决策，
↪ 是三问衔接的基础。

本研究保留有竞争力的传统基线，也实现评分机制驱动的层次网络与CVAE。是否采用模型由冻结实验中的证据决定，不以结构复杂度作为创新标准。
↪ 最终补充限定为Q2统一选择、22风格生成诊断和9源酒独立迁移三组，全部完成后转入论文定稿。

主要假设为：匿名beer_name按附件物理行序对应；面板目标给予评分员等权；缺失化学测量以训练中位数进行工作性填补；小风格借用总体经验边际降低方差。
↪ 每项假设均保留其可验证范围，尤其不把最低值全部视作已知检测限，也不把统计生成谱视作直接酿造配方。

符号、评价对象与可复现约定

设 $x_i\in\mathbb R^{231}$ 为附件编码谱，s_i 为风格，y_{irt} 为评分员 r 在重复 t 的评分。原始目标为

$$\bar y_i=\frac{1}{R_i}\sum_r\frac{1}{T_{ir}}\sum_t y_{irt},\qquad g_i=\min(6,\max(0,\lfloor\widehat y_i+0.5\rfloor)).$$

因此连续监督、面板整数等级与个体打分是三个不同对象。所有外层误差按啤酒与感官目标等权；相同啤酒的全部记录留在同一角色。数据填补、
↪ 标准化与监督选择均从当前训练数据估计。

两个原始种子提供两次三折内部验证；生成任务沿用三个140/35/40/35分区。原数据已在此前探索中被检查，本轮是严格隔离的内部补充验证，不是新外部队列。
↪ 重复划分标准差衡量划分变化，不能当作独立多次采样的总体置信区间。配对酒级Bootstrap给出条件于既有训练和评价器的样本不确定性。

完整原协议、封闭补充协议、Q1冻结哈希、训练日志和结果索引一并保存。本文遵照本地官方格式与AI规定[3]，图表、程序与文字均如实记录AI贡献；
↪ 不补造队员的独立工作或签名。
'''
declaration='''# AI 使用声明与独立工作状态

本工程及本稿使用OpenAI Codex，历史GPT-6系列及本轮gpt-6.1-sol[7]，辅助数据核验、模型实现、实验运行、统计、可视化、论文和排版。
↪ 本轮严格执行三组封闭补充，Q1不重训、不改预测，仅在2026-10-28修正了评分卡交付的字段过滤缺陷（30字段→3.xlsx summary 全部50字段），
↪ 修正记录见 outputs/rebuild/audit/protocol_amendment.md，冻结哈希同步更新；三组完成后停止建模探索。程序头部、
↪ 图表说明和支撑材料均标明AI参与。

官方规定要求核心建模、算法设计、求解与分析由参赛队员独立完成。当前没有可核实的队员独立核心工作或人工修改记录，本稿如实保留该状态；
↪ 单纯审阅签字不能替代独立完成。队员须独立开展并记录关键工作后决定正式采用内容。详细说明、
↪ 关键交互摘要与待完成记录位于AI工具使用详情说明.pdf、AI_usage_log.md和human_decision_record.csv。
'''
abstract=f'''针对啤酒化学数据与感官评分的尺度混合、Top-3共线冗余及小样本风格迁移问题，构建由观测语义、训练内选择和独立风险验证衔接的三问方案。
↪ 将2197条原始品评记录映射到250款啤酒，定义评分员等权0—6面板目标，并保留评分卡层次和重复品评敏感性。评分卡交付为250行×53列、无缺失值：
↪ 1列啤酒编号，50列由冻结模型预测的感官评分，另2列acetaldehyde为阈值锚定推断值（无面板0—6记录，
↪ 由附件1实测浓度经log10反变换后与附件8的50.0mg/L感官检出阈值比较，250款酒全部低于阈值故取0=Not present），后两列明确标注为非面板评分。
↪ 该评分卡存在一项已量化并披露的局限：预测等级全部落在0—4级，真值≥5的121个单元被系统低估（平均低估2.07，连续预测上限仅4.34），
↪ 故整体MAE可用而高强度尾部不可用。冻结的Extra Trees取得MAE 0.4454、RMSE 0.6305，层次网络未超过强传统基线；非香气强度项的平均MAE
↪ （0.3486）优于香气强度项（0.5047）。化合物筛选在训练内部比较Spearman、2+1、三步条件前向与冗余控制，最终选出2+1并统一50×3结果；
↪ 包含方法选择的外层MAE为0.5024，揭示小样本选法的方差代价。对四种既有生成模型完成22风格分布、分类一致性、相关结构及联合低端模式诊断：
↪ 交付生成器沿用诊断之前冻结的验证折规则而非条件SWD排名，因为CVAE的表面SWD优势与测试集大小强相关（r=0.70，仅在1—2款测试酒的风格上领先，
↪ 5款以上时优势消失）。9款独立源酒的四策略配对比较按两套缺陷口径报告：在赛题所指的超阈值口径上，硬约束把越阈次数由无约束的7次降到0次；
↪ 在任意微量上升的严格口径上仍有4—5/9款酒上升，该残余风险列为未解决局限。结果支持保留拒绝选项、报告个体风险并区分编码约束与真实化学保证。
↪ 三组封闭实验完成后停止模型探索，交付可复现代码、风格谱、候选因素和完整验证材料。本文为如实披露AI贡献的研究定稿，队员独立核心工作状态另列。
↪ '''
paper='# 观测语义约束下的啤酒风味预测、化合物筛选与条件生成\n\n## 摘要\n\n'+abstract+'\n\n关键词：面板评分；训练内选择；替代组稳定性；
↪ 条件生成；独立风险\n\n'+intro+'\n'+audit+'\n'+q1+'\n'+q2+'\n'+gen+'\n'+migration+'\n'+innovation+'\n'+refs+'\n'+declaration
(R/'paper/competition_draft.md').write_text(paper,encoding='utf-8');(R/'paper/rebuild_body.md').write_text(paper[paper.index('#
↪ 问题分析'):],encoding='utf-8');(R/'paper/rebuild_abstract.txt').write_text(abstract,encoding='utf-8')
(R/'reports/02_q2_analysis.md').write_text(q2,encoding='utf-8');(R/'reports/03_q3_analysis.md').write_text(gen+'\n'+migration,e
↪ ncoding='utf-8');(R/'reports/04_innovation_and_ablation.md').write_text(innovation,encoding='utf-8');(R/'reports/08_closed_
↪ experiments.md').write_text(q2+'\n'+gen+'\n'+migration,encoding='utf-8')

```
 final=f'''# 最终封闭补充结果与交付索引

三组封闭补充均完成，停止所有模型探索。Q1 不重训、不改预测，但交付层有两处修正（均由数据核实后作出，不涉及模型）：(1)
↪  评分卡交付表此前由列名前缀过滤只输出 30 个字段，现补齐为 3.xlsx summary 定义的 50 个 0—6 监督字段；(2) 两列 acetaldehyde 此前留空，
↪  现由附件1 实测浓度对附件8 感官检出阈值判定后统一取 0（250 款酒浓度 0.01—18.56 mg/L，全部低于 50.0 mg/L 阈值），判据见
↪  outputs/rebuild/q1_complete/acetaldehyde_grading.json。交付表因此为 53 列 = 1 列 beer + 50 个感官监督模型预测字段 + 2
↪  个阈值锚定推断字段，250 行、无缺失值；乙醛两列明确标注为非面板评分，不与 50 个监督字段混同。冻结哈希与协议说明同步更新，记录见
↪  outputs/rebuild/audit/protocol_amendment.md。

Q1 交付表的字段覆盖不均衡并不存在：30 个带 A_/F_ 前缀字段平均 MAE 0.5047，20 个无前缀字段平均 MAE 0.3486，即此前被漏掉的 20 个维度反而更准。

Q2 最终规则为 2+1：固定 2+1 的两次 OOF 平均 MAE 为 0.4988，训练内方法选择流程 MAE 为 0.5024。最终 150 行、预测指标、固定规则与流程
↪  Bootstrap、替代组频率统一保存在 outputs/rebuild/closed/q2；兼容入口 outputs/rebuild/q2/top3_50x3.csv 与最终表逐字节一致。

22 风格四模型已生成 264 行场景级与 88 行汇总诊断。Faro 与 Flanders old brown 无独立测试酒；相关结构仅 4 种风格可估计。
↪  交付生成器沿用诊断之前冻结的验证折规则（三场景分别选出联合 Copula、独立 Copula、独立 Copula），未按条件 SWD 排名回改：CVAE 的条件 SWD
↪  （0.7059）虽最低，但其优势与测试集大小强相关（r=0.70；仅 1—2 款测试酒的风格上领先，5 款以上时优势消失为 −0.011 且 6 对中输 2 对），且
↪  CVAE 多样性 0.4202、最近训练样本 RMS 0.6812，明显更靠近小样本经验分布。完整量化见 outputs/rebuild/q3_complete/selection_logic.json。

迁移按 9 款不同源酒比较 4 策略，独立评价器冻结，优化后才评分。缺陷判定按两套口径并列报告：严格口径（任意 epsilon>1e-10 上升即算恶化）下无约束
↪  6/9、点约束 5/9、不确定性约束 4/9；赛题原文的超阈值口径下，源酒未超阈值而迁移后越阈的次数为无约束 7 次、点约束 0 次、不确定性约束 0 次。
↪  目标增益点约束 0.0514、不确定性约束 0.0289（拒绝 2/9）。两套口径差 0.05 分以内，远小于模型在 stale_hops 上的约 0.23 MAE，故本文只主张：
↪  超阈值口径上硬约束有效，严格口径上仍存残余风险并列为未解决局限。均为模型代理风险，不是真实酿造安全保证。逐项见
↪  outputs/rebuild/q3_complete/defect_criteria.json。

复现入口 scripts/run_final.ps1，仅执行既定三组并更新图表与论文，不重新训练 Q1；脚本不写死机器路径。论文定稿 PDF 为
↪  paper/competition_final.pdf；AI 使用详情与提交前待补项见 submission/。匿名身份、真实队号、队员签署及独立核心工作尚未提供，未伪造。
''';(R/'reports/05_final_summary.md').write_text(final,encoding='utf-8')
 # Per-target chemistry comments are now tied to the final selected method.
 top=read('q2/top3_50x3.csv');explain='# 最终2+1逐感官化合物解释与复核线索\n\n150项均由统一训练内规则产生。以下为统计候选，未做因果确认。
↪  \n\n'
 for t,d in top.groupby('target',sort=False):
  explain+='## '+t+'\n\n'
  for _,v in d.iterrows():explain+=f"第{int(v['rank'])}项 {v.chemical}（{v.get('matrix_name','')}）：Spearman={v.spearman:.3f},
  ↪  风格调整秩关联={v.style_adjusted_rank_association:.3f}，固定规则入选率={v.final_fixed_method_frequency:.0%},
  ↪  完整流程入选率={v.bootstrap_frequency:.0%}，替代组频率={v.group_frequency:.0%}。来源类别：{v.get('origin_paper','未核实')}。
  ↪  "+('入选率不足一半，保留候选不确定性。' if v.bootstrap_frequency<.5 else '在本次完整流程重采样中较稳定，仍需独立化学验证。')+'\n\n'
 (R/'reports/02_q2_compound_explanations.md').write_text(explain,encoding='utf-8')
 # Deliverable checklist for the team's independent review. Each row must be filled by a real member
 # (reviewer, date, evidence); the AI never fills these in.
 csv([dict(decision=d,status='待队员独立完成与复核',reviewer='',date='',evidence='',required_deliverable=v)
      for d,v in [('匿名评分映射','确认 beer_name 物理行序与官方 beer 编号的对应关系'),
                  ('面板监督目标','确认评分员等权面板均分口径'),
                  ('结构模型采用或弃用','确认 ET 主线与层次网络消融的判断'),
                  ('Top-3 最终规则','确认 2+1 与相关性基线差距的解释'),
                  ('缺陷阈值解释','确认 50.0 mg/L 等阈值只作感官参考'),
                  ('生成及迁移可行性','确认两套缺陷口径的表述'),
                  ('正式论文独立重写','确认正文数字与 CSV 一致')]],'audit/human_decision_record.csv')
 # Appendices: all 22 styles, all 36 decisions, all 150 factors.
 append='# 附录：统一50×3候选化合物\n\n最终规则为训练数据内部选择的2+1。频率、单位和风格调整关联见完整CSV。\n\n';pivot=top.pivot(index='t
↪  arget',columns='rank',values='chemical').reset_index();pivot.columns=['感官','候选1','候选2','候选3'];append+=md(pivot)+'\n\n#
↪  附录：22风格独立测试覆盖\n\n';ct=coverage[['style','n_total','n_unique_test','n_correlation_contexts']].copy();ct.columns=['风
↪  格','总样本','独立测试酒数','相关可估计场景'];append+=md(ct)+'\n\n# 附录：
↪  逐风格四模型分布与联合低端结果\n\n';z=read('generation/metrics_22styles_summary.csv')[['style','model','SWD_mean','style_accur
↪  acy_mean','joint_floor_MAE_mean']];z.model=z.model.map(LABEL);z.columns=['风格','模型','SWD','风格一致性','联合低端误差'];appe
↪  nd+=md(z)+'\n\n# 附录：
↪  9酒四策略逐项结果\n\n';z=mig[['beer','policy','target_gain','stale_change','change_RMS','refused']].copy();z.policy=z.policy.
↪  map(LABEL);z.columns=['源beer','策略','目标增益','stale变化','化学RMS','拒绝'];append+=md(z);(R/'paper/rebuild_appendix.md').w
↪  rite_text(append,encoding='utf-8')
 ai=R/'reports/AI_usage_log.md';text=ai.read_text(encoding='utf-8');tag='## 最后三组封闭补充与定稿（gpt-6.1-sol）'
 if tag not in text:ai.write_text(text+'\n\n'+tag+'\n\n用户限定Q2统一、22风格条件生成、9源酒独立迁移三组，禁止增加模型，Q1冻结。OpenAI
↪  Codex gpt-6.1-sol执行训练内选择、原四模型评价与四迁移策略，所有三组于日志记载完成后停止建模探索。此后仅整理CSV、图表、统计、论文、
↪  PDF与提交包。未伪造Faro等无测试类别的指标，未伪造人工签署或队员独立建模记录。\n',encoding='utf-8')
 dump({'Q1':'frozen','Q2':qsum,'generation_macro':g.to_dict('records'),'migration':pmetrics.reset_index().to_dict('records'),'n_
↪  styles_distribution':n_cov,'n_styles_correlation':n_corr,'n_sources':9,'model_exploration':'stopped after exactly 3
↪  groups'},'final_summary.json');log('Final Chinese manuscript and reports complete')
if __name__=='__main__':main()
```

E.5 closed_tables.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Post-experiment tables only."""
from closed import *
def main():
 finalize_generation_metrics()
 names=pd.read_csv(R/'data/processed/chemical_names.csv').rename(columns={'feature':'chemical'});meta=pd.read_csv(R/'data/proces
↪  sed/thresholds.csv').rename(columns={'field_name_R':'chemical'});t=pd.read_csv(F/'q2/top3_50x3.csv');t=t[['method','final_f
↪  ixed_method','target','rank','chemical','selection_score','spearman','style_adjusted_rank_association','bootstrap_frequency
↪  ','alternative_group','group_frequency','final_fixed_method_frequency','label_flag']].merge(names,on='chemical',how='left',
↪  validate='many_to_one').merge(meta,on='chemical',how='left',validate='many_to_one');put(t,'q2/top3_50x3.csv');t.to_csv(O/'q
↪  2/top3_50x3.csv',index=False,encoding='utf-8-sig')
 # Distinguish pipeline uncertainty from fixed-method stability without changing results.
 stab=pd.read_csv(F/'q2/stability.csv').groupby('method')[['compound_jaccard','group_jaccard','within_top3_abs_rho']].mean().res
↪  et_index();put(stab,'q2/stability_summary.csv');bw=pd.read_csv(F/'q2/bootstrap_method_choice.csv');put(bw.winner.value_coun
↪  ts().rename_axis('method').reset_index(name='count'),'q2/bootstrap_choice_summary.csv')
 # Aggregate eligible correlation pairs fairly; record precisely which style-contexts support them.
 df=pd.read_csv(F/'generation/metrics_22styles_by_context.csv');put(df[df.correlation_pairs>0][['seed','style','n_test','model',
↪  'correlation_pairs','correlation_MAE']],'generation/correlation_support.csv')
 # Add measured physical-unit conversions. The nonchemical stale_hops proxy stays separate.
 d=pd.read_csv(F/'migration/defect_checks.csv');trans=pd.read_csv(R/'data/processed/transforms.csv').set_index('feature');th=pd.
↪  read_csv(R/'data/processed/thresholds.csv').set_index('field_name_R');out=[]
 for _,v in d.iterrows():
  rr=v.to_dict();c=v.defect
  if c=='stale_hops':rr.update(unit='sensory score',source_physical=v.source,generated_physical=v.generated,threshold_mean=np.na
  ↪  n,threshold_median=np.nan,conversion='not a unique chemical')
  else:
   tr=trans.loc[c];tt=str(tr['transform']);convert=lambda x:10**x if tt=='log10' else x if tt.startswith('identity') else
   ↪  np.nan;rr.update(unit=tr.unit,source_physical=convert(v.source),generated_physical=convert(v.generated),threshold_mean=th
   ↪  .loc[c,'threshold_mean'] if c in th.index else np.nan,threshold_median=th.loc[c,'threshold_median'] if c in th.index else
   ↪  np.nan,conversion=tt)
  out.append(rr)
 put(out,'migration/defect_checks_physical.csv');m=pd.read_csv(F/'migration/per_beer.csv');accepted=[]
 for policy,g in m.groupby('policy'):
  a=g[~g.refused];accepted.append(dict(policy=policy,n_attempted=len(g),n_accepted=len(a),rejection_rate=g.refused.mean(),all_be
  ↪  er_target_gain=g.target_gain.mean(),accepted_target_gain=a.target_gain.mean() if len(a) else
  ↪  np.nan,all_beer_risk_rate=g.independent_stale_increase.mean(),accepted_risk_rate=a.independent_stale_increase.mean() if
  ↪  len(a) else np.nan))
 put(accepted,'migration/accepted_and_all.csv');dump({'modeling_status':'stopped','completed_groups':3,'new_model_families':0,'Q
↪  1_status':'frozen','next_stage':'paper and submission packaging only'},'modeling_closed.json');log('Final evidence tables
↪  complete; no additional fitting')
if __name__=='__main__':main()
```

E.6 closed_verify.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Closed-result consistency checks."""
from closed import *
def main():
 check=[]
 def add(k,v):check.append(dict(check=k,passed=bool(v)))
 cfg=json.loads((R/'configs/closed_final_protocol.json').read_text())
 for name,sha in cfg['frozen_hashes'].items():add('frozen '+name,hashlib.sha256((R/name).read_bytes()).hexdigest()==sha)
 for _,v in pd.read_csv(O/'audit/raw_manifest.csv').iterrows():add('raw
↪  '+v.file,hashlib.sha256((R/v.file).read_bytes()).hexdigest()==v.sha256)
 q=pd.read_csv(F/'q2/top3_50x3.csv');add('150 distinct factors',len(q)==150 and q.target.nunique()==50 and
↪  q.groupby('target').chemical.nunique().eq(3).all());add('canonical table exact',hashlib.sha256((F/'q2/top3_50x3.csv').read_
↪  bytes()).hexdigest()==hashlib.sha256((O/'q2/top3_50x3.csv').read_bytes()).hexdigest());add('same final
↪  method',q.final_fixed_method.nunique()==1)
 for col in ['bootstrap_frequency','group_frequency','final_fixed_method_frequency']:add(col,q[col].between(0,1).all())
 bs=np.load(F/'q2/bootstrap_selections.npz');add('100 bootstraps all five rules',all(bs[m].shape==(100,50,3) for m in
↪  METHODS+['TrainSelected']))
 df=pd.read_csv(F/'generation/metrics_22styles_by_context.csv');add('22 x4 x3 generation rows',len(df)==264 and
↪  len(df[['seed','style','model']].drop_duplicates())==264);add('no test means NA
↪  distance',df.loc[df.n_test==0,'SWD'].isna().all());add('small n correlation
↪  NA',df.loc[df.n_test<4,'correlation_MAE'].isna().all());add('common correlation-pair
↪  support',df.groupby(['seed','style']).correlation_pairs.nunique().le(1).all());add('style diagnostic
↪  bounded',df.style_accuracy.between(0,1).all())
 sp=pd.read_csv(O/'q3/splits.csv');manifest=json.loads((F/'models/evaluator_manifest.json').read_text())
```

```
 for row in manifest:
  s=row['seed'];p=F/f'models/evaluator_{s}.joblib';add(f'evaluator frozen
  ↪  {s}',hashlib.sha256(p.read_bytes()).hexdigest()==row['sha256']);add(f'evaluator original role
  ↪  {s}',set(row['training_rows'])==set(sp[(sp.seed==s)&(sp.role=='evaluator')].row))
 m=pd.read_csv(F/'migration/per_beer.csv');add('9 x4 migration rows',len(m)==36 and
↪  m.groupby('policy').beer.nunique().eq(9).all());add('no duplicate beer-policy',not
↪  m.duplicated(['beer','policy']).any());r=m[m.policy=='Reject'];add('refusal zero
↪  change',np.allclose(r[['target_gain','stale_change','change_RMS']],0,atol=1e-12) and
↪  r.refused.all());v=m[m.policy.isin(['PointConstraint','UncertaintyConstraint'])];add('chemical clipping on observed
↪  only',v.chemical_observed_increases.eq(0).all());add('design risk constraint',v.design_risk.le(1e-10).all())
 src=pd.read_csv(F/'migration/nine_sources.csv')
 for _,v in src.iterrows():
  add(f'source held out {int(v.beer)}',int(v.row) in set(sp[(sp.seed==v.seed)&(sp.role=='test')].row))
 quality=json.loads((F/'pdf_quality.json').read_text());add('current PDF matches
↪  inspected',hashlib.sha256((R/'paper/competition_final.pdf').read_bytes()).hexdigest()==quality['pdf_sha256']);add('body
↪  limit',quality['within_30']);add('front matter',quality['abstract_first'] and quality['contents_second']);add('no missing
↪  glyphs',not quality['missing_glyphs'])
 for f in ['10_q2_unified','11_generation_styles','12_generation_structure','13_migration_paired','14_migration_risk','15_transf
↪  er_chemistry']:
  for ext in ['png','svg','pdf']:add(f+'.'+ext,(F/f'figures/{f}.{ext}').stat().st_size>1000)
 add('exactly three groups then stopped',json.loads((F/'modeling_closed.json').read_text())['completed_groups']==3);put(check,'v
↪  erification.csv');dump({'checks':len(check),'passed':sum(v['passed'] for v in check),'all_passed':all(v['passed'] for v in
↪  check)},'verification.json');log(f'Closed checks {sum(v["passed"] for v in check)}/{len(check)}');assert all(v['passed'] for
↪  v in check),[v for v in check if not v['passed']]
if __name__=='__main__':main()
```

E.7 figures.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Figures derive from saved outputs."""
from run import *
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import networkx as nx
plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['Arial','DejaVu Sans'],'font.size':10,'axes.spines.top':False
↪  ,'axes.spines.right':False,'axes.linewidth':.8,'svg.fonttype':'none','pdf.fonttype':42})
BLUE='#3775BA';RED='#B64342';GREEN='#42949E'
def save(fig,name):
 fig.tight_layout()
 for ext in ['png','pdf','svg']:fig.savefig(O/f'figures/{name}.{ext}',dpi=300,bbox_inches='tight')
 plt.close(fig)
def main():
 q1=pd.read_csv(O/'q1/repeat_metrics.csv');q2=pd.read_csv(O/'q2/fold_metrics.csv');q3=pd.read_csv(O/'q3/test_metrics.csv');mg=pd
 ↪  .read_csv(O/'q3/migration_metrics.csv')
 fig,ax=plt.subplots(1,2,figsize=(10,4));st=pd.Series(style).value_counts();ax[0].barh(st.index[::-1],st.values[::-1],color=BLUE
 ↪  );ax[0].tick_params(axis='y',labelsize=7);ax[0].set_xlabel('Number of
 ↪  beers');floor=pd.read_csv(O/'audit/features.csv').nlargest(12,'minimum_fraction');ax[1].barh(floor.feature[::-1],floor.mini
 ↪  mum_fraction[::-1],color=GREEN);ax[1].tick_params(axis='y',labelsize=7);ax[1].set_xlabel('Fraction at observed
 ↪  minimum');save(fig,'01_data')
 fig,ax=plt.subplots(1,2,figsize=(10,3.6));g=q1.groupby('model').MAE.agg(['mean','std']).sort_values('mean');ax[0].barh(g.index[
 ↪  ::-1],g['mean'][::-1],xerr=g['std'][::-1],color=BLUE);ax[0].set_xlabel('Raw panel mean MAE (two
 ↪  repeats)');p=pd.read_csv(O/'q1/per_target.csv');v=p[p.model=='ET'].sort_values('MAE',ascending=False).head(12);ax[1].barh(v
 ↪  .target[::-1],v.MAE[::-1],color=GREEN);ax[1].set_xlabel('ET MAE by difficult sensory target');save(fig,'02_q1_accuracy')
 curves=pd.read_csv(O/'q1/training_curves.csv');fig,ax=plt.subplots(1,3,figsize=(10,3));
 for a,m in zip(ax,['MLP','Hierarchy','HierarchyRater']):
  z=curves[curves.model==m].groupby('epoch')[['train_mae','validation_mae']].mean();a.plot(z.index,z.train_mae,label='Training',
  ↪  color=BLUE);a.plot(z.index,z.validation_mae,label='Validation',color=RED);a.set_title(m);a.set_xlabel('Epoch');a.set_ylabe
  ↪  l('MAE');a.legend(fontsize=8)
 save(fig,'03_q1_curves')
 fig,ax=plt.subplots(1,2,figsize=(10,3.6));sta=pd.read_csv(O/'q2/stability.csv');g=q2.groupby('model').MAE.mean().sort_values();
 ↪  ax[0].barh(g.index[::-1],g.values[::-1],color=BLUE);ax[0].set_xlabel('Mean fold MAE, 3
 ↪  compounds');v=sta.groupby('method')[['compound_jaccard','group_jaccard']].mean();v.plot.bar(ax=ax[1],color=[BLUE,GREEN]);ax
 ↪  [1].tick_params(axis='x',rotation=25);ax[1].set_ylabel('Bootstrap Jaccard');ax[1].legend(['Compound','Alternative
 ↪  group'],fontsize=8);save(fig,'04_q2_selection')
```

```
 e=pd.read_csv(O/'q2/signed_edges.csv');G=nx.from_pandas_edgelist(e,'source','target',edge_attr='rho');pos=nx.spring_layout(G,se
↪  ed=20261028,weight=None,k=.8,iterations=400);fig,ax=plt.subplots(figsize=(8,6));nx.draw_networkx_nodes(G,pos,node_size=[10+
↪  4*G.degree(n) for n in G],node_color=BLUE,ax=ax,alpha=.8);nx.draw_networkx_edges(G,pos,edge_color=[RED if G[u][v]['rho']<0
↪  else '#A0B8CD' for u,v in G.edges],width=.7,alpha=.65,ax=ax);labels={n:str(i+1) for i,n in
↪  enumerate(sorted(G,key=G.degree,reverse=True)[:10])};ax.text(1.01,.9,'Highest-degree compounds
↪  (rank)\n'+'\n'.join(labels[n]+': '+n for n in labels),transform=ax.transAxes,fontsize=8,va='top');ax.set_title('Signed
↪  Spearman network, |rho| >= 0.65 (non-isolated nodes)');ax.axis('off');save(fig,'05_q2_network')
 fig,ax=plt.subplots(1,3,figsize=(11,3.5));
 for a,col,title in zip(ax,['SWD','correlation_MAE','floor_fraction_MAE'],['Distribution distance','Rank-dependence
↪  error','Minimum-mass error']):
  v=q3.groupby('model')[col].agg(['mean','std']);a.bar(np.arange(len(v)),v['mean'],yerr=v['std'],color=[BLUE,GREEN,'#8BCF8B',RED
 ↪  ],capsize=3);a.set_xticks(range(len(v)),v.index,rotation=35,ha='right',fontsize=8);a.set_title(title)
 save(fig,'06_q3_quality')
 fig,ax=plt.subplots(1,2,figsize=(10,3.5));g=mg.assign(prob_change=mg.target_probability_after-mg.target_probability_before).gro
↪  upby('method');a=g.prob_change.mean();b=g.independent_stale_change.mean();ax[0].bar(a.index,a.values,color=BLUE);ax[0].set_
↪  ylabel('Independent Tripel probability
↪  change');ax[1].bar(b.index,b.values,color=RED);ax[1].axhline(0,color='black',lw=.8);ax[1].set_ylabel('Independent stale-hops
↪  prediction change');
 for a in ax:a.tick_params(axis='x',rotation=20,labelsize=8)
 save(fig,'07_migration')
 sens=pd.read_csv(O/'q3/generated_sensory50.csv');cols=[c for c in ['A_malt_all','A_hops_all','A_esters_all','F_malt_all','F_hop
↪  s_all','F_esters_all','alcohol','body','sweet','bitter','stale_hops'] if c in
↪  sens];sel=['Tripel','Kriek','Lambic'];mat=sens.groupby('style')[cols].mean().loc[sel];fig,ax=plt.subplots(figsize=(9,2.7));
↪  im=ax.imshow(mat,vmin=0,vmax=6,cmap='YlGnBu',aspect='auto');ax.set_xticks(range(len(cols)),cols,rotation=35,ha='right');ax.
↪  set_yticks(range(3),sel);fig.colorbar(im,ax=ax,label='Predicted raw panel mean');save(fig,'08_generated_portraits')
 fig,ax=plt.subplots(1,2,figsize=(10,3.5));sen=pd.read_csv(O/'q1/sensitivity.csv');v=sen[sen.experiment.str.startswith('ET_mask'
↪  )].groupby('experiment').MAE.mean();ax[0].plot([.05,.1,.2],[v[f'ET_mask_{p}'] for p in
↪  [.05,.1,.2]],marker='o',color=BLUE);ax[0].set_xlabel('Additional random missing fraction');ax[0].set_ylabel('ET
↪  MAE');cv=pd.read_csv(O/'q3/training_curves.csv').groupby('epoch')[['reconstruction','validation_reconstruction']].mean();ax
↪  [1].plot(cv.index,cv.reconstruction,label='Train',color=BLUE);ax[1].plot(cv.index,cv.validation_reconstruction,label='Valid
↪  ation',color=RED);ax[1].set_yscale('log');ax[1].set_xlabel('CVAE epoch');ax[1].set_ylabel('Reconstruction MSE (log
↪  axis)');ax[1].legend();save(fig,'09_sensitivity')
 log('publication figures complete')
if __name__=='__main__':main()
```

E.8 paths.py

```
# AI-assisted OpenAI Codex/GPT-6 series, 2026-10-29. Input-path resolution only; no heavy imports.
"""Resolve input file locations without importing the training pipeline.

`run.py` executes `load()` at module level, so importing anything from it also requires 1.csv, 4.csv
and the per-taster file to exist. Scripts that only need to locate an input (or that should be able to
report "which file is missing" without a traceback) import from here instead.

Search order: data/raw (official competition attachments, supplied by the user) then
data/supplementary (the author's public per-taster file, redistributed with this package).
"""
from pathlib import Path

R = Path(__file__).resolve().parents[2]
BASES = (R / "data/raw", R / "data/supplementary")


def find_input(name, required=True):
    """Return the first existing path for `name`, or None when `required` is False."""
    for base in BASES:
        p = base / name
        if p.is_file():
            return p
    if required:
        raise FileNotFoundError(
            f"cannot find {name} under data/raw or data/supplementary; see README_REBUILD.md "
            f"(official attachments go in data/raw, the per-taster file ships in data/supplementary)"
        )
    return None


def missing_inputs(names):
```

```
    """Names from `names` that are not present anywhere; empty list means all are available."""
    return [n for n in names if find_input(n, required=False) is None]
```

E.9 q1_acetaldehyde.py

```
# AI-assisted OpenAI Codex/GPT-6 series, 2026-10-29. Acetaldehyde grading evidence; no retraining.
"""Grade A_acetaldehyde / F_acetaldehyde with evidence instead of leaving them blank.

Facts established on the released data:
  * 附件 1 `acetaldehyde` is log10-transformed (data/processed/transforms.csv: log10_rmse 2.9e-10
    over 249 paired cells, ln_rmse 0.89, identity_rmse 5.27).
  * Back-transformed concentration spans 0.01-18.56 mg/L (median 4.10) across all 250 beers.
  * 附件 8 gives a sensory detection threshold of 50.0 mg/L for acetaldehyde (origin_paper=Yeast,
    aging=increase, off_flavors="pungent, rotten-apples, green apple").
  * No beer in the dataset exceeds that threshold, so on the sensory scale the whole dataset sits in
    the "not detectable" band.
  * The per-taster file's column that corresponds to acetaldehyde (附件 7 aroma_acetaldehyde) puts
    240/250 beers at its floor value, and that column is uncorrelated with the measured
    concentration (Spearman -0.058), i.e. it is a recoded presence indicator, not a 0-6 magnitude.
  * The officially released 6.csv cross-correlation matrix has 50 sensory columns and NONE of them is
    acetaldehyde, confirming that the authors never published per-taster 0-6 acetaldehyde scores.

Therefore the problem's 3.xlsx legend "0 = Not present" is directly answerable: the defensible grade
is 0 for every beer, derived from a measured concentration that is 2.7x-5000x below the sensory
detection threshold. This is an inference from measurement and threshold, NOT a constant filler and
NOT a fabricated panel score, so it is written to a separate file and labelled as such.

Writes outputs/rebuild/q1_complete/acetaldehyde_graded.csv (beer, concentration, grade, basis).
"""
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.stats import spearmanr

sys.path.insert(0, str(Path(__file__).resolve().parent))
from paths import find_input  # lightweight: does not import the training pipeline

R = Path(__file__).resolve().parents[2]
O = R / "outputs/rebuild"
OUT = O / "q1_complete"
THRESHOLD_MG_L = 50.0  # 附件 8 Sensory thresholds, acetaldehyde, threshold_mean


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    x = pd.read_csv(find_input("1.csv"), usecols=["beer", "acetaldehyde"]).set_index("beer")
    conc = 10 ** x["acetaldehyde"]

    grade = np.where(conc > THRESHOLD_MG_L, 1, 0).astype(int)
    out = pd.DataFrame(
        {
            "beer": conc.index.to_numpy(),
            "acetaldehyde_mg_per_L": conc.to_numpy(),
            "A_acetaldehyde": grade,
            "F_acetaldehyde": grade,
            "basis": " 附件 1 实测浓度经 log10 反变换后仍低于附件 8 感官检出阈值 50.0 mg/L；按 3.xlsx 图例 0=Not present 判定",
        }
    )
    assert out.shape == (250, 5)
    assert (conc <= THRESHOLD_MG_L).all(), "a beer exceeded the threshold; rule must be revisited"
    assert out[["A_acetaldehyde", "F_acetaldehyde"]].isin([0, 1]).all().all()
    out.to_csv(OUT / "acetaldehyde_graded.csv", index=False, encoding="utf-8")

    # supporting evidence, recomputed so the numbers in the paper are traceable
    s7 = pd.read_csv(find_input("7.csv"), usecols=["beer", "aroma_acetaldehyde", "taste_acetaldehyde"]).set_index("beer")
    s7 = s7.reindex(conc.index)
```

```
    floor = float(s7["aroma_acetaldehyde"].min())
    ev = dict(
        transform="log10 (data/processed/transforms.csv, log10_rmse=2.93e-10 over 249 pairs)",
        concentration_mg_per_L=dict(min=round(float(conc.min()), 3), median=round(float(conc.median()), 3),
                                    max=round(float(conc.max()), 3)),
        sensory_detection_threshold_mg_per_L=THRESHOLD_MG_L,
        n_beers_above_threshold=int((conc > THRESHOLD_MG_L).sum()),
        max_concentration_over_threshold_ratio=round(float(conc.max() / THRESHOLD_MG_L), 4),
        panel_floor_fraction_附件 7_aroma=round(float((s7["aroma_acetaldehyde"] <= floor + 1e-9).mean()), 4),
        panel_floor_value_附件 7_aroma=round(floor, 4),
        chemical_vs_panel_spearman=round(float(spearmanr(conc, s7["aroma_acetaldehyde"])[0]), 4),
        official_6csv_has_acetaldehyde_column=False,
        grade_rule="grade = 0 if C <= threshold else 1, then panel-scale mapping; here every beer is below threshold so grade =
        ↪  0",
        caveat=(
            "This is a threshold-anchored inference, not a measured panel score. It answers the 0-6 request "
            "only in the degenerate sense that the whole dataset is below the sensory detection threshold, so "
            "every beer takes the 0 = Not present level. The paper reports it separately from the 50-panel "
            "columns and states that no per-taster 0-6 acetaldehyde record exists in any released file."
        ),
    )
    (OUT / "acetaldehyde_grading.json").write_text(json.dumps(ev, ensure_ascii=False, indent=2), encoding="utf-8")

    print(f"concentration {conc.min():.2f}-{conc.max():.2f} mg/L, threshold {THRESHOLD_MG_L} mg/L")
    print(f"beers above threshold: {(conc > THRESHOLD_MG_L).sum()}/250 -> grade all 0")
    print(f"panel floor fraction: {ev['panel_floor_fraction_附件7_aroma']:.3f}, chemical-vs-panel spearman
    ↪  {ev['chemical_vs_panel_spearman']:+.4f}")
    print("wrote", str(OUT.relative_to(R)) + "/acetaldehyde_graded.csv")


if __name__ == "__main__":
    main()
```

E.10 q1_complete.py

```
# AI-assisted OpenAI Codex/GPT-6 series, 2026-10-29. Q1 complete scorecard assembly; no retraining.
"""Assemble the complete 3.xlsx scorecard. Deterministic: the acetaldehyde columns are always graded.

Provenance of the 53 delivered columns (250 rows, no missing values):
  * 1 column  `beer`
  * 50 columns  0-6 scores predicted by the frozen Q1 model (`q1/grades_oof_panel_mean.csv`),
                supervised by the equal-rater panel mean built from 附件 4 + the author's per-taster file
  * 2 columns  `A_acetaldehyde`, `F_acetaldehyde`: threshold-anchored inference, NOT panel scores.
                No released file carries a 0-6 acetaldehyde record (3.xlsx summary, 4.csv, 6.csv and
                the per-taster file all omit it), but 附件 1 measures the concentration and 附件 8 gives a
                50.0 mg/L sensory detection threshold; every beer is below it, so the score is 0
                (= "Not present" in the 3.xlsx legend). Computed by q1_acetaldehyde.py.

This script has no "if a previous file exists" branch: the acetaldehyde grade is recomputed from the
raw chemistry on every run, so a clean checkout and a dirty workspace produce identical output.

Also fixes the original delivery bug: an earlier version filtered the frozen OOF grade table by the
`A_`/`F_` column prefix, which silently dropped the 20 unprefixed scorecard fields (body, bitternes,
co2, aftertaste, alcohol, sweetness, acidity, dms, metallic, 4vg, t2n, diacetyl, stale_hops, orange,
coriander, clove, lactic, acetic, barnyard, overall) and left only 30 graded fields.

Outputs:
  q1/aroma_flavour_template_with_unobserved.csv   the deliverable (53 columns, kept at its frozen path)
  q1_complete/scorecard_complete.csv              50 graded fields only, no acetaldehyde columns
  q1_complete/scorecard_complete_provenance.csv   per-column source
  q1_complete/acetaldehyde_graded.csv             concentration, grade and basis per beer
"""
import os
import sys
from pathlib import Path

import numpy as np
import pandas as pd
```

```
sys.path.insert(0, str(Path(__file__).resolve().parent))
from paths import find_input  # lightweight: does not import the training pipeline

R = Path(__file__).resolve().parents[2]
O = R / "outputs/rebuild"
OUTD = O / "q1_complete"
THRESHOLD_MG_L = 50.0  # 附件 8 Sensory thresholds, acetaldehyde, threshold_mean

# 3.xlsx `summary` sheet order (verified: 52 columns = Beer + 50 sensory + comments).
SCORECARD_ORDER = [
    "A_malt_all", "A_malt_grain", "A_malt_bread", "A_malt_cara", "A_malt_burn",
    "A_hops_all", "A_hops_citrus", "A_hops_tropical", "A_hops_noble", "A_hops_woody",
    "A_esters_all", "A_esters_ethac", "A_esters_isoaa", "A_esters_flower", "A_ester_fruity",
    "F_malt_all", "F_malt_grain", "F_malt_bread", "F_malt_cara", "F_malt_burn",
    "F_hops_all", "F_hops_citrus", "F_hops_tropical", "F_hops_noble", "F_hops_woody",
    "F_esters_all", "F_esters_ethac", "F_esters_isoaa", "F_esters_flower", "F_ester_fruity",
    "body", "bitternes", "co2", "aftertaste", "alcohol", "sweetness", "acidity",
    "dms", "metallic", "4vg", "t2n", "diacetyl", "stale_hops",
    "orange", "coriander", "clove", "lactic", "acetic", "barnyard", "overall",
]
UNOBSERVED = ["A_acetaldehyde", "F_acetaldehyde"]

# 3.xlsx name -> model target name used throughout src/rebuild.
ALIAS = {"A_ester_fruity": "A_esters_fruity", "F_ester_fruity": "F_esters_fruity", "4vg": "X4vg"}

# Which 3.xlsx scorecard column each canonical field is scored in (per-beer sheet labels).
AROMA_SIDE = [
    "A_malt_all", "A_malt_grain", "A_malt_bread", "A_malt_cara", "A_malt_burn",
    "A_hops_all", "A_hops_citrus", "A_hops_tropical", "A_hops_noble", "A_hops_woody",
    "A_esters_all", "A_esters_ethac", "A_esters_isoaa", "A_esters_flower", "A_ester_fruity",
    "body", "bitternes", "co2", "aftertaste", "dms", "metallic", "4vg",
    "orange", "coriander", "clove", "overall",
]
FLAVOUR_SIDE = [
    "F_malt_all", "F_malt_grain", "F_malt_bread", "F_malt_cara", "F_malt_burn",
    "F_hops_all", "F_hops_citrus", "F_hops_tropical", "F_hops_noble", "F_hops_woody",
    "F_esters_all", "F_esters_ethac", "F_esters_isoaa", "F_esters_flower", "F_ester_fruity",
    "alcohol", "sweetness", "acidity", "t2n", "diacetyl", "stale_hops",
    "lactic", "acetic", "barnyard",
]
SIDE = {c: "Aroma" for c in AROMA_SIDE} | {c: "Flavour" for c in FLAVOUR_SIDE}
assert len(AROMA_SIDE) == 26 and len(FLAVOUR_SIDE) == 24 and set(SIDE) == set(SCORECARD_ORDER)


def acetaldehyde_grades():
    """Recompute the threshold-anchored acetaldehyde score from raw chemistry, every run."""
    x = pd.read_csv(find_input("1.csv"), usecols=["beer", "acetaldehyde"]).set_index("beer")
    conc = 10 ** x["acetaldehyde"]          # 附件 1 is log10-transformed (data/processed/transforms.csv)
    grade = (conc > THRESHOLD_MG_L).astype(int)
    if (grade == 1).any():
        # If any beer ever crosses the threshold the binary rule is no longer adequate and a graded
        # mapping must be designed; fail loudly rather than silently shipping a degenerate column.
        raise AssertionError(
            f"{int(grade.sum())} beers exceed {THRESHOLD_MG_L} mg/L; the threshold-anchored rule needs review")
    out = pd.DataFrame(
        {
            "beer": conc.index.to_numpy(),
            "acetaldehyde_mg_per_L": conc.to_numpy(),
            "A_acetaldehyde": grade.to_numpy(),
            "F_acetaldehyde": grade.to_numpy(),
            "basis": " 附件 1 实测浓度经 log10 反变换后仍低于附件 8 感官检出阈值 50.0 mg/L；按 3.xlsx 图例 0=Not present 判定",
        }
    )
    assert out.shape == (250, 5)
    return out


def complete_scorecard(with_acetaldehyde=True):
    """beer + 50 canonical 0-6 fields, plus the two threshold-anchored acetaldehyde columns."""
    grades = pd.read_csv(O / "q1/grades_oof_panel_mean.csv")
    assert grades.shape == (250, 51), grades.shape
```

```
    assert "beer" in grades.columns and sorted(grades.beer.tolist()) == list(range(1, 251))

    missing = [c for c in SCORECARD_ORDER if ALIAS.get(c, c) not in grades.columns]
    assert not missing, f"scorecard fields absent from the frozen OOF grades: {missing}"

    out = pd.DataFrame({"beer": grades.beer.to_numpy()})
    for field in SCORECARD_ORDER:
        out[field] = grades[ALIAS.get(field, field)].to_numpy()

    if with_acetaldehyde:
        acet = acetaldehyde_grades().set_index("beer").reindex(out.beer.to_numpy())
        for field in UNOBSERVED:
            out[field] = acet[field].to_numpy()
        assert out[UNOBSERVED].notna().all().all()
        assert out[UNOBSERVED].isin(range(0, 7)).all().all()

    assert out.shape == (250, 51 if not with_acetaldehyde else 53)
    assert out[SCORECARD_ORDER].notna().all().all()
    assert out[SCORECARD_ORDER].isin(range(0, 7)).all().all()
    assert set(AROMA_SIDE) | set(FLAVOUR_SIDE) == set(SCORECARD_ORDER)
    return out


def main():
    OUTD.mkdir(parents=True, exist_ok=True)
    acet = acetaldehyde_grades()
    acet.to_csv(OUTD / "acetaldehyde_graded.csv", index=False, encoding="utf-8")

    out = complete_scorecard(with_acetaldehyde=True)
    out.to_csv(O / "q1/aroma_flavour_template_with_unobserved.csv", index=False, encoding="utf-8-sig")
    out.to_csv(OUTD / "scorecard_complete.csv", index=False, encoding="utf-8-sig")

    prov = pd.DataFrame(
        {
            "column": SCORECARD_ORDER + UNOBSERVED,
            "scorecard_side": [SIDE[c] for c in SCORECARD_ORDER] + ["Aroma", "Flavour"],
            "kind": ["grade_0_6_panel_model"] * len(SCORECARD_ORDER)
            + ["grade_0_6_threshold_anchored"] * len(UNOBSERVED),
            "supervision": [" 附件 4 + 逐评分员原始记录，同评分员重复记录先平均，评分员间等权"] * len(SCORECARD_ORDER)
            + ["无 0-6 记录；由附件1实测浓度(log10反变换)对附件8感官检出阈值50.0mg/L判定，全部低于阈值故为0=Not present"] *
            ↪  len(UNOBSERVED),
            "is_panel_score": ["yes"] * len(SCORECARD_ORDER) + ["no"] * len(UNOBSERVED),
            "model_target": [ALIAS.get(c, c) for c in SCORECARD_ORDER] + ["", ""],
        }
    )
    assert len(prov) == 52
    prov.to_csv(OUTD / "scorecard_complete_provenance.csv", index=False, encoding="utf-8")

    # Supplementary standardized acetaldehyde concepts, when the official attachment is present.
    try:
        s = pd.read_csv(find_input("7.csv"), usecols=["beer", "aroma_acetaldehyde", "taste_acetaldehyde"])
    except FileNotFoundError:
        print("skip acetaldehyde_supplementary.csv: 附件 7 not found")
    else:
        s = s.set_index("beer").reindex(out.beer.to_numpy())
        sup = pd.DataFrame(
            {
                "beer": out.beer.to_numpy(),
                "A_acetaldehyde_z": s["aroma_acetaldehyde"].to_numpy(),
                "F_acetaldehyde_z": s["taste_acetaldehyde"].to_numpy(),
            }
        )
        assert sup.shape == (250, 3) and sup.notna().all().all()
        sup.to_csv(OUTD / "acetaldehyde_supplementary.csv", index=False, encoding="utf-8")

    print(f"wrote q1/aroma_flavour_template_with_unobserved.csv rows={len(out)} columns={out.shape[1]} "
          f"(50 panel-modelled + 2 threshold-anchored + beer)")
    print(f"Aroma={len(AROMA_SIDE)} Flavour={len(FLAVOUR_SIDE)} graded={len(SCORECARD_ORDER)} "
          f"acetaldehyde_all_zero={bool((acet[UNOBSERVED].to_numpy() == 0).all())} "
          f"NaN cells={int(out.isna().sum().sum())}")
```

```
if __name__ == "__main__":
    main()
```

E.11 q1_grade_range.py

```
""" AI-assisted OpenAI Codex/GPT-6 series, 2026-10-29. Diagnostics for Q1 grade-range compression."""
from pathlib import Path

import numpy as np
import pandas as pd

R = Path(__file__).resolve().parents[2]
O = R / "outputs/rebuild"
OUT = O / "q1_complete"


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    off = pd.read_csv(R / "data/raw/4.csv")
    targets = list(off.columns[3:])
    raw = pd.read_csv(R / "data/supplementary/Raw sensory data anonymized.csv", encoding="cp1252").rename(
        columns={"A_ester_fruity": "A_esters_fruity", "F_ester_fruity": "F_esters_fruity"}
    )
    Y = raw.groupby("beer_name")[targets].mean().reindex(range(1, 251)).set_index(off.beer)

    grades = pd.read_csv(O / "q1/grades_oof_panel_mean.csv")
    cont = pd.read_csv(O / "q1/continuous_oof.csv")
    truth_grade = np.floor(Y[targets].to_numpy() + 0.5)
    pred_grade = grades[targets].to_numpy()
    pred_cont = cont[targets].to_numpy()

    rows = []
    for lo, hi, label in [(0, 2, "[0,2)"), (2, 3, "[2,3)"), (3, 4, "[3,4)"), (4, 5, "[4,5)"), (5, 7, "[5,7)")]:
        sel = (truth_grade >= lo) & (truth_grade < hi)
        n = int(sel.sum())
        if not n:
            continue
        rows.append(
            dict(
                truth_band=label,
                n_cells=n,
                truth_mean=float(truth_grade[sel].mean()),
                pred_grade_mean=float(pred_grade[sel].mean()),
                pred_continuous_mean=float(pred_cont[sel].mean()),
                grade_bias=float(pred_grade[sel].mean() - truth_grade[sel].mean()),
                continuous_bias=float(pred_cont[sel].mean() - truth_grade[sel].mean()),
                underpredicted_rate=float((pred_grade[sel] < truth_grade[sel]).mean()),
            )
        )
    band = pd.DataFrame(rows)
    band.to_csv(OUT / "grade_range_diagnostic.csv", index=False, encoding="utf-8")

    # continuous predictions are clipped at 6, so check whether the ceiling itself is the cause
    capped = dict(
        truth_cells_ge5=int((truth_grade >= 5).sum()),
        pred_cells_ge5=int((pred_grade >= 5).sum()),
        pred_cells_ge5_before_rounding=int((pred_cont >= 4.5).sum()),
        pred_continuous_max=float(pred_cont.max()),
        truth_grade_hist={str(k): int(v) for k, v in pd.Series(truth_grade.ravel()).value_counts().sort_index().items()},
        pred_grade_hist={str(k): int(v) for k, v in pd.Series(pred_grade.ravel()).value_counts().sort_index().items()},
        n_cells=int(truth_grade.size),
    )
    summary = dict(
        finding=(
            "The shipped scorecard never exceeds grade 4, while the rounded panel mean does reach 5 (112 cells) "
            "and 6 (9 cells). The compression is not a rounding artefact: the continuous predictions themselves "
            "are pulled toward the middle, so the model systematically under-predicts high-intensity samples and "
            "slightly over-predicts low ones. This is a genuine limitation of predicting a panel mean from "
```

```
            "chemistry with MAE ~0.45 on a 0-6 scale, and it must be reported rather than hidden."
        ),
        by_truth_band=band.to_dict("records"),
        counts=capped,
        consequence=(
            "Grades 5-6 are effectively unavailable in the deliverable, so any downstream use that depends on "
            "identifying intense samples (e.g. flagging a beer as strongly hoppy or strongly defective) will miss "
            "them. The 50 predicted columns are unbiased only in the aggregate sense of a small MAE, not in the "
            "tails."
        ),
    )
    (OUT / "grade_range_summary.json").write_text(
        __import__("json").dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )

    print(band.to_string(index=False))
    print()
    print("truth cells >=5:", capped["truth_cells_ge5"], "| predicted cells >=5:", capped["pred_cells_ge5"])
    print("continuous max:", round(capped["pred_continuous_max"], 4),
          "| continuous >=4.5 (would round to 5):", capped["pred_cells_ge5_before_rounding"])
    print("wrote", str(OUT.relative_to(R)) + "/grade_range_diagnostic.csv + grade_range_summary.json")


if __name__ == "__main__":
    main()
```

E.12 q1_scale_check.py

```
# AI-assisted OpenAI Codex/GPT-6 series, 2026-10-28. Q1 scale-comparability diagnostic; no model selection, no frozen artifact
↪  rewritten.
"""Put the ET_official_Z_scale control on the same 0-6 footing as the shipped Q1 results.

`supplement.py` trains ET on 附件 4's standardized values `official = off[targets]` and scores MAE
against those same standardized values, so its 0.3413 is in standardized units, not score units.
Because 附件 4 is a per-dimension affine recoding of the panel mean (mean |residual| 0.1143,
correlation 0.9672 across the 50 targets), the control can be mapped back through the per-dimension
affine fit learned on the training fold only and then compared with the shipped MAE.

Writes outputs/rebuild/q1_complete/scale_comparability.csv only; nothing under q1/ is touched.
"""
import sys
from pathlib import Path

import numpy as np
import pandas as pd

sys.path.insert(0, str(Path(__file__).resolve().parent))
from run import C, O, Prep, X, Y, basic, folds, off, r2_score, targets  # noqa: E402

OUTD = O / "q1_complete"


def main():
    official = off[targets].to_numpy()
    rows = []
    for s in C["seeds"]:
        f = folds(s)
        for k in range(3):
            tr = np.flatnonzero(f != k)
            te = np.flatnonzero(f == k)
            pre = Prep().fit(X[tr])
            a = pre.transform(X[tr])

            direct = basic("ET", s=s).fit(a, Y[tr]).predict(pre.transform(X[te]))
            mae_direct = float(np.abs(direct - Y[te]).mean())

            z = basic("ET", s=s).fit(a, official[tr]).predict(pre.transform(X[te]))
            mae_z_units = float(np.abs(z - official[te]).mean())
```

```
            # Per-dimension affine map Z -> Y fitted on the training fold only.
            fitted = np.zeros_like(z)
            for j in range(len(targets)):
                yy = Y[tr][:, j]
                zz = official[tr][:, j]
                if np.std(zz) < 1e-12:
                    fitted[:, j] = yy.mean()
                else:
                    b = float(np.polyfit(zz, yy, 1)[0])
                    a0 = float(np.polyfit(zz, yy, 1)[1])
                    fitted[:, j] = a0 + b * z[:, j]
            fitted = np.clip(fitted, 0, 6)
            mae_z_mapped = float(np.abs(fitted - Y[te]).mean())
            rows.append(
                dict(
                    seed=s,
                    fold=k,
                    MAE_direct_score_units=mae_direct,
                    MAE_official_Z_units=mae_z_units,
                    MAE_official_Z_mapped_to_score=mae_z_mapped,
                    R2_official_Z_mapped=float(r2_score(Y[te], fitted)),
                    score_unit_scale=float(mae_z_mapped / mae_z_units) if mae_z_units else np.nan,
                )
            )
            print(f"seed {s} fold {k}: direct {mae_direct:.4f} | Z-units {mae_z_units:.4f} | Z->score {mae_z_mapped:.4f}")

    df = pd.DataFrame(rows)
    OUTD.mkdir(parents=True, exist_ok=True)
    df.to_csv(OUTD / "scale_comparability.csv", index=False, encoding="utf-8")
    print("\nmean over 6 outer folds:")
    print(df[["MAE_direct_score_units", "MAE_official_Z_units", "MAE_official_Z_mapped_to_score"]].mean().round(4).to_string())
    print(f"wrote q1_complete/scale_comparability.csv rows={len(df)}")


if __name__ == "__main__":
    main()
```

E.13 q2.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28; fold-isolated selection."""
from run import *
import networkx as nx
from scipy.cluster.hierarchy import linkage,fcluster
from scipy.spatial.distance import squareform
EXCLUDE={'color.EBC.','pH','bitterness','CO2.PSI.','GFS','unfermentables','kcalperc','betaglucan.mg.L.','protein.g.L.','esters_s
↪  um','aroma_hops_sum','acids_sum','sulfur_sum'}
allowed=np.array([j for j,c in enumerate(chem) if c not in EXCLUDE])
def unit(a):
 a=a-a.mean(0);return a/np.maximum(np.sqrt((a*a).sum(0)),1e-12)
def select(a,y,method):
 z=unit(rankdata(a[:,allowed],axis=0));t=unit(rankdata(y,axis=0));rho=z.T@t;result=[];scores=[]
 for j in range(y.shape[1]):
  chosen=[];ss=[]
  for k in range(3):
   if method=='Spearman' or (method=='Hybrid2plus1' and k<2):score=abs(rho[:,j])
   elif method=='Redundancy':score=abs(rho[:,j])-(.5*np.max(abs(z.T@z[:,chosen]),axis=1) if chosen else 0)
   elif not chosen:score=rho[:,j]**2
   else:
    b=z[:,chosen];inv=np.linalg.inv(b.T@b+.02*np.eye(k));xr=z-b@inv@(b.T@z);yr=t[:,j]-b@inv@(b.T@t[:,j]);score=(xr.T@yr)**2/(np.
    ↪  sum(xr*xr,axis=0)+.02)
   score=score.copy();score[chosen]=-np.inf;ii=int(np.argmax(score));chosen.append(ii);ss.append(float(score[ii]))
  result.append(allowed[chosen]);scores.append(ss)
 return np.array(result),np.array(scores)
def residual(a,sty):
 b=a.copy()
 for st in np.unique(sty):ix=sty==st;b[ix]-=b[ix].mean(0)
 return b

def main():
```

```
 rows=[];sr=[];preds={m:[] for m in C['q2_methods']+['Forward3_styleAdjusted']}
 for s in C['seeds']:
  f=folds(s);out={m:np.zeros_like(Y) for m in preds}
  for k in range(3):
   tr=np.flatnonzero(f!=k);te=np.flatnonzero(f==k);pre=Prep().fit(X[tr]);a,b=pre.transform(X[tr]),pre.transform(X[te])
   for m in preds:
    ar,yr=(residual(a,style[tr]),residual(Y[tr],style[tr])) if m.endswith('styleAdjusted') else
    ↪  (a,Y[tr]);idx,score=select(ar,yr,'Forward3' if m.endswith('styleAdjusted') else m)
    for j in range(50):
     model=Ridge(alpha=8).fit(a[:,idx[j]],Y[tr,j]);out[m][te,j]=np.clip(model.predict(b[:,idx[j]]),0,6)
     for rank,q in enumerate(idx[j]):sr.append(dict(seed=s,fold=k,method=m,target=targets[j],rank=rank+1,chemical=chem[q],score=
     ↪  score[j,rank]))
    rows.append(dict(seed=s,fold=k,model=m,**metric(Y[te],out[m][te])));log(f'Q2 {s}_{k} {m} {rows[-1]["MAE"]:.4f}')
  for m in out:preds[m].append(out[m]);csv(pd.DataFrame(out[m],columns=targets).assign(beer=xdf.beer),f'q2/oof_{m}_{s}.csv')
 csv(rows,'q2/fold_metrics.csv');csv(sr,'q2/fold_selections.csv');statistics(preds,Y,'q2','Forward3')
 names=pd.read_csv(R/'data/processed/chemical_names.csv').set_index('feature').matrix_name;matrix=pd.read_csv(raw_path('2.csv'),
↪  index_col=0);mapped=[names[c] for c in chem];rho=matrix.loc[mapped,mapped].to_numpy(float)
G=nx.Graph();G.add_nodes_from(range(231));edges=[]
for i in range(231):
 for j in range(i):
  if abs(rho[i,j])>=.65:G.add_edge(i,j,weight=float(abs(rho[i,j])),rho=float(rho[i,j]));edges.append(dict(source=chem[i],target
  ↪  =chem[j],rho=rho[i,j],sign='positive' if rho[i,j]>0 else 'negative'))
 csv(edges,'q2/signed_edges.csv');nx.write_graphml(G,O/'q2/network.graphml');groups=fcluster(linkage(squareform(np.clip(1-abs(rh
↪  o),0,1),checks=False),method='complete'),.2,criterion='distance');csv(pd.DataFrame({'chemical':chem,'alternative_group':gro
↪  ups}),'q2/alternative_groups.csv')
 a=Prep().fit(X).transform(X);rng=np.random.default_rng(C['seeds'][0]);boots={m:[] for m in C['q2_methods']}
 for it in range(C['q2_bootstraps']):
  ix=np.concatenate([rng.choice(np.flatnonzero(style==st),sum(style==st),replace=True) for st in
 ↪  np.unique(style)]);aa=Prep().fit(X[ix]).transform(X[ix])
  for m in boots:boots[m].append(select(aa,Y[ix],m)[0])
  if (it+1)%20==0:log(f'Q2 bootstrap {it+1}')
 np.savez_compressed(O/'q2/bootstrap_selections.npz',**{k:np.array(v) for k,v in
↪  boots.items()});top=[];stability=[];compare=[];finals={m:select(a,Y,m) for m in
↪  boots};adj,_=select(residual(a,style),residual(Y,style),'Forward3');rawrho=unit(rankdata(a,axis=0)).T@unit(rankdata(Y,axis=
↪  0));adjrho=unit(rankdata(residual(a,style),axis=0)).T@unit(rankdata(residual(Y,style),axis=0))
 for m,(idx,score) in finals.items():
  bs=np.array(boots[m])
  for j,t in enumerate(targets):
   jac=[];gjac=[]
   for b in range(len(bs)-1):
    u,v=set(bs[b,j]),set(bs[b+1,j]);gu,gv=set(groups[list(u)]),set(groups[list(v)]);jac.append(len(u&v)/len(u|v));gjac.append(le
    ↪  n(gu&gv)/len(gu|gv))
   stability.append(dict(method=m,target=t,compound_jaccard=np.mean(jac),group_jaccard=np.mean(gjac),within_top3_abs_rho=float(n
   ↪  p.mean([abs(rho[idx[j,u],idx[j,v]]) for u,v in [(0,1),(0,2),(1,2)]]))))
   for rank,q in enumerate(idx[j]):top.append(dict(method=m,target=t,rank=rank+1,chemical=chem[q],name_in_2csv=names[chem[q]],se
   ↪  lection_score=score[j,rank],spearman=rawrho[q,j],style_adjusted_rank_association=adjrho[q,j],frequency=float((bs[:,j,:]==
   ↪  q).any(1).mean()),alternative_group=int(groups[q]),group_frequency=float((groups[bs[:,j,:]]==groups[q]).any(1).mean()),in
   ↪  _style_adjusted_forward=bool(q in adj[j]),label_flag='suspected formula alias' if t=='barnyard' else ''))
   for m2,(i2,_) in finals.items():
    u,v=set(idx[j]),set(i2[j]);compare.append(dict(method1=m,method2=m2,target=t,jaccard=len(u&v)/len(u|v)))
 csv(top,'q2/top3_all_methods.csv');csv([r for r in top if
↪  r['method']=='Forward3'],'q2/top3_50x3.csv');csv(stability,'q2/stability.csv');csv(compare,'q2/method_jaccard.csv');js({'n_
↪  nodes':231,'edges':len(edges),'excluded':sorted(EXCLUDE),'candidate_count':len(allowed),'final_rule':'Prespecified Forward3;
↪  all alternatives reported, not selected by outer test.'},'q2/summary.json')
if __name__=='__main__':main()
```

E.14 q3.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Disjoint generator/evaluator fits."""
from run import *
from sklearn.metrics import accuracy_score,balanced_accuracy_score
ST=sorted(np.unique(style));SI={s:i for i,s in enumerate(ST)}
class CV(nn.Module):
 def __init__(self):
  super().__init__();self.enc=nn.Sequential(nn.Linear(253,64),nn.ReLU());self.mu=nn.Linear(64,12);self.lv=nn.Linear(64,12);self.
  ↪  dec=nn.Sequential(nn.Linear(34,64),nn.ReLU(),nn.Linear(64,231))
 def forward(self,a,c):
```

```
  h=self.enc(torch.cat([a,c],1));mu=self.mu(h);lv=self.lv(h).clamp(-8,5);z=mu+torch.randn_like(mu)*torch.exp(.5*lv);return
  ↪  self.dec(torch.cat([z,c],1)),mu,lv
class Generator:
 def __init__(self,kind,s):self.kind=kind;self.s=s
 def fit(self,data,sty,va=None,vs=None):
  self.pre=Prep().fit(data);self.a=self.pre.imp.transform(data);self.sty=np.array(sty);self.low=self.a.min(0);self.high=self.a.m
  ↪  ax(0);self.floor=(self.a==self.low).mean(0)>=.25;self.z=self.pre.transform(data);self.history=[]
  if self.kind.startswith('Copula'):
   ranks=rankdata(self.a,axis=0);u=(ranks-.5)/len(self.a);z=norm.ppf(u);z=z-z.mean(0);cov=LedoitWolf().fit(z).covariance_;d=np.s
   ↪  qrt(np.diag(cov));self.corr=cov/d[:,None]/d[None,:];self.chol=np.linalg.cholesky(self.corr+1e-6*np.eye(231));self.sorted=
   ↪  np.argsort(self.a,axis=0)
  elif self.kind=='PCA_Gaussian':
   self.pca=PCA(n_components=12,random_state=self.s).fit(self.z);self.lat=self.pca.transform(self.z);self.cov=LedoitWolf().fit(s
   ↪  elf.lat).covariance_
  else:
   seed(self.s);self.net=CV().to(DEVICE);opt=torch.optim.AdamW(self.net.parameters(),lr=.002,weight_decay=.001);a=torch.tensor(s
   ↪  elf.z,dtype=torch.float32,device=DEVICE);c=torch.eye(22,device=DEVICE)[[SI[s] for s in sty]];best=np.inf;state=None
   if va is not
   ↪  None:av=torch.tensor(self.pre.transform(va),dtype=torch.float32,device=DEVICE);cv=torch.eye(22,device=DEVICE)[[SI[s] for s
   ↪  in vs]]
   for ep in range(160):
    self.net.train();opt.zero_grad();re,mu,lv=self.net(a,c);rec=((re-a)**2).mean();kl=-.5*(1+lv-mu**2-lv.exp()).mean();loss=rec+
    ↪  .05*kl;loss.backward();opt.step();self.net.eval()
    with torch.no_grad():
     if va is not None:
      h=self.net.enc(torch.cat([av,cv],1));r=self.net.dec(torch.cat([self.net.mu(h),cv],1));v=float(((r-av)**2).mean())
     else:v=float(rec)
    self.history.append(dict(epoch=ep+1,reconstruction=float(rec.detach()),KL=float(kl.detach()),validation_reconstruction=v))
    if v<best:best=v;state={k:v.detach().cpu().clone() for k,v in self.net.state_dict().items()}
   self.net.load_state_dict(state);self.params=sum(p.numel() for p in self.net.parameters())
  return self
 def sample(self,st,n,rng):
  ix=np.flatnonzero(self.sty==st);w=len(ix)/(len(ix)+10)
  if self.kind.startswith('Copula'):
   z=rng.normal(size=(n,231));z=z@self.chol.T if self.kind=='CopulaJoint' else
   ↪  z;u=norm.cdf(z);weights=np.full(len(self.a),(1-w)/len(self.a));weights[ix]+=w/max(1,len(ix));out=np.empty((n,231))
   for j in range(231):
    order=self.sorted[:,j];cw=np.cumsum(weights[order]);out[:,j]=self.a[order[np.minimum(np.searchsorted(cw,u[:,j]),len(order)-1
    ↪  )],j]
   return out
  if self.kind=='PCA_Gaussian':
   mu=w*self.lat[ix].mean(0)+(1-w)*self.lat.mean(0) if len(ix) else self.lat.mean(0);z=rng.multivariate_normal(mu,self.cov,size=
   ↪  n);out=self.pre.scaler.inverse_transform(self.pca.inverse_transform(z))
  else:
   self.net.eval()
   with torch.no_grad():out=self.net.dec(torch.cat([torch.tensor(rng.normal(size=(n,12)),dtype=torch.float32,device=DEVICE),torc
   ↪  h.eye(22,device=DEVICE)[[SI[st]]*n]],1)).cpu().numpy()
   out=self.pre.scaler.inverse_transform(out)
  return np.clip(out,self.low,self.high)
def split(s,rep):
 rng=np.random.default_rng(s);pils=np.flatnonzero(style=='Pils/Lager');test=list(pils[(np.arange(3)+rep*3)%len(pils)]);left=set(
 ↪  range(250))-set(test);g=[];e=[]
 for st in ST:
  ids=np.array(sorted(left &
  ↪  set(np.flatnonzero(style==st))));rng.shuffle(ids);g.append(int(ids[0]));e.append(int(ids[1]));left-=set(ids[:2])
 a=np.array(sorted(left));rng.shuffle(a);pos=0
 def take(n):
  nonlocal pos
  v=list(a[pos:pos+n]);pos+=n;return v
 g+=take(140-len(g));e+=take(40-len(e));v=take(35);test+=take(35-len(test));assert pos==len(a)
 return [np.array(z,int) for z in (g,v,e,test)]
def distances(real,gen,pre,train,rng):
 a,b=pre.transform(real),pre.transform(gen);d=np.random.default_rng(801).normal(size=(231,48));d/=np.linalg.norm(d,axis=0);sw=np
 ↪  .mean([wasserstein_distance(a@d[:,i],b@d[:,i]) for i in
 ↪  range(48)]);floor=pre.imp.transform(train).min(0);eligible=(pre.imp.transform(train)==floor).mean(0)>=.25
 ca=pd.DataFrame(a).corr(method='spearman').to_numpy();cb=pd.DataFrame(b).corr(method='spearman').to_numpy();mask=np.triu(np.one
 ↪  s((231,231),bool),1)&np.isfinite(ca)&np.isfinite(cb);nn=cdist(b,pre.transform(train))/np.sqrt(231)
 return dict(SWD=float(sw),marginal_W1=float(np.mean([wasserstein_distance(a[:,j],b[:,j]) for j in
 ↪  range(231)])),correlation_MAE=float(abs(ca[mask]-cb[mask]).mean()) if mask.any() else np.nan,floor_fraction_MAE=float(abs((
 ↪  pre.imp.transform(real)==floor).mean(0)[eligible]-(gen==floor).mean(0)[eligible]).mean()),nearest_train_RMS=float(nn.min(1)
 ↪  .mean()),exact_copy_fraction=float((nn.min(1)<1e-8).mean()),diversity=float(np.std(b,axis=0).mean()))
```

```
def assess(model,real,realsty,train,clf,pre_eval,rng):
 generated=np.vstack([model.sample(st,1,rng) for st in realsty]);d=distances(real,generated,model.pre,train,rng);pred=clf.predic
 ↪  t(pre_eval.transform(generated));d['style_accuracy']=float(np.mean(pred==realsty));return d,generated

def migrate(gen,train_ix,eval_ix,sources,rep,scope):
 rng=np.random.default_rng(20261028+rep);pre=Prep().fit(X[train_ix]);a=pre.transform(X[train_ix]);models=[]
 for k in range(12):
  ids=rng.integers(len(train_ix),size=len(train_ix));models.append(ExtraTreesRegressor(n_estimators=60,min_samples_leaf=3,max_fe
  ↪  atures=.7,n_jobs=2,random_state=k+rep).fit(a[ids],Y[train_ix[ids]]))
 ep=Prep().fit(X[eval_ix]);ec=ExtraTreesClassifier(n_estimators=180,min_samples_leaf=2,max_features=.7,n_jobs=2,random_state=rep
 ↪  ).fit(ep.transform(X[eval_ix]),style[eval_ix]);er=ExtraTreesRegressor(n_estimators=180,min_samples_leaf=2,max_features=.7,n
 ↪  _jobs=2,random_state=rep).fit(ep.transform(X[eval_ix]),Y[eval_ix]);tj=list(ec.classes_).index('Tripel');sj=targets.index('s
 ↪  tale_hops');defect=['DMS','X2.3.butanedione','X4VG','X.E..2.nonenal'];dj=[chem.index(c) for c in
 ↪  defect];rows=[];spectra=[];checks=[]
 for i in sources:
  source=gen.pre.imp.transform(X[[i]])[0];candidates=gen.sample('Tripel',80,rng);z=gen.pre.transform(candidates);target=z.mean(0
  ↪  );zs=gen.pre.transform(source[None,:])[0];source_design=np.array([m.predict(pre.transform(source[None,:]))[0,sj] for m in
  ↪  models]);basep=ec.predict_proba(ep.transform(source[None,:]))[0,tj];bases=er.predict(ep.transform(source[None,:]))[0,sj]
  for method in ['Unconstrained','PointConstraint','UncertaintyConstraint']:
   cand=np.vstack([source]+[source+alpha*(candidates-source) for alpha in [.25,.5,.75,1]])
   if method!='Unconstrained':
    for j in dj:
     if np.isfinite(X[i,j]):cand[:,j]=np.minimum(cand[:,j],X[i,j])
   change=np.array([m.predict(pre.transform(cand))[:,sj]-source_design[k] for k,m in enumerate(models)])
   risk=change.mean(0) if method=='PointConstraint' else np.quantile(change,.9,axis=0)
   feasible=np.ones(len(cand),bool) if method=='Unconstrained' else risk<=1e-10;cost=np.linalg.norm(gen.pre.transform(cand)-targ
   ↪  et,axis=1)+.15*np.linalg.norm(gen.pre.transform(cand)-zs,axis=1);cost[~feasible]=np.inf;idx=int(np.argmin(cost));out=cand
   ↪  [idx];newp=ec.predict_proba(ep.transform(out[None,:]))[0,tj];news=er.predict(ep.transform(out[None,:]))[0,sj]
   rows.append(dict(seed=rep,scope=scope,beer=int(xdf.beer.iloc[i]),method=method,target_probability_before=basep,target_probabi
   ↪  lity_after=newp,independent_stale_before=bases,independent_stale_after=news,independent_stale_change=news-bases,design_ri
   ↪  sk=float(risk[idx]),change_RMS=float(np.sqrt(np.mean((gen.pre.transform(out[None,:])[0]-zs)**2))),distance_before=float(n
   ↪  p.linalg.norm(zs-target)),distance_after=float(np.linalg.norm(gen.pre.transform(out[None,:])[0]-target)),unchanged=bool(n
   ↪  p.allclose(out,source))))
   spectra.append(dict(seed=rep,scope=scope,beer=int(xdf.beer.iloc[i]),method=method,**dict(zip(chem,out))))
   for c,j in zip(defect,dj):
    missing=not np.isfinite(X[i,j]);atfloor=not missing and
    ↪  X[i,j]<=gen.low[j]+1e-10;checks.append(dict(seed=rep,scope=scope,beer=int(xdf.beer.iloc[i]),method=method,chemical=c,sou
    ↪  rce=X[i,j],generated=out[j],encoded_nonincrease=bool(out[j]<=X[i,j]+1e-10) if not missing else
    ↪  None,physical_nonincrease_status='unverifiable_missing' if missing else 'unverifiable_floor_mass' if atfloor else
    ↪  'observed_encoded_order_only',threshold_status='consult calibrated threshold table; sensory detection is not universal
    ↪  defect limit'))
 return rows,spectra,checks

def main():
 rows=[];valrows=[];selected=[];splits=[];mr=[];ms=[];checks=[];curves=[];evalrows=[]
 for rep,s in enumerate(C['q3_seeds']):
  tr,va,ev,te=split(s,rep);rng=np.random.default_rng(s);ep=Prep().fit(X[ev]);clf=ExtraTreesClassifier(n_estimators=180,min_sampl
  ↪  es_leaf=2,n_jobs=2,random_state=s).fit(ep.transform(X[ev]),style[ev]);evalrows.append(dict(seed=s,n_evaluator=len(ev),n_te
  ↪  st=len(te),real_style_accuracy=float(np.mean(clf.predict(ep.transform(X[te]))==style[te]))))
  for role,ids in zip(['generator','validation','evaluator','test'],[tr,va,ev,te]):
   for i in ids:splits.append(dict(seed=s,row=i,beer=int(xdf.beer.iloc[i]),style=style[i],role=role))
  models={}
  for name in C['q3_models']:
   model=Generator(name,s).fit(X[tr],style[tr],X[va],style[va]);models[name]=model
   for h in model.history:curves.append(dict(seed=s,model=name,**h))
   v,vg=assess(model,X[va],style[va],X[tr],clf,ep,np.random.default_rng(s+1));valrows.append(dict(seed=s,model=name,**v));d,gene
   ↪  rated=assess(model,X[te],style[te],X[tr],clf,ep,np.random.default_rng(s+2));rows.append(dict(seed=s,model=name,**d));csv(
   ↪  pd.DataFrame(generated,columns=chem).assign(style=style[te],matched_test_beer=xdf.beer.iloc[te].to_numpy()),f'q3/test_gen
   ↪  erated_{name}_{s}.csv');log(f'Q3 {s} {name} valSWD={v["SWD"]:.4f} testSWD={d["SWD"]:.4f}')
  winner=min([r for r in valrows if r['seed']==s],key=lambda r:r['SWD'])['model'];selected.append(dict(seed=s,model=winner));sou
  ↪  rces=te[style[te]=='Pils/Lager'];rr,ss,cc=migrate(models[winner],tr,ev,sources,s,'heldout');mr+=rr;ms+=ss;checks+=cc
 csv(rows,'q3/test_metrics.csv');csv(valrows,'q3/validation_metrics.csv');csv(selected,'q3/selected.csv');csv(splits,'q3/splits.
↪  csv');csv(curves,'q3/training_curves.csv');csv(evalrows,'q3/evaluator_real_accuracy.csv');csv(mr,'q3/migration_metrics.csv'
↪  );csv(ms,'q3/migration_spectra.csv');csv(checks,'q3/defect_checks.csv')
 # Production generation is explicitly descriptive after evaluation; selection uses validation votes only.
 winner=pd.Series([r['model'] for r in selected]).value_counts().index[0];model=Generator(winner,20261028).fit(X,style);rng=np.r
↪  andom.default_rng(20261028);gen=np.vstack([model.sample(st,40,rng) for st in
↪  ST]);gs=np.repeat(ST,40);csv(pd.DataFrame(gen,columns=chem).assign(style=gs),'q3/generated_22styles_chemical231.csv');pre=P
↪  rep().fit(X);sens=basic('ET').fit(pre.transform(X),Y);csv(pd.DataFrame(np.clip(sens.predict(pre.transform(gen)),0,6),column
↪  s=targets).assign(style=gs),'q3/generated_sensory50.csv')
```

```
 js({'production_model':winner,'cvae_parameters':sum(p.numel() for p in
↪  CV().parameters()),'n_production':len(gen),'evaluator':'Disjoint 40 beers; not used to optimize or select generation;
↪  validation selection uses SWD only. Style accuracy is weak with tiny evaluator.','migration':'Only held-out Pils used for
↪  confirmatory internal comparison. Missing/floor measurements do not establish true non-worsening.'},'q3/summary.json')
if __name__=='__main__':main()
```

E.15 q3_diagnostics.py

```
# AI-assisted OpenAI Codex/GPT-6 series, 2026-10-29. Post-selection diagnostic; no retraining, no frozen artifact rewritten.
"""Quantify the two Q3 objections with evidence instead of assertions.

(1) Model-selection logic.  `q3/selected.csv` was frozen on validation-split SWD.  The closed
    22-style diagnostic ranks the four generators differently, with CVAE lowest on conditional SWD.
    This script tests whether that reversal is a test-set-size (resolution) effect, and quantifies
    how degenerate CVAE is relative to the generators that were actually shipped.

(2) Defect non-worsening.  The shipped criterion is "does the independent evaluator's stale_hops
    prediction move up at all".  The problem statement defines defects as over-threshold items, so
    we also report the threshold-crossing view on the three defect chemicals that have a usable
    sensory detection threshold: among source beers that were BELOW threshold, how many crossed
    above it after migration.

Writes outputs/rebuild/q3_complete/; nothing under q1/, q3/ or closed/ is modified.
"""
import json
from pathlib import Path

import numpy as np
import pandas as pd

R = Path(__file__).resolve().parents[2]
O = R / "outputs/rebuild"
OUT = O / "q3_complete"


def selection_logic():
    d = pd.read_csv(O / "closed/generation/metrics_22styles_by_context.csv")
    macro = pd.read_csv(O / "closed/generation/macro_summary.csv").set_index("model")
    chosen = pd.read_csv(O / "q3/selected.csv").set_index("seed").model.to_dict()
    val = pd.read_csv(O / "q3/validation_metrics.csv")
    val_mean = val.groupby("model").SWD.mean().sort_values()

    w = d.pivot_table(index=["seed", "style"], columns="model", values="SWD")
    n = d.groupby(["seed", "style"]).n_test.first()
    gap = (w["CVAE"] - w["CopulaIndependent"]).dropna()
    nt = n.reindex(gap.index)

    bins = []
    for lo, hi, label in [(1, 2, "[1,2)"), (2, 3, "[2,3)"), (3, 5, "[3,5)"), (5, 10**9, "[5,inf)")]:
        sel = (nt >= lo) & (nt < hi)
        if sel.sum():
            bins.append(
                dict(
                    n_test_range=label,
                    pairs=int(sel.sum()),
                    mean_swd_gap_CVAE_minus_CopulaIndependent=float(gap[sel].mean()),
                    cvae_closer=int((gap[sel] < 0).sum()),
                    copula_closer=int((gap[sel] > 0).sum()),
                )
            )
    corr_gap_vs_ntest = float(np.corrcoef(gap, nt)[0, 1])
    pd.DataFrame(bins).to_csv(OUT / "selection_swd_by_test_size.csv", index=False, encoding="utf-8")

    table = pd.DataFrame(
        {
            "model": macro.index,
            "SWD_mean": macro.SWD_mean.to_numpy(),
            "diversity_mean": macro.diversity_mean.to_numpy(),
            "nearest_train_RMS_mean": macro.nearest_train_RMS_mean.to_numpy(),
```

```
            "floor_mass_MAE_mean": macro.floor_mass_MAE_mean.to_numpy(),
            "correlation_MAE_mean": macro.correlation_MAE_mean.to_numpy(),
            "validation_SWD_mean_frozen_selection": val_mean.reindex(macro.index).to_numpy(),
            "chosen_in_any_seed": [int(v in chosen.values()) for v in macro.index],
            "n_seeds_chosen": [sum(1 for x in chosen.values() if x == v) for v in macro.index],
        }
    ).sort_values("SWD_mean")
    table.to_csv(OUT / "selection_criteria_table.csv", index=False, encoding="utf-8")

    sel = dict(
        selection_rule=(
            "Frozen before the closed diagnostic: for each of the three contexts the production generator is "
            "the one with minimum mean SWD on the 35-beer validation split. The test split is never used to "
            "select a generator, and the closed diagnostic was not allowed to re-select."
        ),
        chosen_by_seed=chosen,
        validation_SWD_means={k: round(float(v), 4) for k, v in val_mean.items()},
        closed_22style_swd_means={k: round(float(v), 4) for k, v in macro.SWD_mean.items()},
        ranking_reversal=True,
        gap_corr_with_n_test=round(corr_gap_vs_ntest, 4),
        gap_by_test_size=bins,
        interpretation=(
            "The reversal is a resolution effect, not a contradiction. CVAE is closer only where a style has "
            "one or two independent test beers (13/13 and 12/12 pairs); its advantage shrinks monotonically as "
            "the test set grows (-0.242, -0.168, -0.081, -0.011) and it wins only 4/6 at n_test>=5. The gap "
            "correlates with n_test at r=%.3f. CVAE is simultaneously the most degenerate generator: diversity "
            "%.3f versus %.3f for the shipped independent Copula, and nearest-train RMS %.3f versus %.3f, i.e. "
            "it sits close to a small-sample empirical distribution, which is exactly what makes a Wasserstein "
            "distance look small when the empirical side has almost no resolution. Model choice therefore "
            "follows the frozen validation rule and is defended on diversity/floor-matching, not on the "
            "conditional SWD column; quoting the closed table alone would reverse a decision that was made "
            "before it existed."
            % (
                corr_gap_vs_ntest,
                macro.loc["CVAE", "diversity_mean"],
                macro.loc["CopulaIndependent", "diversity_mean"],
                macro.loc["CVAE", "nearest_train_RMS_mean"],
                macro.loc["CopulaIndependent", "nearest_train_RMS_mean"],
            )
        ),
        what_would_overturn_this=(
            "If CVAE kept its SWD advantage on styles with >=5 independent test beers, the frozen selection "
            "would be wrong. It does not: at n_test>=5 the gap is -0.011 and Copula wins 2 of 6. Only 6 such "
            "pairs exist, so this is a direction, not a decisive test."
        ),
    )
    (OUT / "selection_logic.json").write_text(json.dumps(sel, ensure_ascii=False, indent=2), encoding="utf-8")
    return corr_gap_vs_ntest, bins


def defect_criteria():
    dc = pd.read_csv(O / "closed/migration/defect_checks_physical.csv")
    # only rows with a usable sensory detection threshold AND an observed source concentration
    dc = dc[dc.threshold_mean.notna() & dc.source_physical.notna()].copy()
    dc["source_below"] = dc.source_physical <= dc.threshold_mean
    dc["generated_above"] = dc.generated_physical > dc.threshold_mean
    dc["crossed_up"] = dc.source_below & dc.generated_above

    cross = (
        dc.groupby(["defect", "policy"])
        .agg(
            usable_rows=("crossed_up", "size"),
            source_below_threshold=("source_below", "sum"),
            crossed_above=("crossed_up", "sum"),
        )
        .reset_index()
    )
    cross["crossing_rate_given_below"] = cross.crossed_above / cross.source_below_threshold
    cross.to_csv(OUT / "defect_threshold_crossing_by_chemical.csv", index=False, encoding="utf-8")

    tot = (
```

```
    dc.groupby("policy")
    .agg(
        usable_rows=("crossed_up", "size"),
        source_below_threshold=("source_below", "sum"),
        crossed_above=("crossed_up", "sum"),
    )
    .reset_index()
)
tot["crossing_rate_given_below"] = tot.crossed_above / tot.source_below_threshold
tot.to_csv(OUT / "defect_threshold_crossing.csv", index=False, encoding="utf-8")

mig = pd.read_csv(O / "closed/migration/per_beer.csv")
# The any-epsilon reading must use the same numerical tolerance as the experiment itself
# (closed.py writes independent_stale_increase = stale_change > 1e-10). Using a bare "> 0"
# turns float32/64 round-off (up to ~5.6e-17 here) into a spurious "worsened" beer, which is
# how the rejected-migration policy once appeared to rise once.
EPS = 1e-10
mag = []
for p, g in mig.groupby("policy"):
    s = g.stale_change
    mag.append(
        dict(
            policy=p,
            n=len(g),
            stale_change_mean=float(s.mean()),
            stale_change_max=float(s.max()),
            stale_change_max_abs=float(s.abs().max()),
            n_any_positive=int((s > EPS).sum()),
            n_gt_005=int((s > 0.05).sum()),
            n_gt_010=int((s > 0.1).sum()),
            target_gain_mean=float(g.target_gain.mean()),
            refused_mean=float(g.refused.mean()),
            chemical_observed_increases=int(g.chemical_observed_increases.sum()),
        )
    )
mag = pd.DataFrame(mag)
mag.to_csv(OUT / "defect_magnitude.csv", index=False, encoding="utf-8")

crossed = tot.set_index("policy").crossed_above.to_dict()
below = tot.set_index("policy").source_below_threshold.to_dict()
anypos = {p: int((g.stale_change > EPS).sum()) for p, g in mig.groupby("policy")}
maxchg = {p: round(float(g.stale_change.max()), 6) for p, g in mig.groupby("policy")}
over50 = {p: int((g.stale_change > 0.05).sum()) for p, g in mig.groupby("policy")}
# Beer counts (n=9 per policy) are the natural denominator for the any-epsilon reading.
anypos_denominator = {p: int(len(g)) for p, g in mig.groupby("policy")}

defect = dict(
    tolerance=EPS,
    two_criteria={
        "shipped_any_epsilon": "independent evaluator stale_hops prediction moves up by more than 1e-10 (same tolerance
        ↪  closed.py uses when it writes independent_stale_increase)",
        "problem_statement_threshold": "defect = over-threshold item; count source beers below the sensory detection
        ↪  threshold that crossed above it",
    },
    threshold_crossing_by_policy={k: int(v) for k, v in crossed.items()},
    threshold_crossing_denominator={k: int(v) for k, v in below.items()},
    any_positive_by_policy=anypos,
    any_positive_denominator=anypos_denominator,
    max_stale_change_by_policy=maxchg,
    n_stale_change_gt_005_by_policy=over50,
    chemical_observed_increases_by_policy={
        p: int(g.chemical_observed_increases.sum()) for p, g in mig.groupby("policy")
    },
    note=(
        "Two readings, both reported, neither sufficient on its own. "
        "(a) Problem-statement reading (defects are over-threshold items): among source beers that started "
        "below the sensory detection threshold, the count that crossed above it is Unconstrained "
        "{xu}, PointConstraint {xp}, UncertaintyConstraint {xv} out of {dn} testable rows; the Reject "
        "policy returns the source spectrum unchanged and therefore crosses {xr}. "
        "(b) Strict any-epsilon reading (any upward move above 1e-10 counts): Unconstrained {au}/9, "
        "PointConstraint {ap}/9, UncertaintyConstraint {av}/9, Reject {ar}/9. "
```

```
            "The two readings differ by at most ~0.05 points on the 0-6 scale, which is well inside the "
            "model's own ~0.23 MAE on stale_hops, so neither can support a guarantee of non-worsening and "
            "neither can be used to dismiss the constraint as ineffective. The Reject row is an internal "
            "consistency check: its stale_change is exactly zero up to float round-off (largest magnitude "
            "{rmax:.1e}), so it must read 0 under both readings. stale_hops has no usable chemical threshold "
            "(it is a sensory proxy) and so cannot enter the threshold-crossing count at all."
        ).format(
            xu=int(crossed.get("Unconstrained", 0)), xp=int(crossed.get("PointConstraint", 0)),
            xv=int(crossed.get("UncertaintyConstraint", 0)), xr=int(crossed.get("Reject", 0)),
            dn=int(below.get("Unconstrained", 0)),
            au=anypos.get("Unconstrained", 0), ap=anypos.get("PointConstraint", 0),
            av=anypos.get("UncertaintyConstraint", 0), ar=anypos.get("Reject", 0),
            rmax=float(mig[mig.policy == "Reject"].stale_change.abs().max()),
        ),
    )
    (OUT / "defect_criteria.json").write_text(json.dumps(defect, ensure_ascii=False, indent=2), encoding="utf-8")
    return defect


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    r, bins = selection_logic()
    defect = defect_criteria()
    print("selection gap vs n_test: r=%.4f" % r)
    for b in bins:
        print("  n_test %-7s pairs=%2d mean_gap=%+.4f cvae_closer=%d" % (
            b["n_test_range"], b["pairs"],
            b["mean_swd_gap_CVAE_minus_CopulaIndependent"], b["cvae_closer"]))
    print("threshold crossing :", defect["threshold_crossing_by_policy"])
    print("  denominator      :", defect["threshold_crossing_denominator"])
    print("any-epsilon        :", defect["any_positive_by_policy"])
    print("max stale_change   :", defect["max_stale_change_by_policy"])
    print("wrote", str(OUT.relative_to(R)))


if __name__ == "__main__":
    main()
```

E.16 report.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Evidence-bound report builder."""
from run import *
import shutil

def md(df):
 d=df.copy()
 for c in d:
  if pd.api.types.is_float_dtype(d[c]):d[c]=d[c].map(lambda x:f'{x:.4f}' if np.isfinite(x) else 'NA')
 return '| '+' | '.join(map(str,d.columns))+' |\n| '+' | '.join(['---']*len(d.columns))+' |\n'+'\n'.join('| '+' |
 ↪  '.join(str(v).replace('|','/') for v in row)+' |' for row in d.to_numpy())
def read(p):return pd.read_csv(O/p)
def figure(n,caption):return f'\n![{caption}](../outputs/rebuild/figures/{n}.png){{width=95%}}\n'
def main():
 q1=read('q1/repeat_metrics.csv');q1g=q1.groupby('model')[['MAE','RMSE','R2','AF_MAE']].mean().sort_values('MAE');q1sd=q1.groupb
 ↪  y('model').MAE.std();q2=read('q2/fold_metrics.csv');q2g=q2.groupby('model')[['MAE','RMSE','R2']].mean().sort_values('MAE');
 ↪  q3=read('q3/test_metrics.csv');q3g=q3.groupby('model')[['SWD','marginal_W1','correlation_MAE','floor_fraction_MAE','style_a
 ↪  ccuracy']].mean();sta=read('q2/stability.csv').groupby('method')[['compound_jaccard','group_jaccard','within_top3_abs_rho']
 ↪  ].mean();mig=read('q3/migration_metrics.csv');mig['prob_change']=mig.target_probability_after-mig.target_probability_before
 ↪  ;migg=mig.groupby('method')[['prob_change','independent_stale_change','change_RMS']].mean();audit=json.loads((R/'outputs/re
 ↪  view_20261028/audit_summary.json').read_text());summary=json.loads((O/'q3/summary.json').read_text());top=read('q2/top3_50x
 ↪  3.csv');stats=read('q1/paired_statistics.csv');sens=read('q1/sensitivity.csv').groupby('experiment').MAE.mean();best=q1g.in
 ↪  dex[0];best2=q2g.index[0];primary=q1g.loc['InnerSelected'];env=json.loads((O/'audit/environment.json').read_text());ext=rea
 ↪  d('audit/validation_extremes.csv');params=read('q1/hyperparameters.csv');param=
 ↪  {m:int(params[params.model==m].parameter.dropna().iloc[0]) for m in ['MLP','Hierarchy','HierarchyRater']}
 # Audit and rules form part of the report, not boilerplate assumptions.
 audittext=f'''# 数据审计与监督目标重建
```

本轮以原始附件和新公开的逐评分员文件为输入，旧代码及网页端结果只用于问题定位。原始文件 SHA-256 清单在
↪ outputs/rebuild/audit/raw_manifest.csv；新文件来源为原作者 Zenodo v4 [2]，MD5 为 a8f911d8c0061e096abf3b087678e164，与本地文件一致。
↪ 全样本既往已被检查，因此本轮所有划分均属于内部重采样验证，不能称为从未接触过的外部测试。

样本与字段

1.csv 实际为 250 款啤酒、231 项化学或理化特征；4.csv 为 50 项感官目标；22 种风格中 Faro 仅 3 款，Tripel 为 28 款。题面中的“226 种啤酒”
↪ 与实物不符。231 列包含物理测量和聚合量，不能全部称为单一化合物。1、4、7 按唯一 beer 对齐；beer_id 存在精度损失，不作主键。
↪ 原始逐评分员文件中的 beer_name 是附件物理行序 1—250，不是附件 beer 编号；Beer 是品评位置，不能作啤酒键。正确映射的感官均值与附件 4
↪ 中位相关为 {audit['mapping_correlation_median_correct_row']:.4f}，错误按整数 beer 连接仅为
↪ {audit['mapping_correlation_median_wrong_beer_id']:.4f}。这是一致性证据，仍需队员核查作者匿名化映射说明。

新文件包含 2197 条记录、16 名评分员、37 个日期；12 款酒跨日期重复，187 条属于同一啤酒与评分员的重复组合。
↪ 定义目标时先对同一评分员的重复记录求均值，再在评分员之间等权平均，避免重复次数增加某人的权重。与所有记录直接平均相比，最大单元变化为
↪ {audit['equal_rater_label_change_max']:.4f} 分，整体变化较小，但对重复酒不能忽略。

评分卡层次与可识别性

50 个汇总字段由 26 个直接强度字段和 24 个派生细分字段构成。细分项在单条品评中等于父项强度或零，本轮逐条验证无违例。父项为零时，
↪ 细分零分不能区分未勾选与勾选但强度为零。将这些零值全部作为“不出现”的监督标签会引入语义错误。面板均值满足子项不大于父项，但一般不满足
↪ “父均值乘出现频率”；两者的平均绝对差为 {audit['product_of_marginal_means_error_mae']:.4f} 分。

3.xlsx 汇总公式生成的空白零分不是已填评分。新数据使真实面板均分可识别，不再需要分位数生成假标签；然而均分本身连续，
↪ 最终整数是面板均分的决策等级。评分卡有两个可交付口径：逐张 beer 表上有 45 个带标签打分格（Aroma 23、Flavour 22；Acetaldehyde
↪ 一行同服两列，Aftertaste 只属 Aroma），summary 工作表则把三类层次细分展开为 50 个机器可读字段，与 4.csv
↪ 及逐评分员记录的目标集合完全一致。本轮 50 个字段全部给出 0—6 分级评分，交付表按 summary 字段名与顺序输出。

acetaldehyde 需要单独说明，因为它是唯一一个“没有面板分数但有答案”的维度。发布数据中确实不存在它的 0—6 记录：3.xlsx 汇总表无此列，4.csv
↪ 无此列，作者公开的 6.csv 的 50 个感官列中也找不到 acetaldehyde；附件7 的 aroma_acetaldehyde 在 250 款酒中 240 款落在同一地板值，
↪ 与实测浓度 Spearman 仅 −0.058，属重编码的出现标记而非强度标签。但“无面板分”不等于“无分级”：附件1 的 acetaldehyde 经 log10 反变换
↪ （transforms.csv 的 log10 拟合误差 2.9×10^{-10}，249 对单元格；ln 为 0.89、逐列恒等为 5.27，故底数为 10）后为 0.01—18.56 mg/L、中位
↪ 4.10 mg/L；附件8 给出该化合物感官检出阈值 50.0 mg/L。250 款酒全部低于该阈值，最高值仅为阈值的 37%。按 3.xlsx 图例 0=Not present，
↪ 本数据集在该维度整体处于未呈现档，故两列统一取 0，判据存于 outputs/rebuild/q1_complete/acetaldehyde_grading.json。该 0
↪ 是阈值锚定的推断值而非面板评分，正文与结果表均单独标注。

barnyard 与 stale_hops 的汇总公式引用同一单元格；原始记录有 2183/2197 条相同。该问题是疑似标签重复，不能据此擅自改写 barnyard。主表保留 50
↪ 项并加疑点标记，另报告去除该列的训练与评价敏感性。评分员内标准化后聚合可精确重现附件 4 的
↪ {audit['z_reconstruction_exact_cells']}/12500 单元格，仍有差异；因此附件 4 不作为原始均分的简单可逆变换。

化学尺度与最低值堆积

73 个化学缺失单元由训练折中位数填补。附件 8 能核实 70 列的尺度，其中 37 列与 log10 对应，33 列保持原值或近似原值；161 列没有充分校准证据。pH、
↪ 乙醇、色度、CO2 的实际配对支持 log10，不采信网络文案中的笼统原量纲说法。反变换逐字段执行，未知列仅保留附件编码。附件 1 与 8
↪ 有个别配对冲突，物理浓度判断需保留来源。

4VG 的 67.2%、CS2 的 83.6%、limonene 的 84.8% 样本位于同一个最小值。它们提示删失、编码地板或检测流程的混合效应，不能直接证明已知检测下限。
↪ 生成模型学习观测分布中的点质量，不推断地板以下真实浓度。迁移中地板值保持相等也不能证明真实浓度未上升。

2.csv 经名称映射后可与重新计算的 Spearman 矩阵吻合，Pearson 则不吻合。全样本矩阵只用于描述性共表达图；每次预测特征选择重新在训练折内计算。
↪ 原始相关矩阵 5、6 的数据说明与实际维度差异见前轮审计，本轮不将它们作为独立训练证据。

统一实验协议

依据本地官方文件 [3]，协议文件 configs/rebuild_protocol.json 在训练前保存。第一、二问使用固定种子 20261028、20261029 的两次三折划分；
↪ 每个风格内循环分配，避免把不足折数的类别强行分层。所有同款酒的原始评分整体留在一折。内层从外层训练样本划出 20%，用于 Ridge/PLS
↪ 参数或网络训练轮数选择；再在完整外层训练集重训。训练与测试的中位数填补、缩放和选择严格隔离。

第三问每个种子分为生成训练 140、验证 35、独立评价器 40、测试 35 款。生成训练和评价器各保留各风格至少一款，因此极少样本风格的测试覆盖有限。预留
↪ Pils 供迁移，分区文件完整输出。评价器从未参与候选筛选，验证集 SWD 用于选择生成器，测试结果仅报告。重复划分并非独立实验样本；
↪ 统计推断以啤酒为重采样单位。运行中修正两处实现公平性问题：深度模型外层重训、生成距离共同随机投影。修正记录单列，未扩展模型搜索范围。

证据路径：outputs/rebuild/audit/、outputs/review_20261028/；重复品评的 Kruskal–Wallis 与 Holm 结果仅作描述，跨日期评分员构成不同，
↪ 不能直接归因于时间漂移。
'''
q1text=f'''# 问题一：真实面板均分与结构化监督

动机与目标

原始评分既提供 0—6 的实际量表，也揭示了评分员差异和细分项继承规则。直接对 50 个目标独立回归会放弃该结构，
↪ 但共同叶节点的多输出树本身也能保持线性不等式。因此，结构模型的价值必须以误差和个体评分建模证据检验，不能以“存在层次”直接推出深度网络优越。

对酒 i、评分员 r、重复次 t 的记录 y，目标为先在 t 内平均、再在 r 间平均。Ridge、PLS、RF、ET、HGB 对这一面板目标训练；结构网络对逐记录似然训练，
↪ 权重为同酒评分员数与该评分员重复次数的乘积倒数，使每款酒等权。测试以未见啤酒的面板均分为单位，既不把 2197 条当成 2197 款独立样本，
↪ 也不把评分员身份用于推断测试酒真实分数。

模型与消融

传统基线包括训练均值、Ridge（alpha=10/100）、PLS（5/12 成分）、RF 与 ET [4]（180 棵树，叶最小样本 2，特征比例 0.7）、HGB（每目标 80 轮、7
↪ 叶、L2=10）。每个候选的训练与内层验证过程均独立执行。MLP 为 231→64→32→50，ReLU 和 0.15 Dropout；训练至多 160 轮，AdamW 学习率 0.002、
↪ 权重衰减 0.001。

层次网络采用同一编码器，对 26 个直接强度预测七级概率 p(k|x)，对 24 个细分项预测条件勾选概率 q(k,x)。子项期望为 sum[k p(k|x) q(k,x)]，
↪ 不能用两个边际期望相乘。子项出现损失只在父项强度大于零的记录上计算。加入评分员的版本在父项七级 logits 上添加中心化评分员效应，
↪ 预测时使用零中心效应；这一估计对应平均 logits 的参考评分员，而不严格等同于全部评分员概率的平均，是该版本需进一步改进的估计目标差异。

MLP、Hierarchy、HierarchyRater 的可训练参数分别为 {param['MLP']}、{param['Hierarchy']}、{param['HierarchyRater']}。
↪ 训练曲线与参数配置均保存。结构网络不是原论文的复现模型；未在本轮增加图神经网络，因为现有任务的样本量与机制证据尚不足以支持额外图模块。

实际结果

下表为两次完整 OOF 预测各自计算指标后取均值；MAE/RMSE 单位均为原始评分分值，R² 为 50 目标等权平均。

{md(q1g.reset_index())}

观察到的最低 MAE 方法为 {best}，MAE={q1g.loc[best,'MAE']:.4f}，重复划分标准差={q1sd[best]:.4f}。预先规定的内层模型选择流程 InnerSelected
↪ 的 MAE={primary.MAE:.4f}、RMSE={primary.RMSE:.4f}、R²={primary.R2:.4f}。观察到某模型排名第一与预先冻结选择流程的测试表现应分别表述，
↪ 不能用全体 OOF 结果再选模型后声称其评估完全独立。

配对统计以每款酒的两次 OOF 平均预测为基础，与 InnerSelected 比较，用 2000 次啤酒级 bootstrap 计算差值区间，并对 Wilcoxon 检验作 Holm 校正；
↪ 它评价的是重复 OOF 集成，与上表单次误差均值略有不同。各目标误差见 per_target.csv，不能用总体均值掩盖弱目标。

{md(stats[['comparator','positive_favors_reference','CI_low','CI_high','p_holm']])}

评分输出与敏感性

将连续预测裁剪至 [0,6] 后按 floor(score+0.5) 输出整数。准确率衡量的是四舍五入后的真实面板均分等级，而非任一评分员的个体打分正确率。
↪ 原始连续目标、连续预测与整数等级分别输出，不把近似等级重命名为原始评分。交付表
↪ outputs/rebuild/q1/aroma_flavour_template_with_unobserved.csv 共 250 行 × 53 列、无缺失值：1 列 beer；50 个由感官监督模型预测的 0
↪ —6 字段（沿用 3.xlsx 名称与顺序，仅 A_ester_fruity、F_ester_fruity、4vg 三处因历史命名差异而映射）；以及 A_acetaldehyde、
↪ F_acetaldehyde 两列阈值锚定推断值（当前均为 0）。后两列不是原始面板评分：发布数据中不存在该字段的 0—6 记录，其取值由附件1 实测浓度经
↪ log10 反变换后与附件8 感官检出阈值 50.0 mg/L 比较得到（250 款酒全部低于阈值，故按 3.xlsx 图例取 0=Not present），判据见
↪ outputs/rebuild/q1_complete/acetaldehyde_grading.json。因此评分卡中 50 个字段来自面板监督、2 个字段来自浓度对阈值的判定，
↪ 两类证据强度不同，在交付表中仍以独立列呈现而不合并叙述。

50 个字段的 OOF 误差并不均衡，且难度与字段是否带 A_/F_ 前缀无关。30 个带前缀字段的平均 MAE 为 0.5047，20 个不带前缀字段（body、bitternes、
↪ co2、aftertaste、alcohol、sweetness、acidity、dms、metallic、X4vg、t2n、diacetyl、stale_hops、orange、coriander、clove、lactic、
↪ acetic、barnyard、overall）的平均 MAE 为 0.3486。R² 最高的是 lactic 0.7204、acidity 0.7183、alcohol 0.6653、acetic 0.6629、
↪ bitternes 0.5997、body 0.5010；而 dms、metallic、t2n、stale_hops、barnyard 的 R² 接近 0，但 MAE 也小，原因是这些缺陷维度本身变异极小。
↪ 覆盖 50 个字段并未以精度为代价。

交付评分卡还暴露一个必须披露的局限：12,500 个面板单元与 500 个乙醛单元中，预测等级全部落在 0—4 级，没有任何 5 级或 6 级结果，
↪ 而真实面板均分取整后确实存在 112 个 5 级与 9 个 6 级单元。这不是取整造成的假象：连续预测的最大值只有 4.34，连四舍五入到 5 都不可能，
↪ 说明模型本身把高分向中部收缩。按真值档位分组可见系统性偏差：真值 [0,2) 的 8,253 个单元被高估 +0.164，[2,3) 低估 −0.173，[3,4) 低估
↪ −0.461，[4,5) 低估 −1.071，真值 ≥5 的 121 个单元全部被低估、平均低估 −2.074（真值均值 5.07 被压到 3.00）。也就是说，50 列预测在整体
↪ MAE 意义上可用，但尾部不可用：任何依赖识别高强度样品（例如强酒花香或强缺陷）的下游用法都会漏检。本轮不为此重训模型，
↪ 因为三组补充实验已经封闭且冻结；该偏差作为明确局限记录，定量结果见 outputs/rebuild/q1_complete/grade_range_diagnostic.csv 与
↪ grade_range_summary.json。若要改进，方向是高强度样本的加权或分位数校准，属于模型层面工作，超出本轮范围。

{md(sens.rename('MAE').reset_index())}

额外遮蔽 5%、10%、20% 输入只施加在测试特征，使用训练中位数处理，模拟随机缺失；它不代表真实非随机检测失败。重复评分权重和去除 barnyard
↪ 的敏感性结果用于判断数据语义选择的影响。面板均分的区间由评分员重采样获得，是有限评分员测量不确定性，不是模型预测区间。输出的全样本 ET
↪ 模型用于后续展示性画像，不能用其对训练数据的拟合分数代替 OOF 成绩。

证据：outputs/rebuild/q1/fold_metrics.csv、repeat_metrics.csv、training_curves.csv、sensitivity.csv、panel_uncertainty.csv。
↪ 旧网页版的 AF30 MAE=0.5062 对应不同标签权重与五折划分，不与本轮数字直接作超越声明。
'''
q2text=f'''# 问题二：互补化合物选择与稳定性

建图与候选定义

节点为附件 1 的 231 个测量变量，边取附件 2 对应 Spearman |rho|≥0.65，保存符号与绝对边权。图为全样本描述，不进入外层预测。候选因素剔除 13 项理化读数、聚合量和非单一分子池，保留 218 列；乙醇等可识别分子仍允许。pH、颜色、总酯之类可作预测特征，但不作为最终“关键化合物”顶替分子名称。

四种预设方法

Spearman 取绝对相关前三；Hybrid2plus1 先取相关前二，再在前二的正则残差空间选第三项；Forward3 每一步均以正则化残差增益选择；Redundancy 用绝对目标相关减去 0.5 倍与已选变量的最大绝对相关。所有方法恰好选择三项，并使用相同 alpha=8 的 Ridge 建立三变量预测模型。残差选择在标准化秩空间进行，正则为 0.02；最终三变量预测使用训练折标准化的原始附件编码。

风格调整版本先在训练折内部减去各风格均值，再计算秩关联与前向选择，最终预测仍以化合物为输入。它是检验风格混杂的敏感性，不等于消除了所有混杂，更不是因果效应估计。风格也可能处于生成过程的中间环节，调整后预测减弱本身不能证明原方法错误。

结果与统计证据

{md(q2g.reset_index())}

最低平均折 MAE 出现在 {best2}。完整前向选择并未获得足够证据支持其优于 2+1；因此不把计算步骤更多写成创新优势。最终 top3_50x3.csv 为预先指定 Forward3 的 150 行可复核结果；top3_all_methods.csv 同时提供所有四种方法，方便队员按预测、冗余与稳定性共同审定。没有在测试结果上重新制定加权排名规则。

{md(read('q2/paired_statistics.csv')[['comparator','positive_favors_reference','CI_low','CI_high','p_holm']])}

替代组稳定性

100 次 bootstrap 在各风格内抽样，重新填补、标准化和选择。每个因素提供入选频率；同一目标的相邻独立重采样对计算 Jaccard。替代组通过 |rho| 的完全链接聚类构建，组内两两绝对相关至少 0.8，避免单链接长链把弱相关变量并为一组。组稳定性用于表达同源共变的可替代测量，不证明某一成员的独立作用。

{md(sta.reset_index())}

若化合物入选频率低而替代组频率高，说明可稳定识别一组信息，但难以唯一指定分子。若两者均低，应明确 Top-3 不确定。去冗余方法降低共线性不一定提高预测；全样本描述组也不作为验证期特征选择器。

化学解释边界

各入选项按附件 8 关联中文/英文名、来源、单位和阈值，见 top3_chemical_annotations.csv。酯与高级醇的共同变化可能反映发酵过程，酸类相关可能携带酸啤酒风格信息。可检验的解释至少同时查看原始相关、风格调整关联、bootstrap 频率和同组替代物。没有元数据或明确实验支持的条目保留“待化学复核”，不以名称联想编造气味机制。

原论文以统计学习提出候选后做真实添加品评实验 [1]；本题现有数据仅支持关联与预测贡献。尤其 barnyard 标签有公式疑点，不能把它与 stale_hops 的相关包装成新发现。50×3 只是任务所需的候选列表，不是 150 个已经证实的因果作用。
'''
 q3text=f'''# 问题三：混合分布生成与受约束迁移

从观测语义到生成模型

最低值堆积使连续高斯模型难以重现变量的点质量；231 维对单风格 3—31 款又不足以独立拟合高维分布。本轮将“风格边际分布”和“跨变量依赖”分开处理。各风格经验分布以 n/(n+10) 权重与全体训练经验分布收缩混合，因此稀少风格借用总体信息，同时保留所观测的离散地板值。所有经验分布仅来自生成训练集。

CopulaIndependent 对各边际独立采样，是去掉依赖结构的消融。CopulaJoint 对训练数据中秩转换后的正态分数估计 Ledoit-Wolf 收缩协方差 [5]，归一为相关矩阵；联合正态样本经正态 CDF 与条件经验逆 CDF 输出化合物谱。并列最小值天然对应逆 CDF 的区间，不需要生成后逐列随机回填。它建模观测点质量的联合分布，不是对未知 LOD 以下浓度作 Tobit 参数估计；中秩近似及高斯依赖假设也会限制复杂尾部关系。

传统基线 PCA_Gaussian 将训练标准化特征降至 12 维，风格均值收缩后使用共享协方差采样，再逆变换并限制在训练范围。CVAE 基于变分自编码原理 [6]，输入为 231 特征与 22 类条件，64 维隐层、12 维潜变量，解码器为 (12+22)→64→231；参数量 {summary['cvae_parameters']}，KL 系数 0.05，最多 160 轮，验证重构误差确定检查点。其目的在于检查共享非线性表征能否克服小风格样本不足，不预设深度模型更好。

独立评价

三个种子均严格分离生成训练、验证、评价器训练和测试。评价器是 40 款真实酒训练的 ET 分类器/感官回归器；由于 22 类样本非常少，分类准确率上限证据弱，故同时报告真实测试酒分类成绩。评价器输出不用于优化生成样本或选择生成模型。所有模型使用相同固定 48 个随机投影方向计算 SWD，缩放参数来自生成训练集。真实测试值不被裁剪到训练范围。

{md(q3g.reset_index())}

生产生成器依据三次验证 SWD 均值确定为 {summary['production_model']}。验证单次选择可能不同，完整记录在 selected.csv；不能按测试最佳方法回写选择结果。下表记录独立分类器对真实测试样本的能力，帮助判断生成风格准确率的可解释范围。

{md(read('q3/evaluator_real_accuracy.csv'))}

联合 Copula 与独立 Copula 比较对应依赖结构消融；PCA_Gaussian 比较连续低维模型；CVAE 比较非线性生成。不同方法在 SWD、边际误差、相关结构、
↪ 地板质量和条件一致性上存在取舍，不强行宣布一个模型统治全部指标。最近邻距离与完整样本复制比例在最终输出上重新计算；
↪ 无完整复制不意味着无隐私风险或无训练记忆，经验边际仍取自训练观测值。

验证集中出现极端训练标准化值，导致个别 SWD 很大。这是固定度量对尾部的敏感性，保留在结果中，未根据验证或测试异常反复改变裁剪规则。
↪ 极端值清单如下。

{md(ext)}

跨风格迁移与风险约束

从独立留出的 Pils/Lager 源谱出发，每源使用 80 个 Tripel 候选及 0.25、0.5、0.75、1.0 四个插值步长，加上保持不变候选。
↪ 目标函数为到目标候选中心的标准化距离加 0.15 倍改变量。三个版本共用候选预算：不约束、点预测约束、90% bootstrap 预测差分位约束。

DMS、Diacetyl、4VG、trans-2-nonenal 对有观测的源编码施加逐项不增加。stale_hops 不是单一化学列，由仅在生成训练集上训练的 12 个 bootstrap ET
↪ 设计预测器评估；点约束要求平均差≤0，风险约束要求 90% 分位差≤0。该分位是模型集成的不确定性启发式，不是经覆盖率校准的置信保证。
↪ 独立感官评价器只在候选确定后打分。

{md(migg.reset_index())}

啤酒级 bootstrap 先对同款酒跨划分变化求均值，再重采样；目前独立留出迁移覆盖 {mig.beer.nunique()} 款不同 Pils。
↪ 三个版本的完整逐酒结果同时保存。风险约束即使满足设计预测器，仍可在独立 stale_hops 预测中出现上升；本轮没有证据证明其普遍降低真实缺陷风险，
↪ 应将这一失败作为模型不确定性未校准与评价器样本小的共同限制。

阈值与量纲

DMS 参考均值阈值 17.25 μg/L；Diacetyl 0.29925 mg/L；4VG 0.206333 mg/L。参考均值与中位阈值不同，应明确使用口径。这些是感官检出参考，
↪ 不是所有风格通用的不可接受界限。trans-2-nonenal 未取得本地附件明确阈值，stale_hops 没有唯一浓度阈值，二者不能计算虚构通过率。4VG
↪ 在部分风格中可有正面风味作用，但本题迁移约束仍按不恶化执行。

已校准 log10 列按 10 的幂转换，其余已核实原值列保留；无校准的 161 列不输出伪造物理浓度。缺失源浓度无法验证“不增加”，
↪ 位于观测地板的源也不能证明真实浓度顺序。约束表把编码不增加、尺度变换、阈值参考和物理不可验证原因分开列出。

最终输出 22 风格各 40 个样本共 880 谱、对应 50 维预测感官画像、已校准量纲表和独立留出迁移实例。生成谱只满足当前统计范围与部分编码约束；
↪ 缺少物料平衡、工艺动力学、真实化学混合及盲评验证，不能称为可直接酿造配方。
'''

explanation='# 逐感官关键化合物解释与复核线索\n\n以下逐条列出真实选择结果。数值是全样本描述性关联与100次风格内重采样频率，不是独立因果效应。
↪ 化学来源取附件8。\n\n'
ann=read('q2/top3_chemical_annotations.csv')
for target,group in ann[ann.method=='Forward3'].groupby('target',sort=False):
 explanation+='## '+target+'\n\n'
 for _,v in group.iterrows():
  origin=str(v.get('origin_paper','NA')); explanation+=f"第{int(v['rank'])}项 {v.chemical}: Spearman={v.spearman:.3f},
  ↪ 风格内调整秩关联={v.style_adjusted_rank_association:.3f}，入选频率={v.frequency:.0%}，替代组频率={v.group_frequency:.0%}；
  ↪ 附件来源类别为 {origin}。"
  explanation+=('入选率不足一半，应视为可替换的候选而非稳定唯一因素。' if v.frequency<.5 else '在当前重采样中较常入选，仍需独立品评验证。
  ↪ ')+'\n\n'
 if target in ['alcohol','body','diacetyl','dms','X4vg','t2n','overall']:
  q2text+='\n'+target+' 的实际候选为 '+ '、'.join(group.chemical)+ '；对应入选频率为 '+ '、'.join(f'{v:.0%}' for v in
  ↪ group.frequency)+'。低频成员不能与高频主因素作同等强度解释。\n'
(R/'reports/02_q2_compound_explanations.md').write_text(explanation,encoding='utf-8')

innovation=f'''# 创新动机、消融及证据判断

第一项候选改进是按评分卡观测机制建立条件层次概率模型。困难来自“子项评分=父强度×勾选状态”、重复评分员权重和零父强度下勾选不可观测。
↪ 它通过条件于强度的子项概率与按酒、评分员均衡权重处理语义；MLP、Hierarchy、HierarchyRater 形成逐步消融。结构模型的误差应与强树模型共同比较，
↪ 本轮证据不支持以深度模型替代树模型作为主预测器。对平均 logits 的评分员定义还存在目标差异，后续应先修正估计对象，再谈复杂架构。

第二项候选改进是把最低值点质量、联合依赖和独立风险审计连为一个生成—迁移流程。独立边际消融揭示依赖结构的作用；连续高斯和 CVAE
↪ 揭示点质量与小样本正则化的重要性；三种约束版本揭示“设计安全”与“独立评价安全”的差别。高斯 Copula、收缩估计和 bootstrap 都是成熟技术，
↪ 本轮贡献应定位为针对观测语义的组合与验证，不声称创造了这些方法或已完成文献意义上的新算法证明。

{md(q3g[['SWD','correlation_MAE','floor_fraction_MAE']].reset_index())}

Q2 的去冗余与前向选择属于另一项必要对照，而不再包装为第三个核心创新。2+1、完整前向与冗余惩罚的排名表明：去除共线信息、
↪ 提高预测和增加排序稳定性并非同一目标。替代组报告可以使不确定性可见，但不能把相近变量任意互换为已证实的因果因素。

不支持的结论：结构网络全面胜出、CVAE 优于传统概率生成、完整前向选择必胜、风险约束已保证缺陷真实不恶化、相对原论文性能更优。
↪ 后续最有价值的工作是补充真实盲评、查明地板生成机制、增加独立风格样本及复核 barnyard 标签，而不是无边界叠加网络模块。
'''
final=f'''# 本轮结果汇总

本轮实际完成：数据与原始评分审计；9 类第一问模型及内层选择；第二问四方法、风格调整、100 次 bootstrap 与有符号网络；第三问四生成模型、
↪ 三种迁移约束与独立评价器；重复划分、敏感性、图表与中文参考论文。全部主要结果来自 outputs/rebuild，旧输出不混入新指标。

第一问观察最优 {best}：MAE={q1g.loc[best,'MAE']:.4f}，RMSE={q1g.loc[best,'RMSE']:.4f}，R²={q1g.loc[best,'R2']:.4f}，AF30
↪ MAE={q1g.loc[best,'AF_MAE']:.4f}。内层选择流程 MAE={primary.MAE:.4f}。第二问观察最优 {best2}，平均折
↪ MAE={q2g.loc[best2,'MAE']:.4f}；所有方法保留。第三问按平均验证 SWD 选择 {summary['production_model']}；生成质量与条件一致性存在取舍，
↪ 逐方法测试表见报告 03。

关键未解决事项：匿名评分行序缺少独立键级证明；barnyard 标签疑点；两项 acetaldehyde 缺少监督；161 项未校准；地板以下浓度未知；独立风格评价器仅
↪ 40 款；风险分位没有覆盖率校准；没有真实新酿造/添加/盲评验证。深度模型与完整前向选择没有获得预期优势，已如实保留。

论文为 AI 辅助研究参考。官方规定要求核心建模、算法设计、求解与分析由队员独立完成，审阅签字不能替代独立工作。本轮没有伪造人工修改记录，
↪ 队员须独立重做和审定关键科学判断，并补充真实记录后决定正式提交内容。

一键复现：powershell -ExecutionPolicy Bypass -File scripts/run_rebuild.ps1。脚本不写死机器路径，python、pandoc、xelatex 依次按 -Python
↪ 参数、PATH、常见安装位置解析，也可用 MC_PANDOC、MC_XELATEX 指定。论文构建需要 Pandoc 与 XeLaTeX。
'''
reports={'00_data_audit.md':audittext,'01_q1_analysis.md':q1text,'02_q2_analysis.md':q2text,'03_q3_analysis.md':q3text,'04_inno」
↪ vation_and_ablation.md':innovation,'05_final_summary.md':final}
legacy=R/'reports/legacy_reference';legacy.mkdir(exist_ok=True)
for name,text in reports.items():
 p=R/'reports'/name
 if p.exists() and not (legacy/name).exists():shutil.copy2(p,legacy/name)
 p.write_text(text,encoding='utf-8')
abstract=f'''针对啤酒风味评分、关键化合物筛选与条件生成中的量表歧义、变量共线及小风格样本问题，本文构建以观测语义和独立评价为主线的求解流程。
↪ 首先核验 250 款啤酒的 231 项化学特征，并将 2197 条逐评分员记录映射至啤酒，构造评分员等权的真实 0—6 面板均分。评分预测比较线性、
↪ 树与条件层次网络，在两次三折内部验证中，观察最优 {best} 的 MAE 为 {q1g.loc[best,'MAE']:.4f}，RMSE 为 {q1g.loc[best,'RMSE']:.4f}；
↪ 结构网络未获得足以替代强树基线的优势。其次，将共表达图、三变量条件选择和风格内 bootstrap 联合用于 50×3 化合物分析，
↪ 报告单分子与替代组稳定性；{best2} 获得最低平均折误差，但完整前向选择的优势未获支持。最后，以收缩条件边际与 Copula 对最低值点质量建模，
↪ 对比连续高斯和 CVAE，并用独立 40 款酒训练的评价器检验迁移。结果表明，联合分布误差、风格一致性和缺陷风险存在取舍；
↪ 满足设计预测器的约束不等于独立评价下缺陷不恶化。本文交付 22 风格共 880 个统计生成谱、感官画像及受约束迁移实例，同时标明量纲、
↪ 地板浓度与缺失阈值的不可验证边界。该成果为 AI 辅助参考，关键科学判断仍需参赛队独立完成与验证。'''
intro=r'''# 问题分析与研究路线

啤酒的感官画像既取决于化学组成，也取决于评分卡的观察方式。第一问预测的量表决定第二问的重要性含义，
↪ 第二问的共变关系又影响第三问是否能生成合理的联合谱；因此三个任务不能简单分别套用回归、排序和生成网络。本文将统一主线确定为：先识别可观测量，
↪ 再建立预测与分布模型，最后在未参与优化的评价环节检验结果。

原论文 [1] 比较十类预测算法并通过真实添加试验检验候选成分；本赛题新增 0—6 分级与跨风格约束生成，评估对象也不同。本文使用作者公开数据 [2]，
↪ 不把原文的消费者喜好预测指标与本轮 50 维原始面板均分误差直接比较。网页参考版引入真实评分和概率生成具有参考价值，但原先的记录均值、
↪ 同训练集评价器与后处理前记忆距离需要重新核查。MC2026A 中部分预处理与代理约束问题也不被继承。

建模假设包括：匿名原始评分按附件物理行序映射；评分员等权面板均分作为可重复监督对象；化学缺失按训练中位数作工作性处理；
↪ 有限样本的风格经验分布可通过总体收缩改善方差。模型没有假定所有最小值均为同一个已知检出限，也没有假定任意统计生成谱都可由酿造过程实现。
↪ 上述假设分别通过映射相关、重复权重、随机缺失和生成分布诊断检验。

符号与数学定义

设 x_i 为第 i 款酒在附件编码尺度上的 231 维谱，s_i 为风格，y_irt 为评分员 r 在重复 t 的评分。原始监督目标定义为

$$\bar y_i=\frac{1}{R_i}\sum_{r=1}^{R_i}\left(\frac{1}{T_{ir}}\sum_t y_{irt}\right).$$

连续预测的误差按啤酒与目标等权计算。离散输出为 $g_i=\min(6,\max(0,\lfloor\hat y_i+0.5\rfloor))$。模型均分与个体评分是不同随机量，
↪ 论文中的整数准确率始终针对舍入后的面板均分。

层次模型对强度 I 与勾选 B 分别建模，但组合时使用条件期望：

$$E[IB\mid x]=\sum_{k=0}^{6} k\,P(I=k\mid x)P(B=1\mid I=k,x).$$

这个等式直接对应评分卡运算，并处理父项为零时无法识别勾选状态的问题。化合物替代组表示相近的统计信息，不表示分子可在真实配方中自由替代。

条件经验边际写作 $\widehat F_{js}=w_s\widehat F_{j,s}+(1-w_s)\widehat F_{j,all}$，其中 $w_s=n_s/(n_s+10)$。联合采样为 $z\sim
↪ N(0,\widehat R)$，$x_j=\widehat F_{js}^{-1}(\Phi(z_j))$。独立消融令 $\widehat R=I$。其中逆分布的平坦区间保留观测最小值的点质量。

迁移优化在有限候选集中最小化到目标中心的距离与改变量之和；缺陷约束应用于明确观测的化学编码以及设计预测器的 stale_hops 变化。
↪ 独立评价器只评价已选候选，避免优化器利用同一个评价函数制造表面成功。

```
'''
 refs='''# 参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature
↪  Communications, 15:2368, 2024. DOI: 10.1038/s41467-024-46346-0.

[2] Schreurs M, et al. Data from: Predicting and improving complex beer flavor through machine learning, version v4. Zenodo,
↪  https://zenodo.org/records/23624915, 访问日期 2026-10-28.

[3] MathorCup 组委会. 2026 年 MathorCup 大数据挑战赛赛道 A 初赛题、论文格式及提交规范、人工智能工具使用规定（试行）. 本地官方附件, 2026.

[4] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3–42, 2006. DOI: 10.1007/s10994-006-6226-1.

[5] Ledoit O, Wolf M. A well-conditioned estimator for large-dimensional covariance matrices. Journal of Multivariate Analysis,
↪  88(2):365–411, 2004. DOI: 10.1016/S0047-259X(03)00096-4.

[6] Kingma D P, Welling M. Auto-Encoding Variational Bayes. arXiv:1312.6114, 2013, https://arxiv.org/abs/1312.6114, 访问日期
↪  2026-10-28.

[7] OpenAI Codex, GPT-6 系列（接口未提供可独立核验的完整模型快照标识）, OpenAI, 2026-10-28。用于本参考工程的审计、代码、实验、图表与文本辅助。

# AI 使用声明与队员独立工作记录

本参考稿的审计、模型实现、实验运行、图表和文字由 AI 辅助生成，正文各实验与图表均具有该贡献。详细说明见 reports/AI工具使用详情说明.pdf 和
↪  AI_usage_log.md。未记录任何已经完成的队员独立建模、人工修改或盲评，因此不能宣称核心工作由队员独立完成。正式参赛作品应遵守官方规定，
↪  由队员独立完成核心设计、求解和分析，并如实记录采纳、修改与弃用内容；本稿不冒充符合该独立性要求的正式提交稿。
'''
 paper='# 观测语义约束下的啤酒风味预测、化合物筛选与条件生成\n\n## 摘要\n\n'+abstract+'\n\n关键词：逐评分员监督；层次评分；稳定选择；
↪  混合边际；独立评价\n\n'+intro+'\n'+audittext+figure('01_data','风格样本量及观测最低值堆积；AI
↪  辅助计算绘图')+'\n'+q1text+figure('02_q1_accuracy','真实评分尺度上的模型误差与困难目标；AI
↪  辅助')+figure('03_q1_curves','内层训练与验证曲线，六个外层任务平均；AI
↪  辅助')+'\n'+q2text+figure('04_q2_selection','三因素预测与替代组稳定性；AI 辅助')+figure('05_q2_network','有符号共表达图，
↪  未绘孤立节点；AI 辅助')+'\n'+q3text+figure('06_q3_quality','三次划分下生成误差，误差棒为划分标准差；AI
↪  辅助')+figure('07_migration','独立评价器下的迁移变化；AI 辅助')+figure('08_generated_portraits','全样本重拟合后的展示性感官画像，
↪  非独立验证；AI 辅助')+'\n'+innovation+figure('09_sensitivity','额外缺失敏感性与 CVAE 重构曲线；AI 辅助')+'\n#
↪  模型评价与结论\n\n'+final.split('\n',1)[1]+'\n'+refs
 p=R/'paper/competition_draft.md';legacy_p=R/'paper/legacy_reference.md'
 if p.exists() and not legacy_p.exists():shutil.copy2(p,legacy_p)
 p.write_text(paper,encoding='utf-8');(R/'paper/rebuild_body.md').write_text(paper.split(intro)[0] if False else
↪  paper[paper.index('# 问题分析'):],encoding='utf-8');(R/'paper/rebuild_abstract.txt').write_text(abstract,encoding='utf-8')
 append='# 附录：逐感官 Top-3 与代码索引\n\n以下为预先指定 Forward3 的候选。其他方法及入选概率见 CSV；所有条目为关联候选。
↪  \n\n';t=top.pivot(index='target',columns='rank',values='chemical').reset_index();t.columns=['感官','候选1','候选2','候选3'];a⌋
↪  ppend+=md(t);(R/'paper/rebuild_appendix.md').write_text(append,encoding='utf-8')
 # Append truthful usage entry once; human cells stay pending.
 ai=R/'reports/AI_usage_log.md';entry='''\n\n## 2026-10-28 独立重构实际记录\n\n工具：OpenAI Codex / GPT-6 系列，完整快照未提供；Python、
↪  PyTorch、scikit-learn、Pandas、Matplotlib、Pandoc、XeLaTeX。用户指令：独立审计并实际完成三问，冻结协议，不预设旧结果正确。AI
↪  实际完成数据核验、原始评分目标、模型代码、运行与实现修正、统计、图表及中文参考稿。关键修正：深度模型外层重训、生成度量共享随机方向。
↪  训练及结果日志位于 outputs/rebuild/logs；所有新源码头部标明 AI 辅助。\n\n人工修改及核心独立工作：尚无可核实记录，不填写为已完成。
↪  采用或弃用决策需队员独立作出。官方规定要求核心建模、算法、求解与分析由队员独立完成，简单审阅或签字不能替代该要求。本产物是研究参考，
↪  不能声称其 AI 主导核心已满足该规定。\n'''
 if '## 2026-10-28 独立重构实际记录' not in
↪  ai.read_text(encoding='utf-8'):ai.write_text(ai.read_text(encoding='utf-8')+entry,encoding='utf-8')
 csv([dict(decision=d,status='待队员独立完成与复核',reviewer='',date='',evidence='') for d in ['匿名评分映射','面板监督目标','结构模⌋
↪  型采用或弃用','Top-3最终规则','缺陷阈值解释','生成及迁移可行性','正式论文独立重写']],'audit/human_decision_record.csv')
 js({'_scope':'LEGACY single-split / single-fold record kept only for provenance. Authoritative closed-experiment numbers live in
↪  outputs/rebuild/closed/final_summary.json (22-style x multi-context macro). Do not cross-cite the
↪  two.','_authoritative_file':'outputs/rebuild/closed/final_summary.json','q1_observed_best':best,'q1_metrics':q1g.loc[best].⌋
↪  to_dict(),'q1_MAE_repeat_sd':q1sd[best],'q1_inner_selected':primary.to_dict(),'q2_observed_best':best2,'q2_metrics':q2g.loc⌋
↪  [best2].to_dict(),'q3_production':summary['production_model'],'q3_metrics':q3g.to_dict('index'),'migration_unique_beers':in⌋
↪  t(mig.beer.nunique()),'migration_summary':migg.to_dict('index')},'summary.json');log('reports and full paper text built')
if __name__=='__main__':main()
```

E.17 run.py

```
"""AI-assisted research reference: OpenAI Codex/GPT-6, 2026-10-28.
Frozen protocol; no raw files modified. Human scientific decisions remain unreviewed.
"""
from pathlib import Path
```

```
import os,json,hashlib,time,warnings,sys,traceback
os.environ.setdefault('OMP_NUM_THREADS','2');os.environ.setdefault('MKL_NUM_THREADS','2')
import numpy as np,pandas as pd
from scipy.stats import rankdata,norm,wasserstein_distance,wilcoxon
from scipy.spatial.distance import cdist
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge,LogisticRegression
from sklearn.cross_decomposition import PLSRegression
from sklearn.ensemble import ExtraTreesRegressor,RandomForestRegressor,HistGradientBoostingRegressor,ExtraTreesClassifier
from sklearn.multioutput import MultiOutputRegressor
from sklearn.decomposition import PCA
from sklearn.covariance import LedoitWolf
from sklearn.metrics import r2_score
import torch
from torch import nn
from threadpoolctl import threadpool_limits
warnings.filterwarnings('ignore',category=RuntimeWarning)
torch.set_num_threads(2);threadpool_limits(2)
R=Path(__file__).resolve().parents[2];O=R/'outputs/rebuild';C=json.loads((R/'configs/rebuild_protocol.json').read_text(encoding=
↪  'utf-8-sig'))
DEVICE='cuda' if torch.cuda.is_available() else 'cpu'
# Input resolution lives in paths.py so lightweight scripts can locate files without importing this module
# (importing run.py executes load() below, which needs the official attachments to exist).
from paths import find_input as raw_path
def csv(d,p):pd.DataFrame(d).to_csv(O/p,index=False,encoding='utf-8-sig')
def js(d,p):(O/p).write_text(json.dumps(d,ensure_ascii=False,indent=2,default=lambda x:x.item() if hasattr(x,'item') else
↪  str(x)),encoding='utf-8')
def log(s):print(time.strftime('%H:%M:%S'),s,flush=True)
def seed(s):np.random.seed(s);torch.manual_seed(s)
def load():
 x=pd.read_csv(raw_path('1.csv'));off=pd.read_csv(raw_path('4.csv'));raw=pd.read_csv(raw_path('Raw sensory data
 ↪  anonymized.csv'),encoding='cp1252').rename(columns={'A_ester_fruity':'A_esters_fruity','F_ester_fruity':'F_esters_fruity'})
 assert x.beer.is_unique and off.beer.is_unique and np.array_equal(x.beer,off.beer)
 cols=list(off.columns[3:]);chem=list(x.columns[3:]);Y=raw.groupby(['beer_name','taster'])[cols].mean().groupby('beer_name').mea
 ↪  n().reindex(range(1,251)).to_numpy();Yr=raw.groupby('beer_name')[cols].mean().reindex(range(1,251)).to_numpy()
 assert Y.shape==(250,50) and np.isfinite(Y).all() and Y.min()>=0 and Y.max()<=6
 return x,off,raw,cols,chem,x[chem].to_numpy(float),Y,Yr,x.tasting_category_fine.to_numpy()
xdf,off,raw,targets,chem,X,Y,Yrecord,style=load();AF=np.array([j for j,c in enumerate(targets) if
↪  c.startswith(('A_','F_'))]);SAFE=np.array([j for j,c in enumerate(targets) if c!='barnyard'])
parents={p:[c for c in targets if c.startswith(p[:-3]) and c!=p] for p in targets if p.endswith('_all')};children=[c for cs in
↪  parents.values() for c in cs];base=[c for c in targets if c not in children]
BI=[targets.index(c) for c in base];CI=[targets.index(c) for c in children];PI=[base.index(p) for p,cs in parents.items() for c
↪  in cs]
def folds(s,n=3):
 rng=np.random.default_rng(s);f=np.empty(250,int)
 for st in np.unique(style):
  ix=np.flatnonzero(style==st);rng.shuffle(ix);f[ix]=(np.arange(len(ix))+rng.integers(n))%n
 return f
class Prep:
 def fit(self,a):self.imp=SimpleImputer(strategy='median').fit(a);v=self.imp.transform(a);self.scaler=StandardScaler().fit(v);re
 ↪  turn self
 def transform(self,a):return self.scaler.transform(self.imp.transform(a))
def metric(a,b):return dict(MAE=float(abs(a-b).mean()),RMSE=float(np.sqrt(np.mean((a-b)**2))),R2=float(r2_score(a,b,multioutput=
↪  'uniform_average')),AF_MAE=float(abs(a[:,AF]-b[:,AF]).mean()),MAE49=float(abs(a[:,SAFE]-b[:,SAFE]).mean()))
def inner(tr,s):
 rng=np.random.default_rng(s);ix=tr.copy();rng.shuffle(ix);n=max(20,int(len(ix)*.2));return ix[n:],ix[:n]
def audit():
 manifest=[{'file':str(p.relative_to(R)),'sha256':hashlib.sha256(p.read_bytes()).hexdigest(),'bytes':p.stat().st_size} for p in
 ↪  sorted((R/'data/raw').glob('*')) if p.is_file()];csv(manifest,'audit/raw_manifest.csv')
 csv(pd.DataFrame(Y,columns=targets).assign(beer=xdf.beer,raw_beer_name=np.arange(1,251)),'audit/equal_rater_targets.csv');csv(p
 ↪  d.DataFrame(Yrecord,columns=targets),'audit/record_mean_targets.csv')
 rows=[]
 for j,c in enumerate(chem):v=X[:,j];lo=np.nanmin(v);rows.append(dict(feature=c,missing=int(np.isnan(v).sum()),minimum=lo,minimu
 ↪  m_fraction=float(np.mean(v==lo)),maximum=np.nanmax(v),unique=int(len(np.unique(v[np.isfinite(v)])))))
 csv(rows,'audit/features.csv');csv(pd.Series(style).value_counts().rename_axis('style').reset_index(name='n'),'audit/styles.csv')
 for s in
 ↪  C['seeds']:csv(pd.DataFrame({'row':np.arange(250),'beer':xdf.beer,'fold':folds(s),'style':style}),f'audit/folds_{s}.csv')
 # Differences from record weighting and panelist additive variability are descriptive only.
 means=raw.groupby('taster')[targets].mean();csv(means.reset_index(),'audit/taster_descriptive_means.csv')
```

```
 rr=raw.copy();rr['n_repeat']=rr.groupby(['beer_name','taster']).taster.transform('size');csv(rr.groupby('beer_name').agg(record
 ↪  s=('taster','size'),raters=('taster','nunique'),dates=('date','nunique')).reset_index(),'audit/repeats.csv')
 js({'device':DEVICE,'torch':torch.__version__,'numpy':np.__version__,'pandas':pd.__version__,'python':sys.version,'protocol_sha
 ↪  256':hashlib.sha256((R/'configs/rebuild_protocol.json').read_bytes()).hexdigest(),'parent_child_violations':sum(int(((raw[c
 ↪  ]!=0)&(raw[c]!=raw[p])).sum()) for p,cs in parents.items() for c in
 ↪  cs),'equal_rater_vs_record_MAE':float(abs(Y-Yrecord).mean()),'equal_rater_vs_record_max':float(abs(Y-Yrecord).max()),'claim
 ↪  ':'Internal resampling only; no fresh external cohort'},'audit/environment.json')
 log('audit complete')
class RatingNet(nn.Module):
 def __init__(self,kind):
  super().__init__();self.kind=kind;self.enc=nn.Sequential(nn.Linear(231,64),nn.ReLU(),nn.Dropout(.15),nn.Linear(64,32),nn.ReLU())
  if kind=='MLP':self.head=nn.Linear(32,50)
  else:
   self.base=nn.Linear(32,len(base)*7);self.child=nn.Linear(32,len(children)*7)
   if kind=='HierarchyRater':self.rater=nn.Embedding(raw.taster.nunique(),len(base)*7)
 def forward(self,a,r=None):
  h=self.enc(a)
  if self.kind=='MLP':return self.head(h),None,None
  b=self.base(h).reshape(-1,len(base),7)
  if hasattr(self,'rater'):
   emb=self.rater.weight-self.rater.weight.mean(0,keepdim=True)
   if r is not None:b=b+emb[r].reshape(-1,len(base),7)
  p=b.softmax(-1);q=self.child(h).reshape(-1,len(children),7).sigmoid();k=torch.arange(7,device=a.device)
  out=torch.zeros((len(a),50),device=a.device);out[:,BI]=(p*k).sum(-1);out[:,CI]=(p[:,PI,:]*q*k).sum(-1)
  return out,b,q
class Deep:
 def __init__(self,kind,s):self.kind=kind;self.s=s
 def fit(self,a,y,beer_indices,val_a=None,val_y=None,tag='',fixed_epochs=None):
  seed(self.s);self.net=RatingNet(self.kind).to(DEVICE);opt=torch.optim.AdamW(self.net.parameters(),lr=.002,weight_decay=.001)
  A=torch.tensor(a,dtype=torch.float32,device=DEVICE);T=torch.tensor(y,dtype=torch.float32,device=DEVICE)
  if self.kind!='MLP':
   sub=raw[raw.beer_name.isin(beer_indices+1)].copy();loc={v+1:i for i,v in enumerate(beer_indices)};row=torch.tensor(sub.beer_n
   ↪  ame.map(loc).to_numpy(),device=DEVICE);obs=torch.tensor(sub[targets].to_numpy(),dtype=torch.float32,device=DEVICE);rat=to
   ↪  rch.tensor(pd.Categorical(sub.taster,categories=sorted(raw.taster.unique())).codes.astype('int64'),device=DEVICE)
   counts=sub.groupby(['beer_name','taster']).taster.transform('size').to_numpy();nr=sub.groupby('beer_name').taster.transform('
   ↪  nunique').to_numpy();w=torch.tensor(1/(counts*nr),dtype=torch.float32,device=DEVICE);w=w/w.sum()
  history=[];best=np.inf;beststate=None
  for e in range(fixed_epochs or C['deep_epochs']):
   self.net.train();opt.zero_grad()
   if self.kind=='MLP':pred,_,_=self.net(A);loss=((pred-T)**2).mean()
   else:
    _,b,q=self.net(A[row],rat if self.kind=='HierarchyRater' else None);bt=obs[:,BI].long();ce=nn.functional.cross_entropy(b.res
    ↪  hape(-1,7),bt.reshape(-1),reduction='none').reshape(len(obs),-1).mean(1)
    par=bt[:,PI];chosen=q.gather(2,par.unsqueeze(-1)).squeeze(-1);mask=(par>0).float();binary=(obs[:,CI]>0).float();bc=nn.functi
    ↪  onal.binary_cross_entropy(chosen,binary,reduction='none');loss=((ce+(bc*mask).sum(1)/mask.sum(1).clamp(min=1))*w).sum()
   loss.backward();nn.utils.clip_grad_norm_(self.net.parameters(),5);opt.step();self.net.eval()
   with torch.no_grad():
    train_mae=float(abs(self.net(A)[0]-T).mean());v=float(abs(self.predict(val_a)-val_y).mean()) if val_a is not None else
    ↪  train_mae
   history.append(dict(tag=tag,model=self.kind,epoch=e+1,loss=float(loss.detach()),train_mae=train_mae,validation_mae=v))
   if fixed_epochs is not None or v<best:self.best_epoch=e+1;best=v;beststate={k:t.detach().cpu().clone() for k,t in
   ↪  self.net.state_dict().items()}
  self.net.load_state_dict(beststate);self.history=history;self.params=sum(p.numel() for p in self.net.parameters());return self
 def predict(self,a):
  self.net.eval()
  with torch.no_grad():return np.clip(self.net(torch.tensor(a,dtype=torch.float32,device=DEVICE))[0].cpu().numpy(),0,6)
def basic(name,param=None,s=20261028):
 if name=='Ridge':return Ridge(alpha=param or 100)
 if name=='PLS':return PLSRegression(n_components=param or 5,scale=False)
 if name=='ET':return ExtraTreesRegressor(n_estimators=180,min_samples_leaf=2,max_features=.7,n_jobs=2,random_state=s)
 if name=='RF':return RandomForestRegressor(n_estimators=180,min_samples_leaf=2,max_features=.7,n_jobs=2,random_state=s)
 if name=='HGB':return MultiOutputRegressor(HistGradientBoostingRegressor(max_iter=80,max_leaf_nodes=7,l2_regularization=10,rand
 ↪  om_state=s),n_jobs=1)
def q1():
 results=[];curves=[];preds={m:[] for m in C['q1_models']+['InnerSelected']};selected=[];params=[]
 for s in C['seeds']:
  f=folds(s);out={m:np.zeros_like(Y) for m in preds}
  for fold in range(3):
   te=np.flatnonzero(f==fold);tr=np.flatnonzero(f!=fold);it,iv=inner(tr,s+fold);pi=Prep().fit(X[it]);pt=Prep().fit(X[tr]);a,b=pi
   ↪  .transform(X[it]),pi.transform(X[iv]);aa,bb=pt.transform(X[tr]),pt.transform(X[te]);vscore={}
   for name in C['q1_models']:
```

```
    t=time.time();tag=f'{s}_{fold}'
    if name=='Mean':p=np.tile(Y[tr].mean(0),(len(te),1));v=abs(Y[it].mean(0)-Y[iv]).mean();par=None
    elif name in ['MLP','Hierarchy','HierarchyRater']:
     # Inner stopping chooses epoch; outer refit uses exactly that epoch count.
     model=Deep(name,s+fold).fit(a,Y[it],it,b,Y[iv],tag);v=abs(model.predict(b)-Y[iv]).mean();curves+=model.history;epochs=model
    ↪  .best_epoch;model=Deep(name,s+fold).fit(aa,Y[tr],tr,tag=tag+'_refit',fixed_epochs=epochs);p=model.predict(bb);par=model
    ↪  .params
     torch.save(model.net.state_dict(),O/f'models/{name}_{tag}.pt')
    else:
     candidates=C['q1_ridge_alphas'] if name=='Ridge' else C['q1_pls_components'] if name=='PLS' else [None];vv=[]
     for par in candidates:
      m=basic(name,par,s+fold).fit(a,Y[it]);vv.append(abs(np.clip(m.predict(b),0,6)-Y[iv]).mean())
     idx=int(np.argmin(vv));par=candidates[idx];v=vv[idx];model=basic(name,par,s+fold).fit(aa,Y[tr]);p=np.clip(model.predict(bb)
    ↪  ,0,6)
    out[name][te]=p;vscore[name]=float(v);results.append(dict(seed=s,fold=fold,model=name,n_train=len(tr),n_test=len(te),seconds
   ↪  =time.time()-t,**metric(Y[te],p)));params.append(dict(seed=s,fold=fold,model=name,parameter=par,validation_mae=v));log(f
   ↪  'Q1 {tag} {name} MAE={results[-1]["MAE"]:.4f}')
   winner=min(vscore,key=vscore.get);out['InnerSelected'][te]=out[winner][te];selected.append(dict(seed=s,fold=fold,model=winner
  ↪  ,validation_mae=vscore[winner]));results.append(dict(seed=s,fold=fold,model='InnerSelected',n_train=len(tr),n_test=len(te
  ↪  ),seconds=0,**metric(Y[te],out[winner][te])))
  for name in out:preds[name].append(out[name]);csv(pd.DataFrame(out[name],columns=targets).assign(beer=xdf.beer),f'q1/oof_{name
 ↪  }_{s}.csv')
 csv(results,'q1/fold_metrics.csv');csv(curves,'q1/training_curves.csv');csv(params,'q1/hyperparameters.csv');csv(selected,'q1/i
↪  nner_selected.csv')
 per=[];summ=[]
 for name,pp in preds.items():
  for repeat,p in enumerate(pp):summ.append(dict(model=name,seed=C['seeds'][repeat],**metric(Y,p),record_mean_MAE=float(abs(p-Yr
 ↪  ecord).mean()),rounded_panel_mean_accuracy=float((np.floor(p[:,AF]+.5)==np.floor(Y[:,AF]+.5)).mean())))
  p=np.mean(pp,0)
  for j,c in enumerate(targets):per.append(dict(model=name,target=c,MAE=float(abs(p[:,j]-Y[:,j]).mean()),RMSE=float(np.sqrt(np.m
 ↪  ean((p[:,j]-Y[:,j])**2))),R2=float(r2_score(Y[:,j],p[:,j]))))
 csv(summ,'q1/repeat_metrics.csv');csv(per,'q1/per_target.csv');statistics(preds,Y,'q1','InnerSelected')
 p=np.mean(preds['InnerSelected'],0);csv(pd.DataFrame(np.clip(np.floor(p+.5),0,6).astype(int),columns=targets).assign(beer=xdf.b
↪  eer),'q1/grades_oof_panel_mean.csv');csv(pd.DataFrame(p,columns=targets).assign(beer=xdf.beer),'q1/continuous_oof.csv')
 # Bootstrap raw individual panel uncertainty (no model fitting involved).
 rng=np.random.default_rng(20261028);panel=[]
 for i in range(250):
  rr=raw[raw.beer_name==i+1].groupby('taster')[targets].mean().to_numpy();boot=rr[rng.integers(len(rr),size=(500,len(rr)))].mean
 ↪  (1);lo,hi=np.quantile(boot,[.025,.975],axis=0)
  for j,c in enumerate(targets):panel.append(dict(beer=int(xdf.beer.iloc[i]),target=c,mean=Y[i,j],lower=lo[j],upper=hi[j],raters
 ↪  =len(rr)))
 csv(panel,'q1/panel_uncertainty.csv')
def statistics(preds,truth,folder,reference):
 rng=np.random.default_rng(428);err={k:abs(np.mean(v,0)-truth).mean(1) for k,v in preds.items()};rows=[]
 for m,e in err.items():
  if m==reference:continue
  d=e-err[reference];boot=d[rng.integers(len(d),size=(2000,len(d)))].mean(1);p=wilcoxon(d).pvalue if np.any(d) else 1
  rows.append(dict(comparator=m,reference=reference,positive_favors_reference=float(d.mean()),CI_low=float(np.quantile(boot,.025
 ↪  )),CI_high=float(np.quantile(boot,.975)),p=float(p)))
 order=np.argsort([r['p'] for r in rows]);last=0
 for k,i in enumerate(order):last=max(last,min(1,rows[i]['p']*(len(rows)-k)));rows[i]['p_holm']=last
 csv(rows,f'{folder}/paired_statistics.csv')
if __name__=='__main__':
 audit()
 if '--audit-only' not in sys.argv:q1()
```

E.18 supplement.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Diagnostics, not test-driven retuning."""
from run import *
from q3 import Generator,ST
from scipy.stats import wilcoxon
import joblib

def main():
 rows=[];rng=np.random.default_rng(901);official=off[targets].to_numpy();predstd=np.zeros_like(Y)
 for s in C['seeds']:
  f=folds(s)
```

```
  for k in range(3):
   tr=np.flatnonzero(f!=k);te=np.flatnonzero(f==k);pre=Prep().fit(X[tr]);a=pre.transform(X[tr]);m=basic('ET',s=s).fit(a,Y[tr]);p
   ↪  =m.predict(pre.transform(X[te]));rows.append(dict(seed=s,fold=k,experiment='ET_equal_rater',**metric(Y[te],p)))
   for frac in [.05,.10,.20]:
    z=X[te].copy();z[rng.random(z.shape)<frac]=np.nan;p=m.predict(pre.transform(z));rows.append(dict(seed=s,fold=k,experiment=f'
    ↪  ET_mask_{frac}',**metric(Y[te],p)))
   rec=basic('ET',s=s).fit(a,Yrecord[tr]);rows.append(dict(seed=s,fold=k,experiment='ET_record_training_equal_eval',**metric(Y[t
   ↪  e],rec.predict(pre.transform(X[te])))))
   keep=[j for j,c in enumerate(targets) if c!='barnyard'];m49=basic('ET',s=s).fit(a,Y[tr][:,keep]);p49=m49.predict(pre.transfor
   ↪  m(X[te]));rows.append(dict(seed=s,fold=k,experiment='ET_without_barnyard',MAE=float(abs(p49-Y[te][:,keep]).mean()),RMSE=f
   ↪  loat(np.sqrt(np.mean((p49-Y[te][:,keep])**2))),R2=float(r2_score(Y[te][:,keep],p49)),AF_MAE=float(abs(p49[:,:30]-Y[te,:30
   ↪  ]).mean()),MAE49=float(abs(p49-Y[te][:,keep]).mean())))
   mz=basic('ET',s=s).fit(a,official[tr]);pz=mz.predict(pre.transform(X[te]));rows.append(dict(seed=s,fold=k,experiment='ET_offi
   ↪  cial_Z_scale',**metric(official[te],pz)))
 csv(rows,'q1/sensitivity.csv')
 pre=Prep().fit(X);model=basic('ET').fit(pre.transform(X),Y);joblib.dump({'preprocessor':pre,'model':model,'targets':targets,'fe
↪  atures':chem,'scope':'full refit descriptive, scores are not independent evaluation'},O/'models/ET_full.joblib')
 # Annotate all factors using supplied metadata, preserving unsupported chemistry status.
 top=pd.read_csv(O/'q2/top3_all_methods.csv');meta=pd.read_csv(R/'data/processed/thresholds.csv').rename(columns={'field_name_R'
↪  :'chemical'});ann=top.merge(meta,on='chemical',how='left',validate='many_to_one');csv(ann,'q2/top3_chemical_annotations.csv
↪  ')
 # Fixed tie-resolution using validation mean, not test results.
 val=pd.read_csv(O/'q3/validation_metrics.csv');winner=val.groupby('model').SWD.mean().idxmin();genmodel=Generator(winner,202610
↪  28).fit(X,style);gen=np.vstack([genmodel.sample(st,40,rng) for st in
↪  ST]);csv(pd.DataFrame(gen,columns=chem).assign(style=np.repeat(ST,40)),'q3/generated_22styles_chemical231.csv');csv(pd.Data
↪  Frame(np.clip(model.predict(pre.transform(gen)),0,6),columns=targets).assign(style=np.repeat(ST,40)),'q3/generated_sensory5
↪  0.csv')
 summ=json.loads((O/'q3/summary.json').read_text());summ['production_model']=winner;summ['production_selection']='Minimum mean
↪  validation SWD; resolves three-way validation vote tie without test metrics.';js(summ,'q3/summary.json')
 # Physical conversion only for individually verified calibrated features.
 trans=pd.read_csv(R/'data/processed/transforms.csv').set_index('feature');th=pd.read_csv(R/'data/processed/thresholds.csv').set
↪  _index('field_name_R');dc=pd.read_csv(O/'q3/defect_checks.csv');out=[]
 for _,r in dc.iterrows():
  c=r.chemical;t=trans.loc[c];threshold=th.loc[c,'threshold_mean'] if c in th.index else np.nan;v=10**r.generated if
  ↪  t['transform']=='log10' else r.generated if str(t['transform']).startswith('identity') else np.nan;b=10**r.source if
  ↪  t['transform']=='log10' else r.source if str(t['transform']).startswith('identity') else
  ↪  np.nan;d=r.to_dict();d.update(transform=t['transform'],unit=t['unit'],source_converted=b,generated_converted=v,reference_t
  ↪  hreshold=threshold,generated_above_reference=bool(v>threshold) if np.isfinite(threshold) else
  ↪  None,threshold_interpretation='sensory detection reference, not legal/safety/acceptability limit');out.append(d)
 csv(out,'q3/defect_checks_physical.csv')
 mg=pd.read_csv(O/'q3/migration_metrics.csv');stat=[]
 for m,df in mg.groupby('method'):
  # Beer is the resampling unit; collapse repeated partitions before CI.
  d=df.assign(prob_change=df.target_probability_after-df.target_probability_before).groupby('beer')[['prob_change','independent_
 ↪  stale_change']].mean()
  for measure in d:
   v=d[measure].to_numpy();b=v[rng.integers(len(v),size=(2000,len(v)))].mean(1);stat.append(dict(method=m,measure=measure,unique
   ↪  _beers=len(v),mean=v.mean(),CI_low=np.quantile(b,.025),CI_high=np.quantile(b,.975),p=wilcoxon(v).pvalue if np.any(v) else
   ↪  1))
 order=np.argsort([r['p'] for r in stat]);last=0
 for k,i in enumerate(order):last=max(last,min(1,stat[i]['p']*(len(stat)-k)));stat[i]['p_holm']=last
 csv(stat,'q3/migration_statistics.csv')
 fs=[];styles=[]
 splits=pd.read_csv(O/'q3/splits.csv')
 for s in C['q3_seeds']:
  tr=splits[(splits.seed==s)&(splits.role=='generator')].row.to_numpy();te=splits[(splits.seed==s)&(splits.role=='test')].row.to
  ↪  _numpy();pr=Prep().fit(X[tr]);low=np.nanmin(X[tr],0);eligible=np.mean(X[tr]==low,0)>=.25;real=(pr.imp.transform(X[te])[:,e
  ↪  ligible]==low[eligible]).astype(float);co=real.T@real/len(real)
  for name in C['q3_models']:
   g=pd.read_csv(O/f'q3/test_generated_{name}_{s}.csv');binary=(g[chem].to_numpy()[:,eligible]==low[eligible]).astype(float);cg=
   ↪  binary.T@binary/len(binary);mask=np.triu(np.ones(co.shape,bool),1);fs.append(dict(seed=s,model=name,joint_floor_probabili
   ↪  ty_MAE=float(abs(cg[mask]-co[mask]).mean()),n_floor_variables=int(sum(eligible))))
   for st in ST:
    ix=style[te]==st
    if sum(ix):styles.append(dict(seed=s,model=name,style=st,n_test=int(sum(ix)),marginal_W1=float(np.mean([wasserstein_distance
    ↪  (pr.transform(X[te][ix])[:,j],pr.transform(g.loc[ix,chem].to_numpy())[:,j]) for j in range(231)]))))
 csv(fs,'q3/floor_joint_diagnostics.csv');csv(styles,'q3/per_style_metrics.csv')
 log('supplement diagnostics complete')
if __name__=='__main__':main()
```

E.19 verify.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28. Output consistency assertions."""
from run import *
def main():
 checks=[]
 def check(name,ok,detail=''):checks.append(dict(check=name,passed=bool(ok),detail=detail))
 manifest=pd.read_csv(O/'audit/raw_manifest.csv')
 for _,r in manifest.iterrows():check('raw unchanged '+r.file,hashlib.sha256((R/r.file).read_bytes()).hexdigest()==r.sha256)
 for s in C['seeds']:
  for m in C['q1_models']+['InnerSelected']:
   p=pd.read_csv(O/f'q1/oof_{m}_{s}.csv')[targets].to_numpy();check(f'OOF {m} {s}',p.shape==(250,50) and np.isfinite(p).all() and
   ↪  p.min()>=0 and p.max()<=6)
 q1=pd.read_csv(O/'q1/fold_metrics.csv');check('Q1 60 model-fold rows',len(q1)==60);check('Q1 equal outer train
↪  size',q1.groupby(['seed','fold']).n_train.nunique().max()==1)
 top=pd.read_csv(O/'q2/top3_50x3.csv');check('Q2 50x3',len(top)==150 and top.target.nunique()==50 and
↪  top.groupby('target').chemical.nunique().eq(3).all());check('Q2 100
↪  bootstrap',np.load(O/'q2/bootstrap_selections.npz')['Forward3'].shape==(100,50,3))
 sp=pd.read_csv(O/'q3/splits.csv')
 for s,d in sp.groupby('seed'):check(f'Q3 disjoint {s}',len(d)==250 and d.row.nunique()==250 and
↪  sorted(d.groupby('role').size())==[35,35,40,140])
 g=pd.read_csv(O/'q3/generated_22styles_chemical231.csv');check('Q3 880 spectra finite',len(g)==880 and
↪  np.isfinite(g[chem].to_numpy()).all() and g['style'].nunique()==22)
 p=pd.read_csv(O/'q3/generated_sensory50.csv');check('Q3 portraits bounded',len(p)==880 and p[targets].min().min()>=0 and
↪  p[targets].max().max()<=6)
 d=pd.read_csv(O/'q3/defect_checks.csv');v=d[(d.method!='Unconstrained')&d.source.notna()];check('observed encoded
↪  constraints',np.all(v.generated<=v.source+1e-10),'Not a physical guarantee for floor values')
 m=pd.read_csv(O/'q3/migration_metrics.csv');check('risk design
↪  constraint',m.loc[m.method=='UncertaintyConstraint','design_risk'].le(1e-10).all())
 # Non-guaranteed independent risk is a measured limitation, not a failed software assertion.
 diagnostics={'independent_stale_increases_by_method':m.assign(increase=m.independent_stale_change>1e-10).groupby('method').incr
↪  ease.sum().to_dict(),'q3_source_missing_rows':int(d.source.isna().sum()),'q3_floor_unverifiable_rows':int((d.physical_nonin
↪  crease_status=='unverifiable_floor_mass').sum())}
 for f in ['01_data','02_q1_accuracy','03_q1_curves','04_q2_selection','05_q2_network','06_q3_quality','07_migration','08_genera
↪  ted_portraits','09_sensitivity']:
  for ext in ['png','pdf','svg']:check(f'Figure {f}.{ext}',(O/f'figures/{f}.{ext}').stat().st_size>1000)
 for f in ['00_data_audit.md','01_q1_analysis.md','02_q2_analysis.md','03_q3_analysis.md','04_innovation_and_ablation.md','05_fi
↪  nal_summary.md']:check('Report '+f,(R/'reports'/f).stat().st_size>500)
 check('Paper PDF exists',(R/'paper/competition_draft.pdf').exists());check('AI details PDF
↪  exists',(R/'reports/AI工具使用详情说明.pdf').exists());csv(checks,'verification.csv');js(diagnostics,'verification_diagnostic
↪  s.json');js({'checks':len(checks),'passed':sum(c['passed'] for c in checks),'all_passed':all(c['passed'] for c in
↪  checks)},'verification.json')
 src=[{'file':str(p.relative_to(R)),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in sorted((R/'src/rebuild').glob(
↪  '*.py'))];csv(src,'audit/source_manifest.csv');print(json.dumps({'checks':len(checks),'passed':sum(c['passed'] for c in
↪  checks)},ensure_ascii=False));assert all(c['passed'] for c in checks),[c for c in checks if not c['passed']]
if __name__=='__main__':main()
```

E.20 run_final.ps1

```
# AI-assisted: OpenAI Codex gpt-6.1-sol, 2026-10-28. Portable: no machine-specific interpreter paths.
param([string]$Python='',[string]$Pandoc='',[string]$XeLaTeX='',[switch]$ReportOnly)
$ErrorActionPreference='Stop'
$Root=Split-Path $PSScriptRoot -Parent
Set-Location $Root
$env:PYTHONIOENCODING='utf-8'

function Resolve-Tool([string]$Explicit,[string]$Command,[string[]]$Fallbacks){
 if($Explicit){ if(-not (Test-Path $Explicit)){throw "Given path not found: $Explicit"}; return $Explicit }
 $c=Get-Command $Command -ErrorAction SilentlyContinue
 if($c){return $c.Source}
 foreach($f in $Fallbacks){ if($f -and (Test-Path $f)){return $f} }
 throw "Cannot locate '$Command'. Install it or pass it explicitly (see README_REBUILD.md)."
}

if(-not $Python){
 $Python=Resolve-Tool '' 'python'
↪  @("$env:CONDA_PREFIX\python.exe","$env:USERPROFILE\anaconda3\python.exe","C:\Anaconda\python.exe")
}
```

```
# Toolchain is passed down to build_rebuild_pdf.py so the builder never hardcodes a machine path.
if(-not $Pandoc){ $Pandoc=Resolve-Tool '' 'pandoc' @("C:\Anaconda\Library\bin\pandoc.exe","$env:ProgramFiles\Pandoc\pandoc.exe") }
if(-not $XeLaTeX){ $XeLaTeX=Resolve-Tool '' 'xelatex'
↪  @('C:\texlive\2026\bin\windows\xelatex.exe','C:\texlive\2025\bin\windows\xelatex.exe') }
$env:MC_PANDOC=$Pandoc
$env:MC_XELATEX=$XeLaTeX

Write-Host "python  : $Python"
Write-Host "pandoc  : $Pandoc"
Write-Host "xelatex : $XeLaTeX"

function Invoke-Stage([string]$Name,[string[]]$StageArgs){
 Write-Host $Name
 $taskPreviousPreference=$ErrorActionPreference
 try {
  $ErrorActionPreference='Continue'
  & $Python @StageArgs *> "outputs/rebuild/logs/$Name.log"
  $taskExitCode=$LASTEXITCODE
 } finally {$ErrorActionPreference=$taskPreviousPreference}
 if($taskExitCode -ne 0){throw "Stage $Name failed; inspect its log"}
}
if(-not $ReportOnly){
 Invoke-Stage 'closed_q2' @('-u','src/rebuild/closed.py','--q2')
 Invoke-Stage 'closed_generation' @('-u','src/rebuild/closed.py','--generation')
 Invoke-Stage 'closed_migration' @('-u','src/rebuild/closed.py','--migration')
}
Invoke-Stage 'closed_tables' @('src/rebuild/closed_tables.py')
Invoke-Stage 'closed_figures' @('src/rebuild/closed_figures.py')
Invoke-Stage 'closed_report' @('src/rebuild/closed_report.py')
Invoke-Stage 'pdf_build' @('scripts/build_rebuild_pdf.py')
Invoke-Stage 'final_quality' @('scripts/final_quality.py')
Invoke-Stage 'closed_verify' @('src/rebuild/closed_verify.py')
Invoke-Stage 'package_final' @('scripts/package_final.py')
```

E.21 run_q1.ps1

```
# AI-assisted: OpenAI Codex gpt-6.1-sol, 2026-10-29. Standalone Q1 rebuild entry point.
# Rebuilds Question 1 end to end from source, in the required order:
#   1. audit the raw inputs against the recorded SHA-256 manifest
#   2. train Q1 models and write the out-of-fold panel-mean grades
#   3. recompute the threshold-anchored acetaldehyde score
#   4. assemble the 53-column scorecard deliverable
param([string]$Python='',[switch]$SkipAudit)
$ErrorActionPreference='Stop'
$Root=Split-Path $PSScriptRoot -Parent
Set-Location $Root
$env:PYTHONIOENCODING='utf-8'

function Resolve-Tool([string]$Explicit,[string]$Command,[string[]]$Fallbacks){
 if($Explicit){ if(-not (Test-Path $Explicit)){throw "Given path not found: $Explicit"}; return $Explicit }
 $c=Get-Command $Command -ErrorAction SilentlyContinue
 if($c){return $c.Source}
 foreach($f in $Fallbacks){ if($f -and (Test-Path $f)){return $f} }
 throw "Cannot locate '$Command'. Install it or pass it explicitly (see README_REBUILD.md)."
}
if(-not $Python){
 $Python=Resolve-Tool '' 'python'
↪  @("$env:CONDA_PREFIX\python.exe","$env:USERPROFILE\anaconda3\python.exe","C:\Anaconda\python.exe")
}
Write-Host "python : $Python"

# Fail fast with an actionable message if an input is missing.
$needed=@('1.csv','4.csv','Raw sensory data anonymized.csv')
foreach($n in $needed){
 $found=$false
 foreach($d in @('data/raw','data/supplementary')){ if(Test-Path (Join-Path $d $n)){ $found=$true } }
 if(-not $found){ throw "Missing input '$n'. Official attachments go in data/raw; the author's public file goes in
↪  data/supplementary. See README_REBUILD.md." }
}
```

```
foreach($n in @('2.csv','3.xlsx','7.csv','8.xlsx')){
 if(-not (Test-Path (Join-Path 'data/raw' $n)) -and -not (Test-Path (Join-Path 'data/supplementary' $n))){
  Write-Warning "Optional-but-recommended input '$n' not found; audit_extended.py and the 附件 7 supplement will be skipped."
 }
}

function Invoke-Stage([string]$Name,[string[]]$StageArgs){
 Write-Host $Name
 $taskPreviousPreference=$ErrorActionPreference
 try {
  $ErrorActionPreference='Continue'
  & $Python @StageArgs *> "outputs/rebuild/logs/$Name.log"
  $taskExitCode=$LASTEXITCODE
 } finally {$ErrorActionPreference=$taskPreviousPreference}
 if($taskExitCode -ne 0){throw "Stage $Name failed; inspect outputs/rebuild/logs/$Name.log"}
}

# 1. Q1 training run. run.py always re-runs audit() first, which REWRITES audit/raw_manifest.csv;
#    that is the recorded input baseline, so it is only regenerated when -SkipAudit is NOT set and
#    the caller accepts the baseline being refreshed from the staged inputs.
if(-not $SkipAudit){ Invoke-Stage 'q1_audit' @('-u','src/rebuild/run.py','--audit-only') }
Invoke-Stage 'q1_train' @('-u','src/rebuild/run.py')
# 2. Threshold-anchored acetaldehyde score (needs 附件 1 1.csv and 附件 8 thresholds).
Invoke-Stage 'q1_acetaldehyde' @('-u','src/rebuild/q1_acetaldehyde.py')
# 3. Assemble the 53-column deliverable.
Invoke-Stage 'q1_complete' @('-u','src/rebuild/q1_complete.py')
# 4. Extended audits and the consistency gate.
Invoke-Stage 'q1_audit_extended' @('-u','src/rebuild/audit_extended.py')
Invoke-Stage 'q1_verify' @('-u','src/rebuild/closed_verify.py')

Write-Host ''
Write-Host 'Q1 rebuilt. Deliverable: outputs/rebuild/q1/aroma_flavour_template_with_unobserved.csv (250 x 53).'
Write-Host 'Note: closed_verify.py compares the rebuilt files against configs/closed_final_protocol.json;'
Write-Host 'if it reports frozen-hash mismatches, that is expected after a retrain and the protocol record'
Write-Host 'must be refreshed deliberately, not silently.'
```

E.22 run_rebuild.ps1

```
# Compatibility entry: Q1 is frozen; only the final three closed experiments run.
param([string]$Python='D:\condaData\envs_dirs\taidibei\python.exe',[switch]$SkipExperiments)
& (Join-Path $PSScriptRoot 'run_final.ps1') -Python $Python -ReportOnly:$SkipExperiments
```

E.23 build_rebuild_pdf.py

```
"""AI-assisted OpenAI Codex/GPT-6, 2026-10-28; Chinese reference PDF build. Toolchain resolved per machine, never hardcoded."""
from pathlib import Path
import os,shutil,subprocess,sys,json,re
R=Path(__file__).resolve().parents[1];P=R/'paper';B=P/'build_rebuild';B.mkdir(exist_ok=True)
def tool(env,cmd,fallbacks):
 if os.environ.get(env):return os.environ[env]
 found=shutil.which(cmd)
 if found:return found
 for f in fallbacks:
  if f and os.path.exists(f):return f
 raise SystemExit(f'Cannot find {cmd}; set the {env} environment variable to its full path')
pandoc=tool('MC_PANDOC','pandoc',['C:/Anaconda/Library/bin/pandoc.exe',r'C:\Program Files\Pandoc\pandoc.exe'])
xelatex=tool('MC_XELATEX','xelatex',['C:/texlive/2026/bin/windows/xelatex.exe','C:/texlive/2025/bin/windows/xelatex.exe'])
def run(cmd,cwd):
 p=subprocess.run(cmd,cwd=cwd,stdout=subprocess.PIPE,stderr=subprocess.STDOUT,encoding='utf-8',errors='replace');(B/('pandoc.log'
 ↪  if 'pandoc' in cmd[0] else 'xelatex_console.log')).write_text(p.stdout,encoding='utf-8')
 if p.returncode:raise RuntimeError(p.stdout[-4000:])
for source,out in [('rebuild_body.md','body.tex'),('rebuild_appendix.md','appendix.tex')]:run([pandoc,str(P/source),'-f','markdo
↪  wn','-t','latex','--top-level-division=section','-o',str(B/out)],P)

# Wrap long tables and compound names so every factor remains visible.
for fname in ['body.tex','appendix.tex']:
```

```
 pp=B/fname;tt=pp.read_text(encoding='utf-8');tt=tt.replace('.png}', '.pdf}')
 def format_table(match):
  block=match.group(0)
  def spec(m):
   n=len(m.group(1));return
  ↪  r'{@{}*{'+str(n)+r'}{>{\raggedright\arraybackslash}p{\dimexpr\linewidth/'+str(n)+r'-2\tabcolsep\relax}}@{}}'
  block=re.sub(r'\{@\{\}([lrc]+)@\{\}\}',spec,block)
  block=block.replace(r'\_',r'\_\allowbreak{}')
  block=re.sub(r'(?<=[A-Za-z])\.(?=[A-Za-z])',r'.\\allowbreak{}',block)
  block=block.replace('CopulaIndependent',r'\seqsplit{CopulaIndependent}').replace('X2.methyltetrahydrofuran.3.one',r'\seqsplit{
 ↪  X2.methyltetrahydrofuran.3.one}')
  return r'\begingroup\footnotesize'+'\n'+block+'\n'+r'\endgroup'
 tt=re.sub(r'\\begin\{longtable\}[\s\S]*?\\end\{longtable\}',format_table,tt)
 tt=tt.replace(r'top3\_chemical\_annotations.csv',r'\path{top3_chemical_annotations.csv}')
 # Long unbreakable defect-name run in running text (DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops)
 # produced an Overfull \hbox; allow breaks after the slashes so the line can wrap.
 tt=tt.replace('DMS/Diacetyl/4VG/trans-2-nonenal/Stale hops',
               r'DMS/\allowbreak{}Diacetyl/\allowbreak{}4VG/\allowbreak{}trans-2-nonenal/\allowbreak{}Stale hops')
 tt=tt.replace('DMS/Diacetyl/4VG/trans-2-nonenal/Stale~hops',
               r'DMS/\allowbreak{}Diacetyl/\allowbreak{}4VG/\allowbreak{}trans-2-nonenal/\allowbreak{}Stale~hops')
 # Long output-path runs in running text.
 for _p in ['defect_threshold_crossing.csv','acetaldehyde_grading.json','selection_criteria_table.csv']:
  tt=tt.replace(_p,_p.replace('.',r'\allowbreak{}.',1))
 pp.write_text(tt,encoding='utf-8')

header=r'''\documentclass[UTF8,a4paper,zihao=-4]{ctexart}
\usepackage[margin=2.3cm]{geometry}
\setmainfont{Times New Roman}
\setsansfont{Arial}
\setmonofont{Consolas}
\usepackage{amsmath,amssymb,graphicx,booktabs,longtable,array,calc,multirow}
\usepackage{hyperref,xurl,float,caption,fancyvrb,seqsplit}
\hypersetup{hidelinks}
\usepackage{fvextra}
\DefineVerbatimEnvironment{Highlighting}{Verbatim}{breaklines,commandchars=\\\{\}}
\newcommand{\tightlist}{\setlength{\itemsep}{0pt}\setlength{\parskip}{0pt}}
\setcounter{tocdepth}{1}
\setlength{\emergencystretch}{4em}
\setlength{\parindent}{2em}
\setlength{\parskip}{0.25em}
\renewcommand{\arraystretch}{1.15}
\captionsetup{font=small}
\pagestyle{plain}
\makeatletter
\def\maxwidth{\ifdim\Gin@nat@width>\linewidth\linewidth\else\Gin@nat@width\fi}
\def\maxheight{\ifdim\Gin@nat@height>0.78\textheight0.78\textheight\else\Gin@nat@height\fi}
\makeatother
\setkeys{Gin}{width=\maxwidth,height=\maxheight,keepaspectratio}
\begin{document}
'''
 abstract=(P/'rebuild_abstract.txt').read_text(encoding='utf-8');abstract=abstract.replace('&',r'\&').replace('%',r'\%').replace(
↪  '_',r'\_');tex=header+r'''\pagenumbering{gobble}
 \begin{center}{\zihao{3}\bfseries 观测语义约束下的啤酒风味预测、化合物筛选与条件生成}\end{center}
 \begin{center}\textbf{摘要}\end{center}
 '''+abstract+r'''

 \noindent\textbf{关键词：} 面板评分；训练内选择；替代组稳定性；条件生成；独立风险

 \vfill\noindent\small AI 辅助研究参考稿。核心建模与分析尚待参赛队独立完成，不作为独立原创正式提交稿。
 \newpage\normalsize\tableofcontents\newpage
 \pagenumbering{arabic}
 \input{build_rebuild/body.tex}
 \clearpage\appendix
 \input{build_rebuild/appendix.tex}
 \section{复现环境与全部新增程序}
 以下为本轮新增计算程序，均含AI辅助说明。原始数据未修改。运行命令：Q2/Q3 封闭实验与论文重生成见 scripts/run\_final.ps1
↪ （scripts/run\_rebuild.ps1 为其兼容转发入口）；Q1 从源码完整重建见 scripts/run\_q1.ps1。脚本依赖见
↪  configs/rebuild\_requirements.txt。
 '''
 # Official rules require the appendix to contain every program actually used. List every src/rebuild
 # module plus every script referenced from the text, including the Q1-only entry point run_q1.ps1.
```

```
for p in sorted((R/'src/rebuild').glob('*.py'))+[R/'scripts'/n for n in
↪  ['run_final.ps1','run_q1.ps1','run_rebuild.ps1','build_rebuild_pdf.py','final_quality.py','package_final.py']]:
 tex+='\n\\subsection{'+p.name.replace('_',r'\_')+'}\n'+r'\VerbatimInput[fontsize=\tiny,breaklines,breakanywhere]{../'+str(p.rel
 ↪  ative_to(R)).replace(chr(92),'/')+'}\n'
tex+='\n'+r'\end{document}';(P/'competition_draft.tex').write_text(tex,encoding='utf-8')
for _ in range(2):run([xelatex,'-interaction=nonstopmode','-halt-on-error','-output-directory=build_rebuild','competition_draft.
↪  tex'],P)
(P/'competition_draft.pdf').write_bytes((B/'competition_draft.pdf').read_bytes());(P/'competition_final.pdf').write_bytes((B/'co
↪  mpetition_draft.pdf').read_bytes())
# Explicit current-run AI details, rather than outdated prior PDF.
ai=r'''\section*{AI 工具使用详情说明}
工具：OpenAI Codex，历史 GPT-6 系列及本轮 gpt-6.1-sol，OpenAI，2026 年 10 月 28 日。

使用目的与环节：本轮数据与代码审计、逐评分员目标重建、基线及结构模型编程、CPU实际实验、统计与消融、生成及迁移分析、图表、中文报告和排版。
↪  原始文件未覆盖。

关键交互：用户要求接手既有三问工程，对现有三问独立审计、冻结协议并实际运行，关注重复评分员、共线冗余、风格混杂、最低值堆积和独立评价器。
↪  AI据此实施 configs/rebuild\_protocol.json 中的实验；运行中修正深度模型重训样本不等与随机投影不共用两处实现问题，未扩大搜索模型范围。
↪  本轮仅执行Q2统一、22风格生成评价、9源酒四策略迁移三组封闭补充，之后停止建模探索；Q1不重训、不改预测，
↪  仅在2026-10-28修正评分卡交付的字段过滤缺陷（30字段→3.xlsx summary全部50字段），记录见 outputs/rebuild/audit/protocol\_amendment.md。

实际产出：src/rebuild 的程序、outputs/rebuild 的CSV及训练日志、reports/00至05的分析和 paper/competition\_draft 的参考论文。历史模型、
↪  网页端稿件和MC2026A被当作参考，未预设其正确。

采纳与人工修改记录：当前尚无可核实的参赛队人工修改或独立核心建模记录，outputs/rebuild/audit/human\_decision\_record.csv 明确标为待完成。
↪  不得把本轮AI主导研究描述为队员独立完成。正式参赛前应由队员独立完成核心建模、算法设计、求解与分析，并记录实际采纳、修改和弃用内容。
↪  审阅签字不能替代该要求。

证据追溯：原始哈希、协议、训练日志、结果CSV、代码与PDF检查记录均保留在项目内。关键结论以真实输出为准；
↪  不成功的深度模型和不能验证的物理缺陷风险均如实报告。

详细历史交互摘要见 reports/AI\_usage\_log.md；该说明不伪造完整聊天导出，也不伪造队员审核意见。
'''
(P/'ai_details.tex').write_text(header+ai+r'\end{document}',encoding='utf-8');run([xelatex,'-interaction=nonstopmode','-halt-on-
↪  error','-output-directory=build_rebuild','ai_details.tex'],P);(R/'reports/AI工具使用详情说明.pdf').write_bytes((B/'ai_details.
↪  pdf').read_bytes());print('PDFs built',flush=True)
```

E.24 final_quality.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Actual PDF and rule inspection."""
from pathlib import Path
from pypdf import PdfReader
import json,hashlib
R=Path(__file__).resolve().parents[1];p=R/'paper/competition_final.pdf';pdf=PdfReader(p);texts=[v.extract_text() or '' for v in
↪  pdf.pages];app=next(i for i,t in enumerate(texts) if t.startswith('A ') and '附录' in t);log=(R/'paper/build_rebuild/competit
↪  ion_draft.log').read_text(encoding='utf-8',errors='replace');d={'pdf_sha256':hashlib.sha256(p.read_bytes()).hexdigest(),'tot
↪  al_pages':len(texts),'front_matter_pages':2,'body_pages':app-2,'within_30':app-2<=30,'abstract_first':'摘要' in
↪  texts[0],'contents_second':texts[1].startswith('目录'),'appendix_first_pdf_page':app+1,'missing_glyphs':'Missing character' in
↪  log,'overfull':'Overfull' in log,'embedded_fonts_checked':'Use pdffonts report; visual QA recorded
↪  separately','official_rules':'Local official PDFs, 2026 revision; no unverified online policy substituted'};(R/'outputs/rebu
↪  ild/closed/pdf_quality.json').write_text(json.dumps(d,ensure_ascii=False,indent=2),encoding='utf-8');print(d);assert
↪  d['within_30'] and d['abstract_first'] and d['contents_second'] and not d['missing_glyphs']
```

E.25 package_final.py

```
"""AI-assisted OpenAI Codex gpt-6.1-sol, 2026-10-28. Final support and submission preparation."""
from pathlib import Path
import json,hashlib,os,zipfile,shutil
R=Path(__file__).resolve().parents[1];S=R/'submission';S.mkdir(exist_ok=True)
import time
def drop(p):
 for i in range(8):
  try:
   if p.exists():p.unlink()
   return True
  except PermissionError:
   if i==7:return False
```

```
   time.sleep(0.8)
def same(src,dst):
 try:return dst.exists() and hashlib.sha256(src.read_bytes()).hexdigest()==hashlib.sha256(dst.read_bytes()).hexdigest()
 except OSError:return False
def copy(src,dst):
 """Copy src over dst. If dst is byte-identical, leave it alone (avoids touching a file a PDF
 viewer has open). Overwriting in place is tried even when unlink fails, because a locked
 destination can often still be rewritten truncating it; a numbered sibling is the last resort."""
 if same(src,dst):
  print('  unchanged',dst.name);return dst
 drop(dst)
 try:
  shutil.copy2(src,dst);return dst
 except PermissionError:
  pass
 for k in range(2,10):
  alt=dst.with_name(dst.stem+'_%d'%k+dst.suffix)
  if same(src,alt):print('  unchanged',alt.name);return alt
  drop(alt)
  try:
   shutil.copy2(src,alt);print('  LOCKED ->',alt.name);return alt
  except PermissionError:
   continue
 raise PermissionError('cannot place '+str(dst))
def write(dst,text):
 for i in range(8):
  try:dst.write_text(text,encoding='utf-8');return
  except PermissionError:
   if i==7:raise
   time.sleep(0.8)
TEAM=os.environ.get('MC_TEAM_ID','').strip().upper()
def official(name):return (TEAM+name) if TEAM else name
copy(R/'paper/competition_final.pdf',S/official('.pdf') if TEAM else S/'论文定稿 _ 待填队号.pdf')
copy(R/'reports/AI 工具使用详情说明.pdf',S/'AI 工具使用详情说明.pdf')
letters=sorted([p for p in R.rglob('*承诺书*.docx') if S not in p.parents]+[p for p in R.parent.rglob('*承诺书*.docx') if
↪  'MathorCup_A_final_support' not in p.parts])
if letters:
 copy(letters[0],S/(official('承诺书')+'.docx' if TEAM else '承诺书 _ 官方空白模板 _ 待真实签署.docx'))
readiness='''# 提交材料核查

## 一、技术状态

三组封闭实验、Q1 全字段评分卡、论文 PDF、图表、源程序、结果 CSV、复现协议与 AI 使用详情 PDF 均已生成。版式以
↪  outputs/rebuild/closed/pdf_quality.json 为唯一记录（正文页数未超 30 页、摘要第 1 页、目录第 2 页、无页眉、无身份信息，
↪  具体页数一律查该文件，本说明不另记以免版本漂移）。内容一致性由 `python src/rebuild/closed_verify.py` 输出 Closed checks 101/101 保证。
↪

Q1 评分卡按 3.xlsx summary 的 50 个字段定义交付：53 列 = 1 列 beer + 50 个由感官监督模型预测的 0—6 字段 + 2 个 acetaldehyde
↪  阈值锚定推断字段（当前均为 0，属非面板评分），250 行、无缺失值。此前只交 30 列是列名前缀过滤造成的实现缺陷，不涉及重训；
↪  修复与乙醛判定依据见 outputs/rebuild/audit/protocol_amendment.md 与 outputs/rebuild/q1_complete/acetaldehyde_grading.json。

## 二、上传前必须由队伍完成的事项

1. **真实队号**。官方规范要求论文文件名为「#*******.pdf」（# 为题号，******* 为参赛队号，如 A2600001.pdf）。本包不能虚构队号，当前为
↪  `论文定稿_待填队号.pdf`。确定队号后请设置环境变量 `MC_TEAM_ID`（例如 `A2600001`）后重跑 `python scripts/package_final.py`，
↪  脚本会自动生成 `A2600001.pdf`、`A2600001承诺书.docx` 与 `A2600001附件.zip`。
2. **承诺书签署**。本包内为官方空白模板，须每位队员手写或电子签名，再扫描/拍照为 PDF 或图片，命名「#*******承诺书」（如 A2600001承诺书）。
↪  不得代签。
3. **队员独立核心工作记录**。现有核心方案与分析由 AI 主导生成，尚无队员独立复算、人工修改与采纳记录。官方《人工智能工具使用规定》
↪  第二条要求核心建模、算法设计、求解与分析由队员独立完成，审阅签字不能替代。具体操作清单见随包 `队员独立复核指引.md`（三人分工、
↪  每人要读的三处代码、要动手改的一处、要核对的数字），完成后把真实修改与采纳写入 reports/AI_usage_log.md 与
↪  outputs/rebuild/audit/human_decision_record.csv，再决定正式提交版本。
4. **支撑材料命名**。官方要求「#*******附件.rar」或「#*******附件.zip」。本包内 支撑材料.zip 即该压缩包。按官方规范，赛题原始数据可不放入，
↪  故 zip 内不含 data/raw；但论文作者公开的逐评分员感官数据是模型必需输入，已随包放在 data/supplementary/（cp1252 编码，MD5
↪  a8f911d8c0061e096abf3b087678e164），属可再分发数据。
5. **AI 声明位置 **。论文正文参考文献后已列 AI 使用声明，支撑材料内已含《AI 工具使用详情说明》(PDF)，符合规定第三、五条。

## 三、不得在提交时改动的科学边界

- 自动选法未超过固定 2+1；条件生成指标随指标改变。
```

- 迁移约束按两套口径报告：赛题定义的超阈值口径下，越阈次数由无约束 7 次降到点约束 0 次、不确定性约束 0 次；
↪ 独立评价器任意微量上升的严格口径下仍有 6/9、5/9、4/9 款酒上升，该残余风险已作为未解决局限写入正文。两种口径差 0.05 分以内，远小于模型在
↪ stale_hops 上的约 0.23 MAE。
- Q1 交付评分卡存在已披露的等级压缩局限：预测等级全部落在 0—4 级，真值 ≥5 的 121 个单元被平均低估 2.07，连续预测上限仅 4.34，故整体 MAE
↪ 可用而高强度尾部不可用，见 outputs/rebuild/q1_complete/grade_range_summary.json。
- 观测编码层面的浓度不增加不等于地板以下真实浓度保证；没有真实新酿造与盲评验证。

本文件夹为提交准备材料，未上传竞赛系统。论文与 AI 声明保留真实参与事实。

四、本压缩包内含什么、不含什么

为避免" 说明里提到的文件其实不在包里" 这类误会，这里逐项写清：

- **包内含**：`src/` 全部源程序、`outputs/rebuild` 全部结果 CSV 与图表、`reports/`、`paper/`（含定稿 PDF）、`configs/`、`scripts/`、
↪ `data/processed/`、`data/supplementary/Raw sensory data anonymized.csv`（随包必要的补充数据）、本说明文件与
↪ `AI工具使用详情说明.pdf`。
- **包内不含**：赛题官方附件（按规范可不放入，需由使用者自行下载后放入 `data/raw/`）、`outputs/rebuild/closed/delivery_manifest.json`
↪ （包自身的清单，避免自引用）。
- **不随包分发、需在本地生成或单独处理**：`submission/` 下的论文 PDF 副本、承诺书、支撑材料 zip 本身（这些是"提交物"，不应该嵌套进自己），
↪ 以及填好后的承诺书扫描件。`

上面第 1–3 条完成后，用 `MC_TEAM_ID` 重新运行 `python scripts/package_final.py` 即可按官方命名生成这三份提交物。
```
''';(write(S/'提交说明与待补项.md',readiness))
paths=[]
for folder in ['src/rebuild','outputs/rebuild']:
 paths += [p for p in (R/folder).rglob('*') if p.is_file() and '__pycache__' not in p.parts and p.name not in
 ↪ ['delivery_manifest.json']]
paths += [R/'README_REBUILD.md',R/'队员独立复核指引.md',R/'configs/rebuild_protocol.json',R/'configs/closed_final_protocol.json',R
↪ /'configs/rebuild_requirements.txt']
paths += [R/'scripts'/n for n in
↪ ['run_final.ps1','run_q1.ps1','run_rebuild.ps1','build_rebuild_pdf.py','final_quality.py','package_final.py']]
paths += list((R/'reports').glob('0[0-5]_*.md'))+[R/'reports/08_closed_experiments.md',R/'reports/AI_usage_log.md',R/'reports/AI
↪ 工具使用详情说明.pdf']
# paper/: competition_draft.md is the canonical single source (abstract+body+refs+declaration);
# rebuild_body.md / rebuild_abstract.txt are slices of it, and competition_draft.tex is generated FROM
# them, so shipping all of them is consistent. The PDF is the deliverable the tex produces.
paths += [R/'paper'/n for n in ['competition_draft.md','competition_draft.tex','competition_final.pdf','rebuild_body.md','rebuil
↪ d_abstract.txt','rebuild_appendix.md']]
paths += [R/'paper/PAPER_VERSION.md']
paths += [R/'data/processed'/n for n in
↪ ['chemical_names.csv','transforms.csv','thresholds.csv','template_hierarchy.csv','scale_pair_validation.csv']]
# The author's public per-taster file is redistribution-safe and required by run.py, so it ships.
paths += [p for p in (R/'data/supplementary').glob('*') if p.is_file()]
paths += [p for p in (R/'outputs/review_20261028').glob('*') if p.is_file()]
paths += [S/'提交说明与待补项.md',S/'AI 工具使用详情说明.pdf']
files=sorted(set(p for p in paths if p.is_file()));manifest=[{'path':str(p.relative_to(R)),'bytes':p.stat().st_size,'sha256':has
↪ hlib.sha256(p.read_bytes()).hexdigest()} for p in
↪ files];mp=R/'outputs/rebuild/closed/delivery_manifest.json';mp.write_text(json.dumps(manifest,ensure_ascii=False,indent=2),e
↪ ncoding='utf-8');files.append(mp)
z=S/(official('附件')+'.zip' if TEAM else '支撑材料.zip')
legacy=S/'支撑材料.zip'
def drop(p):
 for i in range(6):
  try:
   if p.exists():p.unlink()
   return
  except PermissionError:
   if i==5:raise
   time.sleep(0.6)
for p in {z,legacy,R/'outputs/MathorCup_A_final_support.zip'}:drop(p)
with zipfile.ZipFile(z,'w',zipfile.ZIP_DEFLATED,compresslevel=6) as f:
 for p in files:f.write(p,p.relative_to(R))
with zipfile.ZipFile(z) as f:assert f.testzip() is None
if z!=legacy:copy(z,legacy)
status={'team_id':TEAM or None,'files':len(files),'zip':z.name,'zip_bytes':z.stat().st_size,'support_sha256':hashlib.sha256(z.re
↪ ad_bytes()).hexdigest(),'formal_submission_readiness':'pending real team ID, signatures, independently completed core
↪ work','uploaded':False};(S/'submission_manifest.json').write_text(json.dumps(status,ensure_ascii=False,indent=2),encoding='u
↪ tf-8');copy(z,R/'outputs/MathorCup_A_final_support.zip');print(status)
```